Robotics Research Lab
Department of Computer Science
University of Kaiserslautern-Landau

Master Thesis

Learning-Based Online Adaptation for Control of a Hydraulic Manipulator

Lernbasierte Online-Adaption zur Regelung eines hydraulischen Manipulators

Kevin Kudakwashe Murera

Friday 18th September, 2026

Master Thesis

Learning-Based Online Adaptation for Control of a Hydraulic Manipulator

Lernbasierte Online-Adaption zur Regelung eines hydraulischen Manipulators

Robotics Research Lab
Department of Computer Science
University of Kaiserslautern-Landau

Kevin Kudakwashe Murera

Day of issue : 23 March 2026
Day of release : Friday 18th September, 2026

Reviewer : Prof. Dr. Karsten Berns
Supervisor : M.Sc. Qi Li

Hereby I declare that I have self-dependently composed the Master Thesis at hand. The sources and additives used have been marked in the text and are exhaustively given in the bibliography.

Friday 18th September, 2026 – Kaiserslautern

(Kevin Kudakwashe Murera)

Acknowledgements

My thanks go first to my supervisor, **Mr. Qi Li**, who followed this work from the first experiment to the last, and whose questions more than once sent me back to check something I had assumed.

I thank **Professor Dr. Karsten Berns** for the resources of the Robotics Research Lab. The simulation campaigns reported here ran for months on that hardware, and the work would look very different without it.

To my family, for backing a long project largely on trust, and to my friends, who heard a great deal about hydraulic valves and stayed anyway: thank you.

I am grateful to everyone else who helped along the way.

Abstract

Hydraulic manipulators are difficult to control accurately because actuator response is nonlinear and changes with load, supply pressure and machine condition. High-fidelity hydraulic simulation captures these effects but is too slow for large-scale reinforcement learning, whereas direct joint control omits the actuator behaviour that governs deployment performance. This thesis investigates how learned hydraulic dynamics can be transferred into efficient robot simulation, used for reinforcement-learning control and extended with online adaptation on a verge-mowing manipulator.

Alternative actuator representations are identified from Simscape data: a modular inverse–forward chain and a structurally constrained, command-conditioned reduced-order model. Open-loop excitation and command ablation test whether each model responds to its command input, since low prediction error alone can conceal a model that merely extrapolates smooth state trajectories. The models are embedded in GPU-parallel PhysX training. A classical controller supplies privileged demonstrations during training, then a distilled policy replaces it completely at deployment. Evaluation covers held-out meshes, varied trajectory families and reserved plant configurations.

The actuator representations support closed-loop control but produce different trade-offs. The inverse–forward chain has lower median lateral error, whereas the reduced-order model has lower median cut-height error; neither mean difference is resolved. Online adaptation extends a frozen direct-valve actor with context columns driven by a payload estimate inferred from the task observation already available to the controller. Under heavy payloads, where the frozen policy loses ground contact, adaptation improves cut-height error, lateral error and ground contact together. The pooled confidence intervals support each improvement, and the effects repeat on an independently reserved mesh. Under a moderate payload that the frozen controller still handles, conditioning instead worsens lateral accuracy. Supplying the true payload outperforms a permuted payload, although plausible but mispaired loads recover most of the gain. The benefit therefore combines conditioning on a plausible load with information from the correct one, and is limited to the regime in which the frozen controller has a control-relevant deficit.

Overall, the results support the use of learned hydraulic models for policy-based trajectory tracking in simulation. Online payload adaptation improves control under heavy loads, but does not fully restore the required cut-height accuracy. Transfer to the physical machine remains to be validated.

Kurzfassung

Hydraulische Manipulatoren sind schwer präzise zu regeln, weil das Aktorverhalten nichtlinear ist und sich mit Last, Versorgungsdruck und Maschinenzustand ändert. Eine detaillierte Hydrauliksimulation bildet diese Effekte ab, ist für umfangreiches Reinforcement Learning aber zu langsam, während eine direkte Gelenkregelung gerade das Aktorverhalten ausblendet, das die Leistung im Einsatz bestimmt. Diese Arbeit untersucht, wie gelernte hydraulische Dynamik in eine effiziente Robotersimulation übertragen, für eine Reinforcement-Learning-Regelung genutzt und an einem Manipulator zum Bankettmähen um eine Online-Adaption erweitert werden kann.

Aus Simscape-Daten werden alternative Aktorrepräsentationen identifiziert: eine modulare Kette aus inversem und Vorwärtsmodell sowie ein strukturell beschränktes, kommandokonditioniertes Modell reduzierter Ordnung. Anregung ohne Rückführung und Ablation des Kommandos prüfen, ob jedes Modell auf seine Kommandogröße reagiert, da ein geringer Prädiktionsfehler allein ein Modell verbergen kann, das nur glatte Zustandsverläufe extrapoliert. Die Modelle werden in ein GPU-paralleles PhysX-Training eingebettet. Ein klassischer Regler liefert im Training privilegierte Demonstrationen; im Einsatz ersetzt ihn eine destillierte Policy vollständig. Die Bewertung umfasst zurückgehaltene Geländenetze, verschiedene Trajektorienfamilien und reservierte Regelstreckenkonfigurationen.

Beide Aktorrepräsentationen ermöglichen eine Regelung im geschlossenen Kreis, führen aber zu unterschiedlichen Kompromissen. Die Kette aus inversem und Vorwärtsmodell weist den geringeren Median des Querfehlers auf, das Modell reduzierter Ordnung jenen des Schnitthöhenfehlers; keine der beiden Mittelwertdifferenzen ist statistisch gesichert. Die Online-Adaption erweitert eine eingefrorene, das Ventil direkt ansteuernde Policy um Kontextspalten, die mit einer Nutzlastschätzung aus der bereits verfügbaren Aufgabenbeobachtung gespeist werden. Bei hohen Nutzlasten, bei denen die eingefrorene Policy den Bodenkontakt verliert, verbessert die Adaption Schnitthöhen- und Querfehler sowie den Bodenkontakt gemeinsam. Die gepoolten Konfidenzintervalle stützen jede Verbesserung, und die Effekte wiederholen sich auf einem unabhängig reservierten Geländenetz. Bei einer mittleren Nutzlast, die der eingefrorene Regler noch bewältigt, verschlechtert die Konditionierung dagegen die Quergenauigkeit. Die Verwendung der wahren Nutzlast liefert bessere Ergebnisse als die einer permutierten Nutzlast, wobei plausible, aber falsch zugeordnete Lasten den größten Teil der Verbesserung erzielen. Der Nutzen beruht somit sowohl auf der Konditionierung auf eine plausible Last als auch auf der Information über die tatsächliche Last und ist auf den Bereich beschränkt, in dem der eingefrorene Regler ein regelungsrelevantes Defizit aufweist.

Insgesamt stützen die Ergebnisse den Einsatz gelernter hydraulischer Modelle für eine policybasierte Trajektorienverfolgung in der Simulation. Die Online-Adaption an die Nutzlast verbessert die Regelung unter hoher Last, stellt die geforderte Schnitthöhengenauigkeit aber nicht vollständig wieder her. Die Validierung der Übertragung auf die reale Maschine steht noch aus.

Contents

1 Introduction **1**

1.1 The Role of Mechanised Verge Maintenance 1
1.2 Learning-Based Control for Hydraulic Manipulators 1
1.3 Challenges of Precise Tracking with Hydraulic Mowing Arms 2
1.4 Contribution of This Thesis 3
1.5 Thesis Structure 7

2 Fundamentals and Related Work **10**

2.1 Manipulator Actuation and Control Fundamentals 10
2.1.1 Application Context and Implement Carriers 10
2.1.2 The Electro-Hydraulic Actuation Chain 11
2.1.3 High-Fidelity Simulation as a Data Source 11
2.2 Kinematics and Task-Space Tracking 12
2.2.1 Differential Kinematics 12
2.2.2 From Joint Space to Cylinder Space 12
2.3 Model-Based and Conventional Control Approaches 13
2.3.1 Fixed-Gain Cascades 13
2.3.2 Nonlinear Cancellation 13
2.3.3 Online Parameter Estimation 13
2.3.4 Retained Elements 14
2.4 Data-Driven System Identification and Learned Dynamics 15
2.4.1 Learned Forward and Inverse Dynamics 15
2.4.2 Closed-Loop Identifiability 15
2.5 Reduced-Order Modelling 15
2.6 Domain Randomisation and Latent Adaptation 15
2.6.1 Robustness Through Randomised Training 15
2.6.2 From Robustness to Adaptation 16
2.7 Reinforcement Learning and Structured Priors 17
2.7.1 Policy Optimisation 17
2.7.2 Structured Priors for Policy Learning 17
2.7.3 Reinforcement Learning on Hydraulic Machines 18
2.8 Research Gap and Thesis Positioning 19

3 System Fundamentals and Problem Formulation 22
3.1 Experimental Task and Engineering Motivation . . . 22
3.1.1 Experimental Task and Research Purpose . . . 22
3.1.2 Why Simscape Supplies the Identification Data . . . 23
3.1.3 A Sim-to-Sim Test of Transfer . . . 23
3.1.4 Why Online Adaptation Is Tested . . . 23
3.2 System Architecture . . . 23
3.3 Manipulator Platform . . . 24
3.4 Kinematic Structure and Coordinate Mappings . . . 27
3.4.1 Joint Space and Cylinder Space . . . 27
3.4.2 Task Space and the Calibrated Cutter Jacobian . . . 27
3.5 Actuation and Measured State . . . 28
3.5.1 Sensing and Observability . . . 28
3.6 High-Fidelity Simscape Model . . . 28
3.6.1 Mechanical Subsystem . . . 28
3.6.2 Hydraulic Subsystem . . . 29
3.6.3 Scope of the Source Model . . . 30
3.6.4 Source-Model Checks . . . 31
3.7 IsaacLab Environment . . . 32
3.7.1 The Deployment Plant . . . 32
3.7.2 Simulation and Control Rates . . . 33
3.7.3 Environment Configuration Details . . . 33
3.8 Experimental Task Specification . . . 33
3.8.1 Task Definition . . . 33
3.8.2 Task Feasibility and Plant Capability . . . 33
3.9 Control Architecture . . . 34
3.10 Problem Formulation . . . 35
3.11 System-Level Evaluation Criteria . . . 35
3.11.1 The Eight System-Level Evaluation Gates . . . 35
3.11.2 Cylinder and Model Checks . . . 36

4 Learned Models and Online Adaptation 38
4.1 Modelling Overview and Common Notation . . . 38
4.2 Inverse Dynamics Model . . . 40
4.2.1 Paired Control Fine-Tuning . . . 41
4.2.2 Inverse Model and Paired Control Response . . . 41
4.2.3 Transfer of the Frozen Inverse Model . . . 41
4.3 Forward Dynamics Model . . . 42
4.3.1 Training Objective . . . 43
4.3.2 Forward-Model Command Sensitivity . . . 43
4.4 Reduced-Order Model . . . 44
4.4.1 Closed-Loop Calibration Against the Deployment Plant . . . 45
4.4.2 ROM Accuracy and Physical Interpretation . . . 45
4.5 Offline AAM Diagnostic for the ID Path . . . 47
4.6 The Deployed Online Payload Estimator . . . 48

5 Control Framework and Simulation Integration 50
5.1 The DLS Teacher and Its Two Roles . . . 50
5.1.1 Scope and Role of the Controller . . . 50
5.1.2 Privileged Information . . . 50
5.1.3 From Task Error to Cylinder Command . . . 51
5.1.4 Execution Through the Student Interface . . . 52
5.1.5 Teacher Performance . . . 52
5.2 Policy Formulation . . . 52
5.2.1 Policy Definition . . . 52
5.2.2 Observation Space . . . 53
5.2.3 Action Space and Bounds . . . 53
5.2.4 The Reward Function . . . 55
5.2.5 Optimisation . . . 56
5.3 Policy Training Protocol . . . 57
5.3.1 Distillation by DAgger . . . 57
5.3.2 Objective Mismatch in Unconstrained Fine-Tuning . . . 57
5.3.3 Action Limits . . . 59
5.4 Reference-Anchored PPO . . . 59
5.4.1 Critic Warm-Up . . . 59
5.4.2 The Reference Anchor . . . 59
5.4.3 Trust Regions and Rollback . . . 60
5.4.4 The Block Ratchet . . . 60
5.4.5 Scope of the Constraint . . . 61
5.5 IF: The Transferred Actuator . . . 61
5.6 R: The Reference Condition . . . 62
5.7 AP: Adaptation on the Policy . . . 62
5.7.1 Why AP Does Not Use the Inverse Model . . . 63
5.7.2 Conditioning on the Payload Estimate . . . 64
5.7.3 Freezing the Base Policy . . . 65
5.7.4 Training of the Context Columns . . . 65
5.7.5 Consistency With the Plant Dynamics . . . 65
5.7.6 Sufficiency of the Forward Model . . . 66
5.8 IsaacLab Integration and Timing . . . 66
5.9 Constraints and Fair-Comparison Rules . . . 66
5.9.1 Limits and Safeguards . . . 66
5.9.2 Matching Rules and Comparison Controls . . . 66

6 Data Generation and Preparation 69
6.1 Dataset Objectives and Design Requirements . . . 69
6.2 Randomised Parameter Space of the Simscape Campaign . . . 71
6.3 Closed-Loop Data Collection . . . 72
6.3.1 The Controller as a Data-Collection Instrument . . . 72
6.4 Open-Loop Excitation Experiments . . . 73
6.4.1 Limitations of the Closed-Loop Batch . . . 73
6.4.2 Design of the Excitation Signal . . . 74
6.4.3 Plant Draws and Acceptance . . . 74
6.5 Combining Closed-Loop and Open-Loop Datasets . . . 74
6.6 Closed-Loop Calibration Data for the Command ROM . . . 75
6.7 The IsaacLab Transfer Collection . . . 76
6.7.1 The Physical Factor Vector . . . 76
6.7.2 Domains and Factor Design . . . 76
6.7.3 Collection Procedure: The Task-Controller Runs . . . 77
6.7.4 The Frozen Dataset . . . 77
6.7.5 Learning Timebase and Hydraulic State . . . 77
6.8 Feature and Target Construction . . . 77
6.8.1 The Actuator Models . . . 77
6.8.2 On-Policy AP Adaptation Rollouts . . . 79
6.9 Preprocessing and Data Splits . . . 80
6.10 Dataset Auditing . . . 80

7 Experimental Evaluation: Representation, Robustness, and the Value of Adaptation 82
7.1 Experimental Protocol . . . 82
7.1.1 Test Conditions and Comparisons . . . 82
7.1.2 Comparative Scope and Scoring . . . 83
7.1.3 Scoring Horizon . . . 84
7.2 Closed-Loop Task Performance . . . 84
7.3 Actuator Representation and Dynamic Response . . . 86
7.3.1 Plant Delay and Step Response . . . 87
7.3.2 Control Effort . . . 88
7.4 Mechanical Robustness and the Authority Margin . . . 89
7.4.1 Actuator Authority . . . 90
7.4.2 The Margin Left for Adaptation . . . 92
7.5 Evaluation of Online Adaptation . . . 92
7.5.1 Experiment and Measurement . . . 92
7.5.2 Resolution of the Instrument . . . 93
7.5.3 The Frozen Controller Has a Payload-Dependent Deficit . . . 93
7.5.4 Adaptation Against the Frozen Policy . . . 94
7.5.5 Repetition on a Second Held-Out Mesh . . . 95
7.5.6 Seed Dependence and Its Source . . . 96
7.5.7 The True Payload Is Not an Upper Bound . . . 96
7.5.8 Correct Payload Against Wrong Payload . . . 97
7.5.9 Scope of the Claims . . . 97
7.6 Conditions for Effective Online Adaptation . . . 98

8 Conclusion and Future Work **100**
8.1 Summary of Findings 100
8.2 Future Work 102

A Reference Data and Reproducibility Record **105**
A.1 Plant Ranges and Frozen Collections 105
A.2 Dataset Audit Values 106
A.3 Timing Contract 106
A.4 Frozen Artifacts 107
A.5 Reproduction Entry Points 107

Bibliography **109**

1. Introduction

1.1 The Role of Mechanised Verge Maintenance

Road verges are cut because they are infrastructure. Vegetation left to grow closes sight lines at junctions and bends, obscures signs and barriers, blocks the drainage channels that keep water off the carriageway, and encroaches on the running surface itself. A national road network carries tens of thousands of kilometres of verge, and each kilometre is cut several times in a growing season, so the work is continuous, repetitive and performed almost entirely by machine.

It is also performed from a moving vehicle. Stopping to cut each section would require closing the lane, so a carrier drives the road at a few kilometres per hour while a boom-mounted mowing head works the verge alongside it. The head must hold a cut height of a few centimetres over ground that rises and falls continuously, follow a grass line that curves with the road, and do so without striking the surface hard enough to damage the tool or lift the machine.

The demands on the manipulator follow from that. The machine studied here, shown in Figure 1.1, carries a heavy rotating tool at the end of a long boom, works in intermittent contact with the ground, and must place that tool accurately while its own base is in motion. Hydraulic actuation is the standard answer at this scale: it supplies the force density and the tolerance to shock loading that electric drives do not, and it does so with components rugged enough for continuous outdoor operation. Precision, however, is not what hydraulics are good at, and that tension is the subject of this thesis.

1.2 Learning-Based Control for Hydraulic Manipulators

Two developments have changed what is achievable on machines of this kind.

The first is that policy optimisation in simulation has become practical for contact-rich manipulation. GPU-parallel physics engines run thousands of environments simultaneously, which makes reinforcement learning feasible on tasks that would take years of real-machine

experience, and randomising the simulator across a family of plants produces policies that transfer without being tuned to one machine. The second is that the actuator itself can be learned. Rather than deriving valve and cylinder behaviour from first principles and fitting its parameters, a network trained on recorded motion can represent the command-to-motion relationship directly, including the nonlinearities that analytical models approximate away. Chapter 2 sets out both lines and the work they rest on.

Combining the two is attractive and not straightforward. A learned actuator model must be cheap enough to evaluate inside the training loop, which rules out the high-fidelity models used for offline analysis, and it must remain valid across the family of machines the policy will meet. This has motivated a line of work in which a compact code describing the current plant is inferred from recent interaction history and supplied to the controller, so that a single policy can serve a family rather than a single machine. Such methods report substantial gains in legged locomotion and in manipulation, and they are the direct antecedent of the architecture examined here.

Figure 1.1: MULAG FME 500 front mower on a Unimog during roadside verge maintenance. Image taken from the FME 500 product documentation [MULAG Fahrzeugwerk 26].

1.3 Challenges of Precise Tracking with Hydraulic Mowing Arms

A valve command does not produce a proportional cylinder motion. The relationship is nonlinear, depends on load and supply pressure, and is subject to dead time between the command and any measurable response. It also differs between machines: seal condition, fluid temperature, hose compliance and the attachment fitted all change it, so two machines of the same model respond differently and one machine responds differently across a season. A controller that inverts only the geometry therefore leaves the largest source of tracking error unaddressed.

That difficulty admits two responses, and they are routinely conflated. The first is robustness: train across a randomised family so that a single fixed model is adequate everywhere. It costs nothing at run time and gives up the difference between the family and the individual. The second is adaptation: infer which member of the family is present from the interaction history and condition the model or the policy on that estimate. It promises to recover that difference, at the cost of an estimator that must work from observable signals alone, since a deployed machine has no access to its own plant parameters.

Whether adaptation repays that cost depends on three things. Plant identity must be present in the signals produced under the available excitation. The robust controller must leave a control-relevant deficit to recover. The correction path must retain enough authority to change the realised command. This thesis measures these conditions on a hydraulic mowing arm and then evaluates whether an online estimate of the operating condition improves closed-loop tracking.

1.4 Contribution of This Thesis

The difficulty described in Section 1.3 is a modelling problem before it is a control problem. A hydraulic actuator can be simulated accurately, but the models that do so are too slow to sit inside a reinforcement-learning loop requiring millions of environment steps, while the usual alternative of commanding joint positions directly removes the load and pressure dependence that makes the machine difficult to control. What is needed is a representation cheap enough to train against and faithful enough that the resulting policy controls something close to the real actuator.

One constraint shapes how that representation can be obtained and how it can be tested. The machine was not available during this work, and no measurement from it enters this thesis. The actuator therefore has to be identified from a high-fidelity model, not from hardware, and whether such a model transfers cannot be settled on the machine it is meant to predict. Both halves are constructed instead. Simscape supplies the data a real machine would have supplied, and a second simulator supplies a plant the identified models have never seen, so that transfer is measured across a gap, not assumed. Section 3.1.3 gives that construction and the limits that come with it.

Learned actuator models and payload-conditioned adaptation are both established techniques. What remains unclear is how such models should be identified so that they retain the command dependence a controller needs, whether they transfer between simulators without retraining, and what a controller must know about its load once the models are used in control. Representation and transfer are established first because the adaptation acts through an identified actuator model, not replacing one. Online adaptation is evaluated last so that any remaining gain can be interpreted against the transferred model and the robustness of the fixed controller. The order follows the dependencies among the claims. The research statement is as follows:

Comparison notation.

Three stacks share the task observation and a cylinder-position interface to PhysX. R and IF form the matched actuator-representation comparison, with separately trained policies. AP has a direct-valve action space and is evaluated against its own frozen base

and context controls. The symbols are those of Section 4.1. At sample k the four cylinders carry extension $\boldsymbol{x}_k$, extension velocity $\dot{\boldsymbol{x}}_k$, chamber pressure difference $\Delta\boldsymbol{p}_k$ and rod force $\boldsymbol{F}_k$, each in $\mathbb{R}^4$; $\boldsymbol{u}_k \in [-1,1]^4$ is the normalised valve command; and

$$\boldsymbol{c}_k = \left[\Delta\boldsymbol{x}_k^*,\ \Delta\dot{\boldsymbol{x}}_k^*\right] \in \mathbb{R}^8 \tag{1.1}$$

is the desired cylinder motion over the next sample. A hat marks a prediction.

Two models are identified from high-fidelity Simscape data and then transferred frozen. The inverse model ID converts desired motion into a valve command, and the forward model FD converts a valve command into a predicted hydraulic transition:

$$\Delta\boldsymbol{u}_k = f_{\mathrm{ID}}\Big(\boldsymbol{x}_k,\ \dot{\boldsymbol{x}}_k,\ \Delta\boldsymbol{p}_k,\ \boldsymbol{F}_k,\ \boldsymbol{c}_k,\ \boldsymbol{u}_{k-1}\Big) \in \mathbb{R}^4, \tag{1.2}$$

$$\left[\Delta\hat{\boldsymbol{x}}_k,\ \Delta\hat{\dot{\boldsymbol{x}}}_k,\ \Delta\widehat{\Delta\boldsymbol{p}}_k\right] = f_{\mathrm{FD}}\Big(\boldsymbol{u}_k,\ \boldsymbol{x}_k,\ \dot{\boldsymbol{x}}_k,\ \Delta\boldsymbol{p}_k,\ \boldsymbol{F}_k\Big) \in \mathbb{R}^{12}, \tag{1.3}$$

where the valve command is accumulated, not issued absolutely, $\boldsymbol{u}_k = \mathrm{clip}(\boldsymbol{u}_{k-1} + \Delta\boldsymbol{u}_k,\ -\boldsymbol{u}_{\max},\ \boldsymbol{u}_{\max})$. The third model, the reduced-order model ROM, replaces that composition with a single command-conditioned step

$$\left[\Delta\hat{\boldsymbol{x}}_k,\ \Delta\hat{\dot{\boldsymbol{x}}}_k,\ \Delta\widehat{\Delta\boldsymbol{p}}_k\right] = f_{\mathrm{ROM}}\Big(\boldsymbol{h}_k,\ \boldsymbol{c}_k\Big), \tag{1.4}$$

in which $\boldsymbol{h}_k$ is a six-sample history of cylinder state and previously predicted valve command, and the transition is written as an explicit hydraulic chain with learned heads rather than as a free regression (Section 4.4). In every case only the extension increment $\Delta\hat{\boldsymbol{x}}_k$ is integrated into the cylinder target; the velocity and pressure outputs are diagnostic.

The three stacks are then

$$\text{R:} \qquad \boldsymbol{o}_k \xrightarrow{\pi} \boldsymbol{c}_k \xrightarrow{f_{\mathrm{ROM}}} \Delta\hat{\boldsymbol{x}}_k \longrightarrow \text{PhysX}, \tag{1.5}$$

$$\text{IF:} \qquad \boldsymbol{o}_k \xrightarrow{\pi} \boldsymbol{c}_k \xrightarrow{f_{\mathrm{ID}}} \boldsymbol{u}_k \xrightarrow{f_{\mathrm{FD}}} \Delta\hat{\boldsymbol{x}}_k \longrightarrow \text{PhysX}, \tag{1.6}$$

$$\text{AP:} \qquad (\boldsymbol{o}_k,\ \hat{\boldsymbol{c}}) \xrightarrow{\pi} \boldsymbol{u}_k \xrightarrow{f_{\mathrm{FD}}} \Delta\hat{\boldsymbol{x}}_k \longrightarrow \text{PhysX}. \tag{1.7}$$

IF (PPO-ID-FD) uses the Simscape-identified ID–FD pair without deployment calibration. *R* (PPO-ROM) uses the command ROM calibrated in IsaacLab and supplies a virtual reference, not a performance upper bound. *AP* (Adaptive Policy stack) omits ID and conditions a frozen direct-valve actor on an online payload estimate. Its forward model remains frozen. The adaptation enters at the actor input, not at its output:

$$\boldsymbol{u} = \pi_{\mathrm{frozen}}\Big(\boldsymbol{o},\ \hat{\boldsymbol{c}}\Big), \qquad \hat{\boldsymbol{c}} = \boldsymbol{0} \ \Rightarrow\ \boldsymbol{u} = \mathrm{base}(\boldsymbol{o}), \tag{1.8}$$

where only the four context columns of the first actor layer are trained. Zeroing the context recovers the frozen controller exactly (Section 5.7.2). The vector $\hat{\boldsymbol{c}}$ carries plant context and is not a predicted cylinder command; of its four channels only the payload varies in any experiment reported here.

The comparison therefore concerns complete policy–actuator stacks. Model transfer, policy learning and payload adaptation are evaluated separately.

To determine how hydraulic actuator dynamics learned from high-fidelity simulation can be reduced, transferred into a computationally efficient robotic simulation and used for reinforcement-learning control, and under what conditions online adaptation improves on the robustness those learned models already provide.

The statement decomposes into six research questions, which are answered in the order given. The first three concern how the actuator is represented, the fourth and fifth concern control through that representation, and the last concerns what remains for adaptation once the others are settled. The six questions are:

1. How can data from a high-fidelity Simscape model be used to learn computationally tractable representations of hydraulic actuation while preserving the command dependence a controller requires?
2. To what extent do actuator models identified from Simscape transfer to an IsaacLab manipulator without deployment-specific retraining?
3. Can a structurally constrained reduced-order hydraulic model supply dynamics that are accurate and stable enough for GPU-parallel policy training?
4. Can a policy trained from a privileged classical teacher become the sole deployed controller while acting through learned actuator dynamics?
5. How does the choice of actuator representation affect tracking, command behaviour, latency and robustness under plant variation?
6. Where plant variation survives robust training, can a controller estimate its own payload online and condition on it, and in which regime does that conditioning improve closed-loop tracking rather than cost it?

RQ6 concerns the deployed payload-conditioned AP controller. It follows the modelling and control questions because adaptation is useful only if plant variation remains important, is observable and can affect the applied command. The validation and control experiments in Chapter 7 answer these questions in sequence.

The work makes the following contributions:

Transferable Hydraulic Actuator Learning: A pipeline that identifies hydraulic actuation from high-fidelity Simscape data and transfers it into IsaacLab through two representations, a modular inverse and forward chain and a structurally constrained reduced-order model. The inverse and forward pair transfers frozen, without deployment-specific retraining. The reduced-order model instead receives a small bounded calibration against the deployment plant, so it serves as a reference condition rather than an independent competitor (Chapter 4).

Control-Relevant Identification and Validation: A demonstration that predictive accuracy can conceal the loss of exactly the property a controller needs, together with the

procedure that detects it. A forward model reached a near-perfect supervised score while learning to ignore the valve command. The validation loss did not show this; ablating the command did. Excitation design paired with input ablation is established here as a required test before a learned actuator model is admitted to control (Chapters 6 and 7).

Reduced-Order Actuator Model: A command-conditioned recurrent actuator surrogate whose transition is written as an explicit hydraulic chain, so the predicted step stays differentiable in named quantities and bounded in form, with learned heads covering valve realisation and unmodelled loading. It is accurate enough to serve as the deployment plant, stable in recursive rollout, and cheap enough to evaluate inside GPU-parallel policy training. It is not an identified plant: the fitted physical constants settle on their clamps and the realised motion comes from the learned heads, as shown in Section 4.4.2.

Policy-Only Control Through Learned Hydraulics: A damped least-squares controller supplies privileged demonstrations during training and is then removed, leaving the distilled policy as the entire deployed controller, acting through learned actuator dynamics. Both representations support closed-loop terrain following in that configuration (Chapters 5 and 7). One training run exists per representation, so this compares two realised controllers and does not estimate what retraining either would produce.

A Controlled Evaluation of Online Adaptation: A paired protocol comparing estimated, zeroed, true and permuted payload context on a shared frozen controller. It separates the effect of conditioning from the value of correct load information (Chapter 7).

Where Online Adaptation Is Effective: An online payload-conditioned controller measured on one machine under one protocol, with the regime in which it helps identified, not assumed. On two held-out meshes it improves cut height, lateral tracking and ground contact together where a heavy payload costs the frozen controller ground contact, and costs lateral accuracy where the frozen controller is not failing. Permuting the payload across environments separates what correct identification contributes from what conditioning on any plausible load contributes (Chapter 7).

Figure 1.2 condenses the logic that connects these contributions. The sequence is a cycle, not a one-way pipeline: evidence from robustness, identifiability and adaptation feeds back into the next actuator representation and the transfer assumptions made for it.

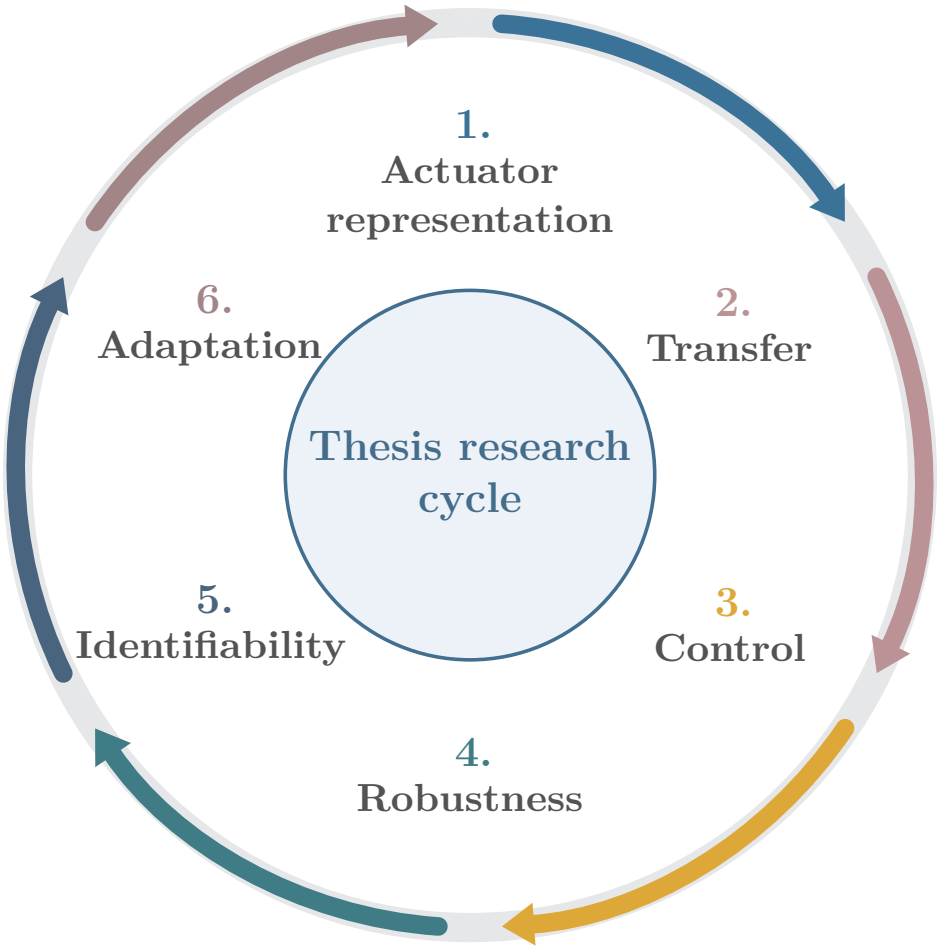

Figure 1.2: Research cycle connecting actuator representation, transfer, closed-loop control, robustness, identifiability, and adaptation. Evidence from the final stages informs the next actuator representation and its transfer assumptions.

1.5 Thesis Structure

The document follows the dependency between the experiments.

Chapter 2 reviews the actuation chain, the kinematic tools behind the classical controller, learned dynamics models and the pitfalls of identifying them from closed-loop data, domain randomisation and latent adaptation. It closes on the gap that remains.

Chapter 3 states the problem the work addresses, that no real machine was available and what stands in for it, and gives the architecture in one figure before any component is specified. It then fixes the platform, the mappings between task and cylinder space, the high-fidelity Simscape model the actuator data derives from, the behavioural checks that model passes and the two limits it does not clear, what the controller may observe, and the gates every later result is scored against.

Chapter 4 develops the inverse, forward and reduced-order models, and scores each one where it is defined: the paired control response that near-perfect supervised accuracy fails to establish, the transfer of the frozen inverse model to the deployment plant, the command sensitivity whose absence forced a second collection, and the reduced-order fit

whose coefficients reproduce the motion without identifying the machine. It closes on the offline AAM diagnostic and the online payload estimator.

Chapter 5 presents the damped least-squares teacher and the performance it reaches, the shared observation and action interfaces, distillation by DAgger and a measured distillation run, the objective mismatch that unconstrained fine-tuning produces and the reference-anchored formulation that answers it, and assembles the three deployed stacks.

Chapter 6 documents the data everything above was identified from: the excitation design, the randomised factor set, the splits, and the identifiability audit that forced a second, open-loop collection. It comes after the models and the controllers so that each collection is read against the use it was built for.

Chapter 7 compares the matched pair on tracking, command behaviour and dynamic response, measures how much of the factor draw reaches each plant, and closes on online adaptation against a frozen base and a permuted-payload control.

Chapter 8 states what the work establishes and what it does not, records the limitations of the evidence, and ends with the future work.

Appendix A holds the reference data and the reproducibility record the main chapters cannot carry: the plant ranges and frozen collections behind Chapter 6, the dataset audit values, the timing contract, the artifact digests and the reproduction entry points.

2. Fundamentals and Related Work

Chapter 1 argued that precise task-space tracking with a hydraulic mowing arm is limited less by kinematics than by the actuator. The path from a valve command to actual cylinder motion is nonlinear and load dependent, and it differs from one machine, season, or attachment to the next. This chapter collects the background needed to judge that argument and to understand the design decisions made later in the thesis. Section 2.1 introduces the machine class and its electro-hydraulic actuation. Section 2.2 recalls the kinematic tools behind the classical controller, and Section 2.3 reviews how conventional model-based control deals with hydraulic uncertainty. The remaining sections cover the learning methods this work builds on. Section 2.4 treats data-driven dynamics models and the pitfalls of identifying them from closed-loop data. Section 2.5 covers reduced-order and grey-box modelling, Section 2.6 domain randomisation and latent adaptation, and Section 2.7 policy optimisation with structured priors. Section 2.8 then states the gap that remains and how the three control stacks of this thesis (R, IF, and AP) address it. As defined in Chapter 1, the first two abbreviate PPO–ROM and PPO–ID–FD; AP is the Adaptive Policy stack, named for where its adaptation sits, at the actor input rather than on the frozen forward model. Only material that is needed for the later chapters is included. Background and results are kept apart: where a later measurement is the reason a line of work was left behind, it is quoted forward and marked as such, and no argument in this chapter rests on it. The concrete system definition and problem formulation follow in Chapter 3.

2.1 Manipulator Actuation and Control Fundamentals

2.1.1 Application Context and Implement Carriers

The MULAG FME 500 mounts a multi-link hydraulic boom and a 1.2 m flail head on a Unimog carrier [MULAG Fahrzeugwerk 26]. It reaches about 6.5 m laterally, below road level and over obstacles, while an operator adjusts cutter height, tilt and lateral position as the vehicle moves. Automating this continuous tracking task is the practical motivation of the thesis.

Hydraulic cylinders suit the required force, shock tolerance and existing vehicle power supply, but their command gain changes strongly with load and configuration. This trade-off between force density and precision is central to hydraulic manipulator control [Mattila 17].

2.1.2 The Electro-Hydraulic Actuation Chain

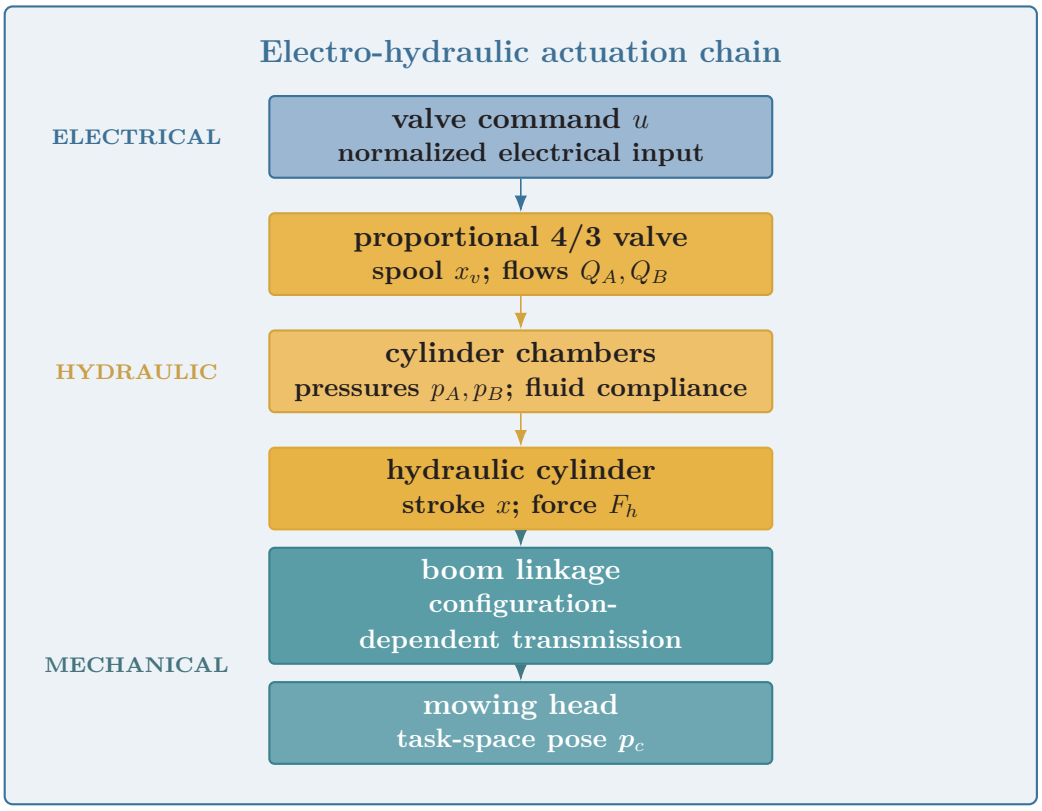

Figure 2.1: The electro-hydraulic actuation chain from valve command to mowing-head motion. The block colors distinguish the electrical, hydraulic and mechanical stages; each subtitle identifies the principal quantity passed to the next stage.

In Figure 2.1, an electrical command moves a proportional valve spool and meters flow to the cylinder chambers. The orifice law [Merritt 67] makes flow depend on spool position and the square root of the pressure margin. Valve authority is therefore load-dependent and directional; overlap creates a dead zone near null and the spool saturates at large commands [Jelali 03].

Compressibility, changing chamber volume, relief limits, leakage and seal friction further separate flow from motion. Several of these effects are non-smooth, while bulk modulus, leakage and friction drift with temperature and wear. The mechanical linkage then converts cylinder translation through configuration-dependent lever arms. Chapter 4 consequently models the hydraulic realisation in cylinder space, before kinematic effects are mixed into cutter error.

2.1.3 High-Fidelity Simulation as a Data Source

The component-level Simscape plant [The MathWorks, Inc. 24a] resolves the stiff hydraulic dynamics but runs one machine at small integration steps. Policy optimisation instead

benefits from thousands of GPU-parallel plants [Makoviychuk 21]. The detailed model is therefore used offline to generate identification data; compact actuator models carry its behaviour into parallel training, as summarised in Figure 2.2.

Speed is not the only reason for that arrangement here. Where the machine itself is unavailable, the detailed model also stands in as the source the identification data would otherwise have come from, which turns the fidelity of what it supplies into a question this thesis has to ask rather than assume (Section 3.1.2).

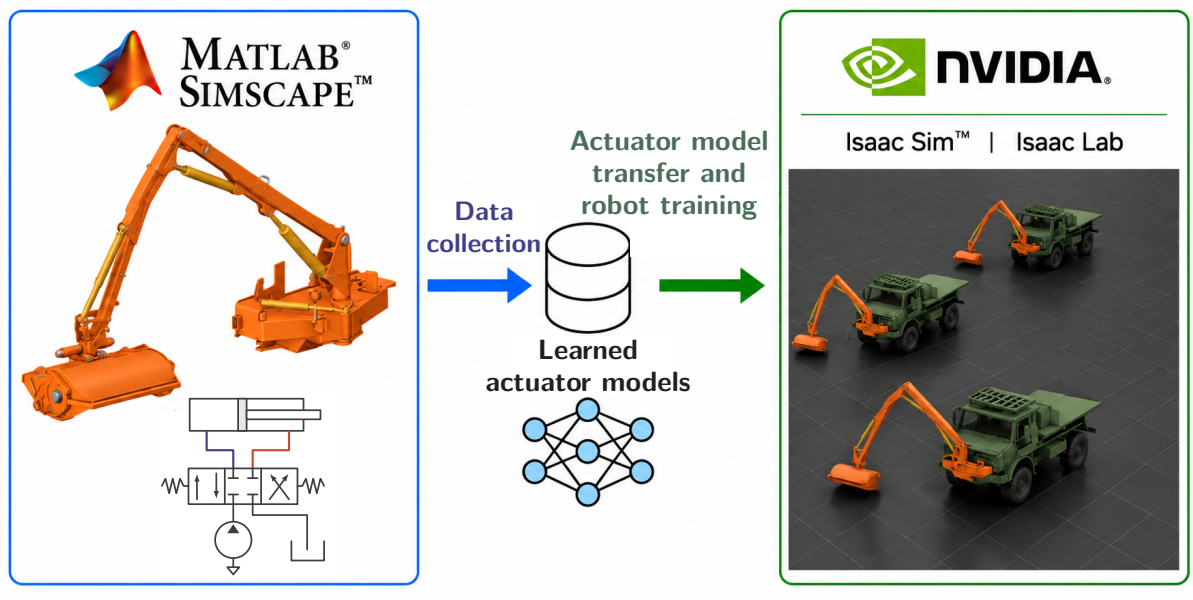

Figure 2.2: Simscape supplies high-fidelity hydraulic data; compact actuator models carry that behaviour into GPU-parallel training in IsaacLab, where the MULAG FME 500 is the controlled machine. Logos: MathWorks and NVIDIA.

2.2 Kinematics and Task-Space Tracking

2.2.1 Differential Kinematics

The cutter path is specified in vehicle coordinates, so the manipulator Jacobian maps joint rates to the required tool velocity [Siciliano 09]. Near singular configurations the ordinary pseudo-inverse produces excessive commands. The damped least-squares inverse of Wampler [Wampler 86] and Nakamura and Hanafusa [Nakamura 86] trades some local accuracy for bounded commands and is retained as the classical controller.

2.2.2 From Joint Space to Cylinder Space

The actuated coordinates are cylinder lengths, and their nonlinear cylinder-to-joint map has a configuration-dependent lever arm. A passive head joint further limits independently controllable directions. The calibrated cutter Jacobian in Chapter 3 composes these mappings. It still assumes that commanded cylinder motion is realised; the learned actuator models address that remaining gap.

2.3 Model-Based and Conventional Control Approaches

Three families of conventional control are relevant here: fixed-gain cascades, nonlinear cancellation, and online parameter estimation. Each of them works on hydraulic manipulators. Each also carries a maintenance or excitation cost, and those costs are the reason this thesis learns the actuator instead.

2.3.1 Fixed-Gain Cascades

Industrial implements commonly map joystick input directly to valve current. The standard improvement is a cascade in which an outer position loop sets a velocity or pressure reference and a fast inner loop tracks it [Jelali 03]. The construction is cheap, needs no plant model beyond a few tuned gains, and is what a commercial machine realistically ships with.

The cost is that the gains are tuned around one operating point. The orifice law makes command-to-velocity gain scale with the square root of the pressure margin, so it changes with load and with direction of travel, and the linkage lever arm changes with configuration. A gain set tuned at the stiff end of that range responds weakly at the compliant end; tuned at the compliant end it oscillates at the stiff end. Scheduling the gains over configuration and load recovers some of the range, but the schedule is machine and attachment specific and has to be remeasured whenever either changes.

2.3.2 Nonlinear Cancellation

Nonlinear model-based control removes the dependence on an operating point by cancelling the known dynamics rather than tuning around them [Slotine 91]. For a hydraulic actuator the cancellation has to span the orifice nonlinearity, chamber compressibility and the rigid-body load, which places the valve command several integrations away from the position error and makes backstepping the usual construction [Jelali 03]. The survey of Mattila et al. [Mattila 17] reports this line, together with virtual decomposition control, as the state of the art for heavy-duty hydraulic arms, and it does deliver accurate tracking on instrumented machines.

Cancellation is only as good as the model it inverts, and the terms it depends on are the terms that drift. Bulk modulus, leakage and seal friction change with temperature and wear over a working day. Some of the structure is non-smooth and cannot be cancelled at all: valve overlap produces a dead zone that has to be inverted explicitly, as Yang et al. [Yang 26] do alongside state constraints. The learned actuator models of Chapter 4 carry an explicit dead-zone term for the same reason. Maintaining this identification per machine, season and attachment is what makes the family expensive on a commercial implement [Mattila 17], not any weakness in the control law.

2.3.3 Online Parameter Estimation

Adaptive control estimates the uncertain parameters during operation instead of fixing them in advance [Åström 95]. Yao's adaptive robust control (ARC) pairs a projection-bounded parameter estimator with a robust feedback term, so that a guaranteed transient

tracking bound does not wait for the estimate to converge, and it is the established result for single-rod hydraulic actuators [Yao 00]. On the face of it this is the closest classical answer to the problem of this thesis, because it targets exactly the drifting parameters just described, and because it is the one classical family that ships a proof, not an empirical trend, in direct contrast to the adaptation studied here (Section 7.5.9). ARC buys that guarantee by assuming a known parametric structure for the plant: the orifice, chamber and friction terms must be derived correctly in advance, with only their coefficients left uncertain. That is exactly the assumption Chapter 4 finds hard to hold on this machine, where the reduced-order model's own physical parameters settle at their optimisation clamps rather than at physically plausible values (Section 4.4).

Two properties limit classical online identification in this task. Smooth closed-loop commands provide weak excitation, while valve saturation can remove the effect of a requested correction. An adaptive controller must account for these limits explicitly [Sofrony 24].

The scope of that argument should be stated plainly. No classical adaptive baseline is run in this thesis. The experiments of Chapter 7 compare actuator representations inside one reinforcement-learning stack, so nothing here shows that adaptive robust control tracks worse than the learned route on this machine. What is claimed is narrower: the two conditions the classical estimator needs, informative excitation and an unsaturated regressor, are not supplied by the mowing task, and the maintenance burden the family carries per machine, season and attachment is the burden the thesis is trying to remove.

The experiment that would settle the comparison is concrete rather than hypothetical. It needs four things. The first is a per-cylinder differential-pressure loop, because ARC's estimator is driven by the same pressure signal this simulation only produces as an uncalibrated model state (Section 5.7.5). The second is an open-loop identification campaign on the physical implement, to fit the orifice and friction structure ARC assumes rather than the clamped values the reduced-order model settles on (Section 4.4). The third is a projection bound and adaptation gain tuned against that identified structure. The fourth is a second control implementation, run on the same excitation schedule and evaluation gates as Chapter 7, so that the two families are compared under one protocol rather than one against the literature's reported numbers for the other. None of that instrumentation exists on the target machine yet (Chapter 8), which is why the comparison is left open rather than run.

2.3.4 Retained Elements

The classical methods are not replaced wholesale. Kinematics is reliable and is kept: the DLS controller of Section 2.2 acts as teacher and as the data-collection controller. What is learned is the part the model-based families struggle to maintain: the mapping from cylinder command to realised motion, and the policy that issues that command (Section 2.7).

2.4 Data-Driven System Identification and Learned Dynamics

2.4.1 Learned Forward and Inverse Dynamics

Robot model learning distinguishes forward models, which predict the next state from state and command, from inverse models, which predict the command for a desired transition [Nguyen-Tuong 11]. Section 4.1 fixes both interfaces. One-step fit is insufficient when a forward model is recursively rolled out, because small errors compound [Nagabandi 18]. The models here are therefore trained with multi-step objectives. Probabilistic ensembles represent growing rollout uncertainty [Chua 18]; recurrent architectures instead retain unobserved pressure and valve history [Hochreiter 97, Cho 14], as used by the reduced-order model.

2.4.2 Closed-Loop Identifiability

Informative identification requires inputs that excite the plant independently of feedback [Ljung 99, Isermann 11]. In closed-loop data, command and state history are correlated; a flexible predictor can score well by ignoring the command, and validation from the same distribution will not reveal the shortcut [Schoukens 19].

That failure occurred here: a closed-loop forward model reached one-step $R^2 \approx 0.999$ yet showed almost no command dependence under ablation. Chapter 6 therefore uses open-loop valve excitation, and every learned model is checked with zeroed and shuffled commands. Prediction score alone is not treated as evidence of control authority.

2.5 Reduced-Order Modelling

Reduced-order models retain dominant behaviour at lower cost [Benner 15]. Here the full Simscape plant is reduced to per-cylinder extension, velocity and pressure difference. The construction is grey-box: known structure is embedded so that data fits what remains unknown [Karniadakis 21]. As in the structured dynamics network of Lutter et al. [Lutter 19], explicit physical layers improve interpretability. The hydraulic ROM retains the orifice, pressure and integration structure; learned terms represent valve realisation, configuration-dependent load and bounded residuals.

Hwangbo et al. [Hwangbo 19] combined analytic rigid-body mechanics with a learned actuator response, a pattern later scaled in GPU-parallel simulation [Rudin 22]. Yamamori et al. [Yamamori 26] instead shaped the physical actuator to match the simulation. A commercial valve and pump cannot be redesigned in that way, so this thesis learns the hydraulic realisation while PhysX retains the rigid-body mechanics.

2.6 Domain Randomisation and Latent Adaptation

2.6.1 Robustness Through Randomised Training

A model trained on one simulated plant inherits its parameters as assumptions. Domain randomisation instead trains across a family, first for visual appearance [Tobin 17] and

then for dynamics such as mass, friction, gain and latency [Peng 18]. The hydraulic family in Chapter 6 varies bulk modulus, valve gain, leakage, friction and payload, producing robustness to physically plausible plant variation.

2.6.2 From Robustness to Adaptation

Figure 2.3 contrasts a policy trained for one robust response across the plant family with an adaptive stack that conditions its response on the current plant.

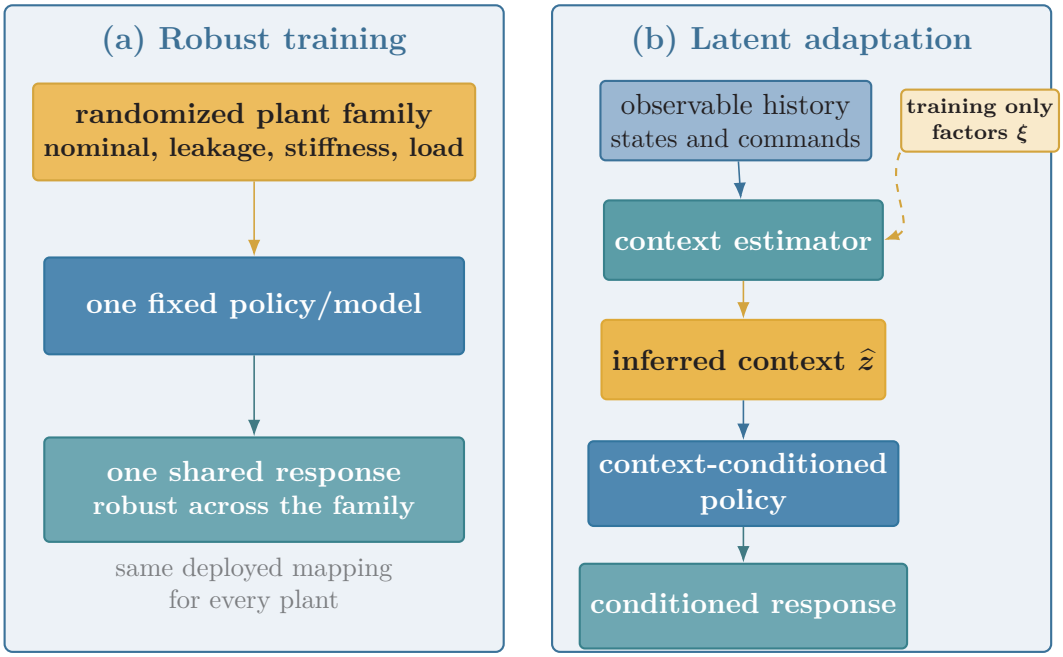

Figure 2.3: Canonical latent-adaptation architecture contrasted with robust training. The adaptive panel summarises a literature pattern, not the deployed AP pipeline. AP replaces the learned history encoder with a ridge regression from the current observation to one named quantity, the payload, fitted after the policy is trained.

Teacher–student methods encode known simulation factors into a privileged latent and train an estimator to recover it from observable history. The construction generalises the wider privileged-learning idea, in which a teacher with access to unavailable state supervises a student restricted to deployable observations [Chen 19]. Lee et al. [Lee 20] used it for hidden terrain properties; Kumar et al. [Kumar 21] developed rapid motor adaptation; Miki et al. [Miki 22] extended the same teacher–student route to fusing proprioception with exteroception under unreliable perception. Distillation from a privileged teacher is the mechanism this thesis uses in Chapter 5, with a classical controller in the teacher's place rather than a privileged policy [Ross 11]. All of them assume that the factor directions the estimator retains are the directions the controller needs. Liu et al. [Liu 25] report the failure when it does not hold, where a low-dimensional adapter drops factors that act through separate channels, and add training phases to recover them. The present thesis uses a named physical quantity, payload, as the conditioning signal.

Unlike classical online parameter estimation [Åström 95], the learned latent need not have a direct physical interpretation. It can be given one. Hu et al. [Hu 26] estimate named

physical quantities on an excavator arm with Bayesian reinforcement learning, which buys interpretation at the price of the explicit plant model that Section 2.3 identified as the maintenance burden. A latent can also condition a dynamics model, as in the contextual neural ODE of Yu et al. [Yu 26]. Here the conditioning enters the policy: a payload estimate from the current observation is supplied through four context columns, with only the payload channel varying (Section 5.7.4).

Randomised training produces a single controller for a family of plants. Identification of that family is what makes a member-specific correction possible. In canonical latent adaptation, adaptation is the online use of that identification: the controller infers which plant is present from observable history and conditions the command on that estimate. The two responses are therefore sequential, not alternatives.

2.7 Reinforcement Learning and Structured Priors

2.7.1 Policy Optimisation

Reinforcement learning optimises a policy from observations to actions for expected cumulative reward [Sutton 18]. This thesis uses PPO [Schulman 17] with generalised advantage estimation [Schulman 16]; Section 5.2.5 gives the exact objective. GPU-parallel simulation supplies the required throughput [Makoviychuk 21, Rudin 22], which is why compact actuator surrogates replace the full hydraulic plant during training.

2.7.2 Structured Priors for Policy Learning

Learning from scratch would discard reliable kinematics. Residual RL instead adds a bounded learned correction to a controller at runtime [Johannink 19, Silver 18]. Policy distillation moves the prior to training: a privileged DLS teacher supplies demonstrations, then the student acts alone [Ross 11]. This thesis takes the second route. The teacher is distilled and PPO updates are anchored to the frozen student (Section 5.4), and the deployed actor remains the sole controller: no analytic controller runs underneath it at deployment, so the runtime correction of residual RL is not the construction used here.

One clarification belongs here, because the vocabulary collides. The adaptive stack of Chapter 5 does modify its own frozen actor, but not by adding a residual in the sense of this subsection. There is no classical controller beneath it to correct, and its purpose is not to supply a prior that accelerates learning. It is an online adaptation mechanism internal to the reinforcement-learning policy, whose only function is to carry the dependence on inferred plant context that a frozen actor cannot express. It therefore belongs to the adaptation line of Section 2.6.2 and is compared against that literature, not against residual policy learning.

The idea parallels real-world fine-tuning that retains a simulation-trained value function [Yin 25], but here the retained object is the reference policy. Anchoring was introduced because unconstrained PPO improved reward while degrading the physical tracking criterion (Section 5.3).

2.7.3 Reinforcement Learning on Hydraulic Machines

Deep RL has reached full-size hydraulic machines. What distinguishes those stacks from the one built here is not scale but the level at which the policy commands the machine, because that level decides how much of the hydraulic chain of Section 2.1.2 the policy has to represent at all.

Egli and Hutter [Egli 20] trained an excavator-arm policy in simulation and transferred it to a full-size walking excavator, then generalised the construction across excavation tasks [Egli 22]. The policy acts on joint-level velocity references, and a low-level tracking layer on the machine converts them into valve currents. Spinelli et al. [Spinelli 24] keep that division on a commercial material handler and add an underactuated tool, so the policy has to exploit passive tool dynamics, not command them, while the actuated joints are still driven through the same kind of tracking layer. In both cases the orifice nonlinearity, the dead zone, the directional asymmetry and the load dependence live below the learned policy, in a layer that was identified and calibrated for that machine before the policy existed.

That division is sound where the tracking layer already exists. It is not available here for two reasons. The FME 500 is a series road-maintenance implement with no calibrated joint-velocity layer to inherit, and building one is precisely the per-machine identification that Section 2.3 identified as the maintenance cost of the model-based families. More importantly, a velocity interface hides the quantity this thesis sets out to measure. If the layer beneath the policy already owns the command-to-motion mapping, the question of whether a learned actuator retains command authority under saturation and dead zone cannot be posed at all; it has been answered by construction, once, by hand. The stacks here therefore command the actuator directly, either as a cylinder increment realised by a learned model or, in AP, as the valve command itself.

Maitan et al. [Maitan 26] take the opposite position and train on the physical cylinder, bounding exploration with a contraction-metric filter. That work does command the actuator at the low level, and training on hardware removes the transfer question outright. No such option existed here, which is why that question is posed explicitly and answered between two simulators instead (Section 3.1.3). The result is force control on a single actuator on a test bench, and it does not obviously scale to a multi-link boom on a moving carrier, where the exploration the policy needs is the exploration the filter has to prevent.

These studies settle scale and feasibility. In each of them the actuator representation is a fixed design choice inside the stack rather than a variable, so none separates the transfer result from the representation that produced it. That separation is criterion V below.

None of these stacks varies the plant within a deployment. They are trained for robustness across one machine and its payload or soil conditions, and a plant that changes during a run is outside the construction. What remains open is the combination: a randomised plant family, online actuator adaptation, a privileged kinematic teacher, and a matched comparison of actuator representations under one protocol.

A policy can also compensate for a poor actuator model. Chapter 7 therefore reports rollout error, command magnitude and smoothness, and cylinder-level residuals alongside task tracking. Equal task error alone is not evidence of equal model quality.

2.8 Research Gap and Thesis Positioning

The literature leaves a specific combination unresolved. Hydraulic control provides structure but requires maintained plant models; learned dynamics replace derivation with data but depend on identifiable excitation; actuator surrogates and latent adaptation are established mainly for electric legged robots; and hydraulic RL studies do not isolate actuator representation under a matched protocol. Table 2.1 summarises this position.

The five criteria are: (I) learned hydraulic actuation, (II) data audited for causal input identifiability, (III) online adaptation to plant variation at deployment, (IV) a classical-control prior transferred through policy distillation, and (V) a matched actuator comparison.

The rows are lines of work, not individual papers, and the criteria are the ones this thesis needs rather than a neutral taxonomy of the field. Every criterion is individually established somewhere, several of them for decades, and single works cover more of a row than a coarse mark can show. What has not been established is the combination on one machine under one protocol: that an actuator identified from high-fidelity simulation retains *command* dependence after transfer, that a policy can control through it with the classical controller removed, and that an online estimate of the operating condition improves closed-loop tracking under exactly the plant conditions where that policy's learned robustness has run out.

Table 2.1: Positioning against the reviewed lines of work. Parenthesized marks denote partial or implicit coverage. Column III is read as the criterion states it, online adaptation to plant variation at deployment; how this thesis meets it, and in which regime, is set out below and measured in Section 7.5.

Line of work	I	II	III	IV	V
Model-based hydraulic control [Mattila 17, Jelali 03]	—	(✓)	(✓)	(✓)	—
Actuator networks for legged robots [Hwangbo 19, Rudin 22, Yamamori 26]	—	✓	—	—	—
RL for hydraulic machines [Egli 20, Egli 22, Spinelli 24, Maitan 26]	(✓)	(✓)	—	—	—
Latent online adaptation [Lee 20, Kumar 21, Yu 26, Liu 25, Hu 26]	—	—	✓	—	—
Residual RL and policy distillation [Johannink 19, Silver 18, Ross 11, Yin 25]	—	—	—	✓	—
This thesis (R / IF / AP)	✓	✓	✓	✓	✓

The contribution remains the complete experimental chain: identifiable cylinder data, structurally constrained models fast enough for parallel training, online adaptation and a common evaluation protocol. Adaptation is reported as a conditional mechanism, with positive and negative training seeds kept in the same analysis. The three tested stacks are:

R. A command-conditioned ROM realises the policy's desired cylinder motion. It is the reference and the only actuator calibrated to the deployment plant (Section 5.9).

IF. The same policy interface drives the frozen inverse model followed by the strict forward model, forming the robust learned baseline.

AP. The policy emits the valve command directly and the inverse model leaves the path. The frozen actor's input is widened by four context columns carrying a payload estimated online, and the same forward model realises the resulting command ahead of PhysX.

All stacks share the task, terrain, trajectories, deployment factors and evaluation gates. R and IF additionally share the action contract and form the matched pair; AP does not, because its policy chooses a valve command, not a cylinder increment, and it is therefore compared with its own frozen base, not ranked against the others. The ROM is calibrated to the deployment plant, so causal architectural claims rely on the matched pair and on within-stack paired ablations rather than on a three-way contrast (Section 5.9.2). Chapter 3 now grounds this comparison in the concrete machine.

Two distinctions separate the adaptation used here from the latent-adaptation line it is closest to. What is estimated is a *named physical quantity*, the payload, rather than an anonymous code whose retained directions are assumed to be the ones control needs; the failure mode Liu et al. [Liu 25] describe therefore cannot arise, because the quantity is chosen, not discovered. And what the estimate feeds is a frozen policy widened by four columns, not a jointly trained encoder, so the unadapted controller is recovered exactly by zeroing the estimate and the comparison has no capacity confound. The cost of those choices is generality: this mechanism adapts to one factor, and Section 7.5 shows it earns its place only in the regime where that factor is what is breaking the controller.

3. System Fundamentals and Problem Formulation

Chapter 2 established the control and modelling concepts this thesis uses. This chapter first defines the experimental task and the two engineering reasons for the study: why Simscape must supply the hydraulic data, and why a fixed learned model may still require online adaptation. It then gives the complete architecture spanning Simscape and IsaacLab before fixing the platform, environments, signals and evaluation contract used by the remaining chapters.

One caveat applies to everything that follows. High fidelity is relative to the reduced representations used for policy training, not a claim about the physical machine: the Simscape model is a component-level hydraulic source model, and this thesis does not claim its complete hydraulic state, particularly absolute pressure, has been validated against measurements from the FME 500. The two simulators share the cylinder ordering, kinematics, control interface and cutter-tip task variables defined here, but they are not assumed to be identical plants.

3.1 Experimental Task and Engineering Motivation

3.1.1 Experimental Task and Research Purpose

The experimental task is moving verge following. A scripted truck carries the FME 500 along a randomised verge while the arm keeps the cutter near the verge centre line and target cut height, maintains ground contact and remains within its cylinder limits. This contact-rich task is deliberately demanding of the actuator: the same reference must be realised across changing geometry, terrain, load and hydraulic response.

The purpose is not to develop a mowing application as an end in itself. The task is an experimental instrument for asking whether hydraulic actuator models learned offline can be embedded in robot simulation, used to train a policy and then support closed-loop robot control. The detailed path families, duration and feasibility limits are fixed in Section 3.8; the task-level criteria are defined in Section 3.11.

3.1.2 Why Simscape Supplies the Identification Data

The physical FME 500 was not available for data collection, and no measurement from the machine enters this thesis. The hydraulic behaviour must therefore come from the component-level model developed in the preceding project. Simscape is used because it resolves the valve, line, chamber, compressibility and load interactions that a rigid-body engine does not. It is the closest available substitute for the actuator data that would otherwise be collected from the machine.

That fidelity has a computational cost. Simscape integrates a stiff, multi-domain system at a small step and is too slow for the millions of steps required by policy optimisation. Its role is therefore restricted to offline data generation: closed-loop tracking supplies the initial ID, FD and ROM data, and open-loop valve excitation supplies the strict FD fine-tuning data. The learned models are then frozen and evaluated inside the faster IsaacLab loop.

3.1.3 A Sim-to-Sim Test of Transfer

IsaacLab supplies the rigid-body arm, terrain and contact, but PhysX has no hydraulic actuator. A command-conditioned ROM converts the requested action to cylinder motion, so ROM plus PhysX is treated as the deployment plant. The inverse and forward models are identified only on Simscape data and transferred to that plant without deployment-specific retraining.

This produces the structure, but not the magnitude, of a sim-to-real test: the actuator representation is identified on one plant and used to control another. R uses the calibrated deployment actuator and is the reference condition; IF substitutes the independently identified ID-to-FD chain. Their gap measures the cost of that transfer. It does not bound the gap that would occur on hardware, and the ROM's bounded calibration against PhysX (Section 4.4.1) prevents R from being interpreted as an independent competitor.

3.1.4 Why Online Adaptation Is Tested

A transferable model still represents only one response. Seal condition, fluid temperature, hose compliance, supply pressure and cutter payload all change the mapping from valve command to motion. Domain randomisation can train one fixed policy that tolerates a family of plants, but that policy cannot deliberately change its response for the member currently encountered because it carries no estimate of plant identity.

Online adaptation provides that missing conditioning. This thesis does not assume it will help: plant variation must remain important after robust training, the relevant change must be observable and the estimate must reach a part of the controller with authority to act. The AP experiments in Chapter 7 test whether conditioning the actor on an online payload estimate improves control.

3.2 System Architecture

Figure 3.1 separates data generation, collection, model learning, policy training and evaluation. Simscape supplies the initial ID, FD and ROM datasets, including separate

open-loop excitation for FD. IsaacLab/PhysX supplies the ROM calibration recordings. Model learning is offline; the resulting frozen models run inside the IsaacLab control task.

The deployed combinations are R (policy and ROM), IF (policy, ID and FD) and AP (payload-conditioned policy and FD). The actuator models stay frozen in all three. Only AP receives an online estimate of the changing payload.

Table 3.1 distinguishes initial identification from deployment calibration. R and IF use separately trained policies; their comparison does not test an unchanged policy transferred between actuators.

Table 3.1: Provenance of each frozen actuator component. The command ROM is the only one that has been calibrated against the deployment plant; the inverse/forward pair is Simscape-identified throughout. This asymmetry limits the interpretation of the comparison.

Component	Identified on	Calibrated on
Command ROM	Simscape closed-loop command export	Five IsaacLab/PhysX reference trajectories, via a bounded command-gated correction head only
Control ID	Simscape closed-loop export	None
Strict FD	Simscape closed-loop export, fine-tuned on the strict open-loop excitation export	None

3.3 Manipulator Platform

The MULAG FME 500 is modelled as a serial-link mechanism with five revolute joints. Four hydraulic cylinders actuate the slew, two boom, and head-pivot joints. The *Messerkopf* rotation has no independent actuator and is locked at the mowing configuration; it is therefore not a controlled degree of freedom. The cylinder order used throughout the thesis is slew (*Drehzapfen*), boom I (*Ausleger I*), boom II (*Ausleger II*), and head pivot (*Messerkopf-Schwenk*).

Table 3.2 gives the cylinder stroke ranges used in the training data and the fixed limit of the *Messerkopf* joint. Task-space errors are measured at the cutter tip, which lies 0.79 m below the *Messerkopf* body origin.

Figure 3.2 identifies the links, joints, and four actuator channels used throughout the model.

Figure 3.3 separates the rigid bodies that form the assembled mechanism. The catalog makes the L0–L5 body assignment explicit and shows the representative barrel and rod geometry used for cylinders C1–C4.

Manufacturer CAD supplies the link geometry, joint locations, masses, centres of mass, and inertia tensors through Phobos [von Szadkowski 20]. The same CAD-derived bodies and joint frames form the mechanical basis of the Simscape and IsaacLab models, keeping dimensions, axes, masses, and cylinder lever arms consistent while the hydraulic implementations remain model-specific.

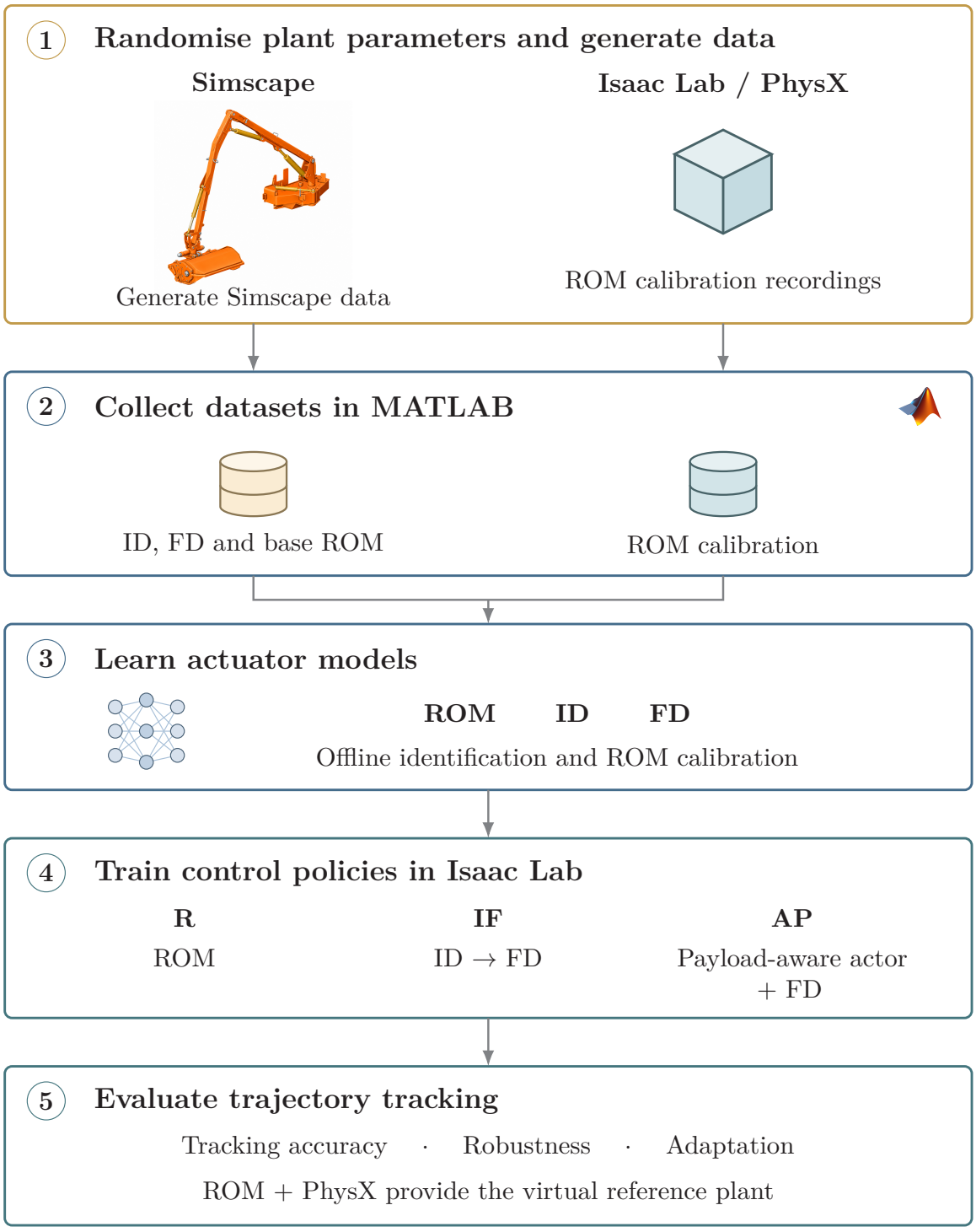

Figure 3.1: Thesis workflow for evaluating learned hydraulic models in robot control. Plant parameters are randomised to generate Simscape identification data for ID, FD and the base ROM. Isaac Lab supplies ROM calibration data. The collected datasets support offline model learning, followed by policy training and trajectory-tracking evaluation. ROM plus PhysX is the virtual reference plant.

Table 3.2: Cylinder stroke ranges in the training data (Chapter 6) and the fixed *Messerkopf* joint limit used for the mowing task.

i	Joint	Function	Limit
1	*Drehzapfen*	slew	0.018–0.493 m
2	*Ausleger I*	boom I	0.275–0.791 m
3	*Ausleger II*	boom II	0.309–0.857 m
4	*Messerkopf-Schwenk*	head pivot	0.269–0.689 m
5	*Messerkopf*	head rotation (locked)	$q_{5,\min} = q_{5,\max} = q_{5,\text{lock}}$

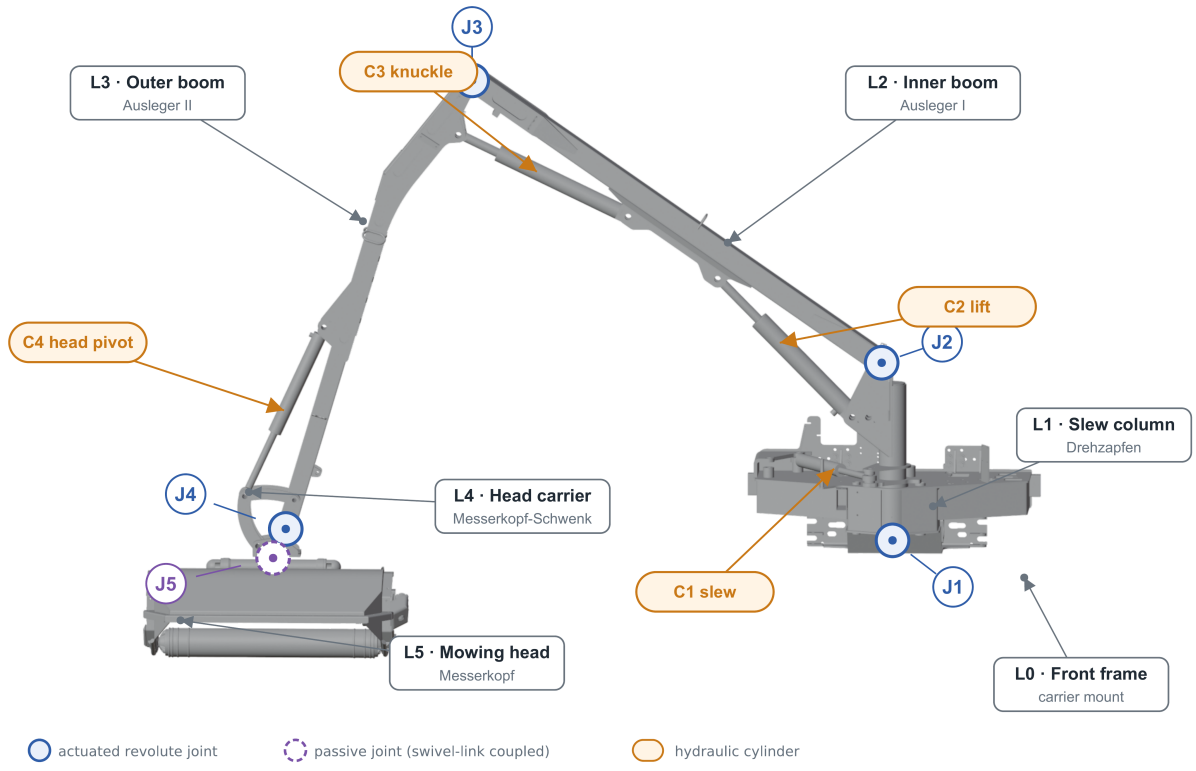

Figure 3.2: Annotated CAD view of the MULAG FME 500. Links L0–L5, joints J1–J5, and the four hydraulic cylinders C1–C4 are identified; J5 denotes the *Messerkopf* rotation, which is locked in the mowing configuration used in this thesis.

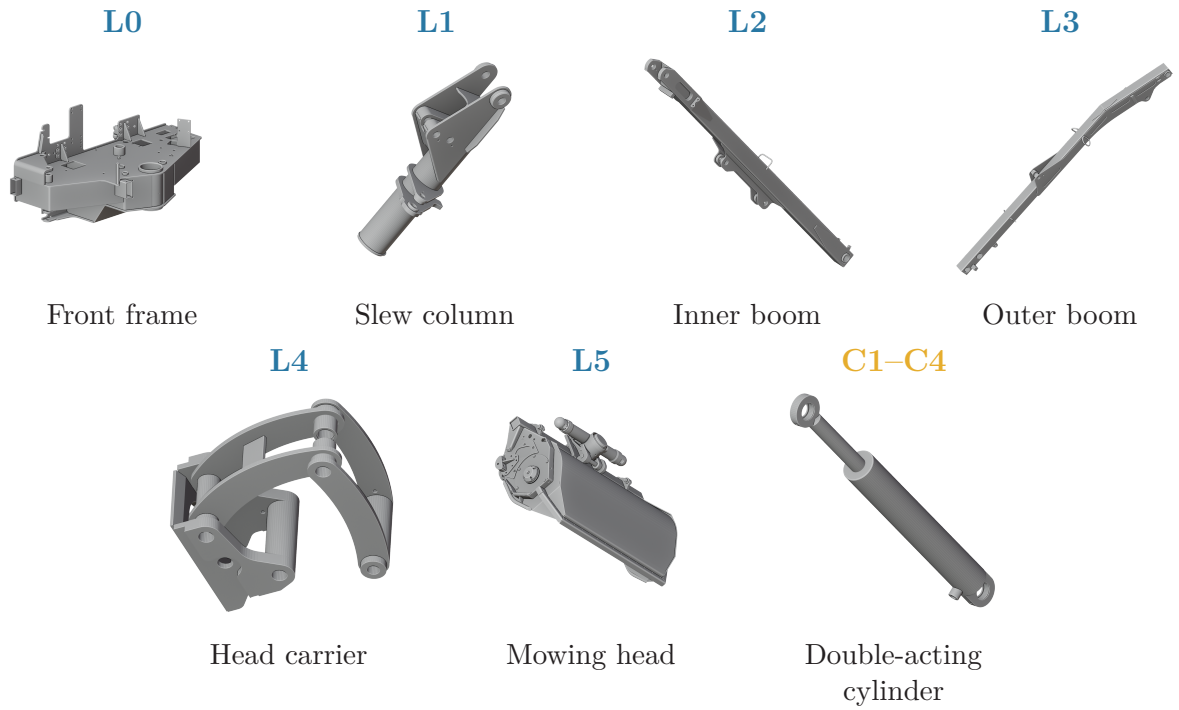

Figure 3.3: Individual CAD-derived bodies of the MULAG FME 500 shown at a common orthographic scale. L0–L5 are the front frame, slew column, inner boom, outer boom, head carrier, and mowing head. The C1–C4 panel shows the representative double-acting cylinder barrel and rod pair.

3.4 Kinematic Structure and Coordinate Mappings

3.4.1 Joint Space and Cylinder Space

The configuration is $\boldsymbol{q} \in \mathbb{R}^5$; the actuated coordinates are the four cylinder extensions $\boldsymbol{x} \in \mathbb{R}^4$. Cylinder i joins CAD attachment points $\boldsymbol{p}_A$ and $\boldsymbol{p}_B$ on adjacent links. Its length and extension are

$$\ell_i(\boldsymbol{q}) = \left\| \boldsymbol{p}_B(\boldsymbol{q}) - \boldsymbol{p}_A(\boldsymbol{q}) \right\|, \qquad x_i = \ell_i(\boldsymbol{q}) - \ell_{0,i}, \tag{3.1}$$

with $\ell_{0,i}$ the retracted reference length. Differentiating (3.1) gives the scalar mapping between joint motion and extension rate,

$$\dot{x}_i = J_{\mathrm{cyl},i}(\boldsymbol{q})\, \dot{\boldsymbol{q}}, \qquad J_{\mathrm{cyl},i} = \boldsymbol{e}_i^{\top} \left(\boldsymbol{J}_B - \boldsymbol{J}_A \right), \tag{3.2}$$

where $\boldsymbol{e}_i$ is the cylinder-axis unit vector and $\boldsymbol{J}_A$ and $\boldsymbol{J}_B$ are attachment-point Jacobians. Virtual work gives $\boldsymbol{\tau}_{\mathrm{act}} = \boldsymbol{J}_{\mathrm{cyl}}^{\top} \boldsymbol{F}_h$. The mapping varies with configuration and stroke.

3.4.2 Task Space and the Calibrated Cutter Jacobian

The cutter tip is expressed in the truck chassis frame (`base_link`); using the moving slew-base frame would corrupt the tracking error. The controlled point includes the $-0.79\,\mathrm{m}$ tool-tip offset. Vehicle motion supplies the along-track direction.

The calibrated Jacobian $\boldsymbol{J}_c \in \mathbb{R}^{3\times 4}$ maps cylinder extension rates to cutter-tip velocity. It was measured at the head-down mowing pose by stepping each cylinder target and recording the settled cutter displacement. Its singular values are 8.24, 5.69, and 2.79, giving rank 3. The damped least-squares controller in Chapter 5 uses this same Jacobian.

3.5 Actuation and Measured State

Table 3.3 lists the signals and their availability.

Table 3.3: Signal inventory of the modelled system. "S" marks signals present in the Simscape data exports, "I" signals available live in the IsaacLab environment, "M" quantities that exist there only through the learned models of Chapter 4, and "P" *privileged* quantities that exist in simulation but must never enter a deployable policy or estimator input.

Signal	Dimension	Availability	Meaning
$\boldsymbol{u}$	$[-1, 1]^4$	S, M	normalised proportional-valve commands
$\boldsymbol{x}$, $\dot{\boldsymbol{x}}$	$\mathbb{R}^4$	S, I	cylinder extensions and velocities
$\Delta\boldsymbol{p}$	$\mathbb{R}^4$	S, M	chamber pressure differences
$\boldsymbol{F}$	$\mathbb{R}^4$	S	logged rod forces
$\boldsymbol{q}$, $\dot{\boldsymbol{q}}$	$\mathbb{R}^5$	S, I	joint positions and velocities
$\boldsymbol{p}_c$	$\mathbb{R}^3$	S, I	cutter-tip position, truck frame
$\boldsymbol{h}$	$\mathbb{R}^9$	I	terrain-height profile ahead of the cutter
$\boldsymbol{c} = [\Delta\boldsymbol{x}^*, \Delta\dot{\boldsymbol{x}}^*]$	$\mathbb{R}^8$	I	desired cylinder command
$\mathbf{e}$	$\mathbb{R}^6$	P	physical randomisation factors of the active plant

In IsaacLab, valve commands and differential pressures are learned-actuator states, not direct measurements. R produces them internally; IF obtains them from the inverse/forward pair, and AP from the frozen forward model alone. The physical factor vector $\mathbf{e}$ is unavailable to the deployed policies. AP receives an online payload estimate regressed from the same 66 task channels it already reads.

3.5.1 Sensing and Observability

The deployable observation contains articulation state, terrain rays, contact signals, the desired cylinder command, and actuator-model pressure state. Plant parameters are not observed. AP receives an online payload estimate regressed from the current observation and appended to the actor input (Section 4.6).

Figure 3.4 separates deployable measurements from privileged plant information. Articulation state is read from the physics engine without encoder noise or quantisation.

3.6 High-Fidelity Simscape Model

The actuator models of Section 3.2 are identified on a multi-domain Simscape model of the FME 500 developed in the preceding master project. This section describes that model and fixes what it may be used for.

3.6.1 Mechanical Subsystem

The mechanism uses the CAD-derived links, joints, and cylinder attachment frames of Section 3.3. It is simulated in Simscape Multibody [The MathWorks, Inc. 24b]. The joint-space dynamics take the standard manipulator form [Siciliano 09]

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) + \boldsymbol{\tau}_{\text{fric}}(\dot{\boldsymbol{q}}) = \boldsymbol{\tau}_{\text{act}} - \boldsymbol{J}^{\top}(\boldsymbol{q})\,\boldsymbol{F}_{\text{ext}}, \tag{3.3}$$

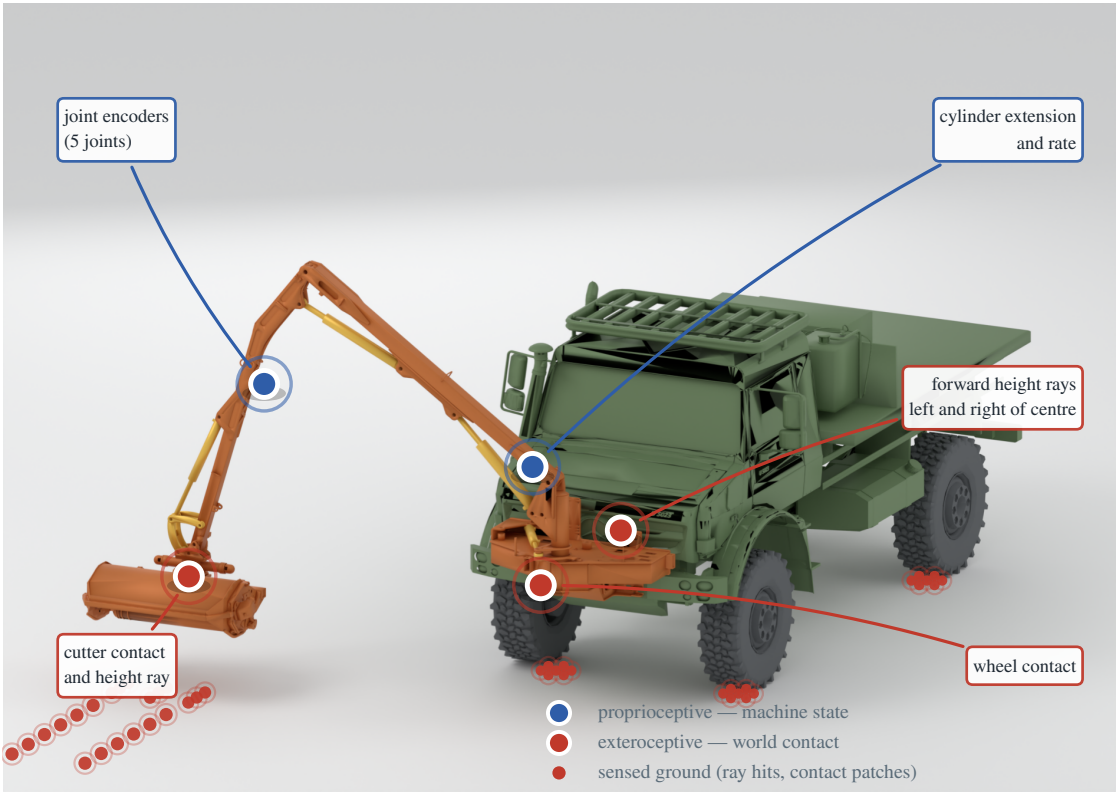

Figure 3.4: Sensor and observability map for the deployed controller. Blue callouts identify proprioceptive inputs: joint encoders and cylinder extension and rate. Red callouts identify exteroceptive inputs: forward terrain-height rays and cutter and wheel contact. The small red ground markers show where those exteroceptive inputs are taken: the tyre contact patches, and the strip and forward ray rows the cutter senses ahead of itself. They are placed on a ground frame fitted to the three visible tyre contact points, so their spacing carries the perspective of the render.

with inertia $\boldsymbol{M}$ from the imported CAD. Friction combines viscous, Coulomb, and Stribeck terms. The cutting load scales with feed velocity and vegetation density. Each cylinder uses a spherical-prismatic-cylindrical loop and hard stops enforce the limits in Table 3.2.

3.6.2 Hydraulic Subsystem

Figure 3.5 shows four 4/3 valve-driven asymmetric cylinders fed by a variable-speed, fixed-displacement pump ($Q_p = D_p\omega$), accumulator, and pressure-relief valve.

Figure 3.6 identifies the cylinder chambers, piston, seal, and rod used in the actuator model.

The 4/3 valve in Figure 3.5 meters flow to the two chambers of each asymmetric double-acting cylinder, cap side (p_1, Q_1) and rod side (p_2, Q_2). The chamber pressures follow the standard continuity relations [Merritt 67, Jelali 03]

$$\dot{p}_1 = \frac{E_{\text{eff}}}{V_1(x)}\big(Q_1 - A_1\dot{x} - C_{\text{leak}}(p_1 - p_2)\big), \qquad \dot{p}_2 = \frac{E_{\text{eff}}}{V_2(x)}\big(-Q_2 + A_2\dot{x} + C_{\text{leak}}(p_1 - p_2)\big), \quad (3.4)$$

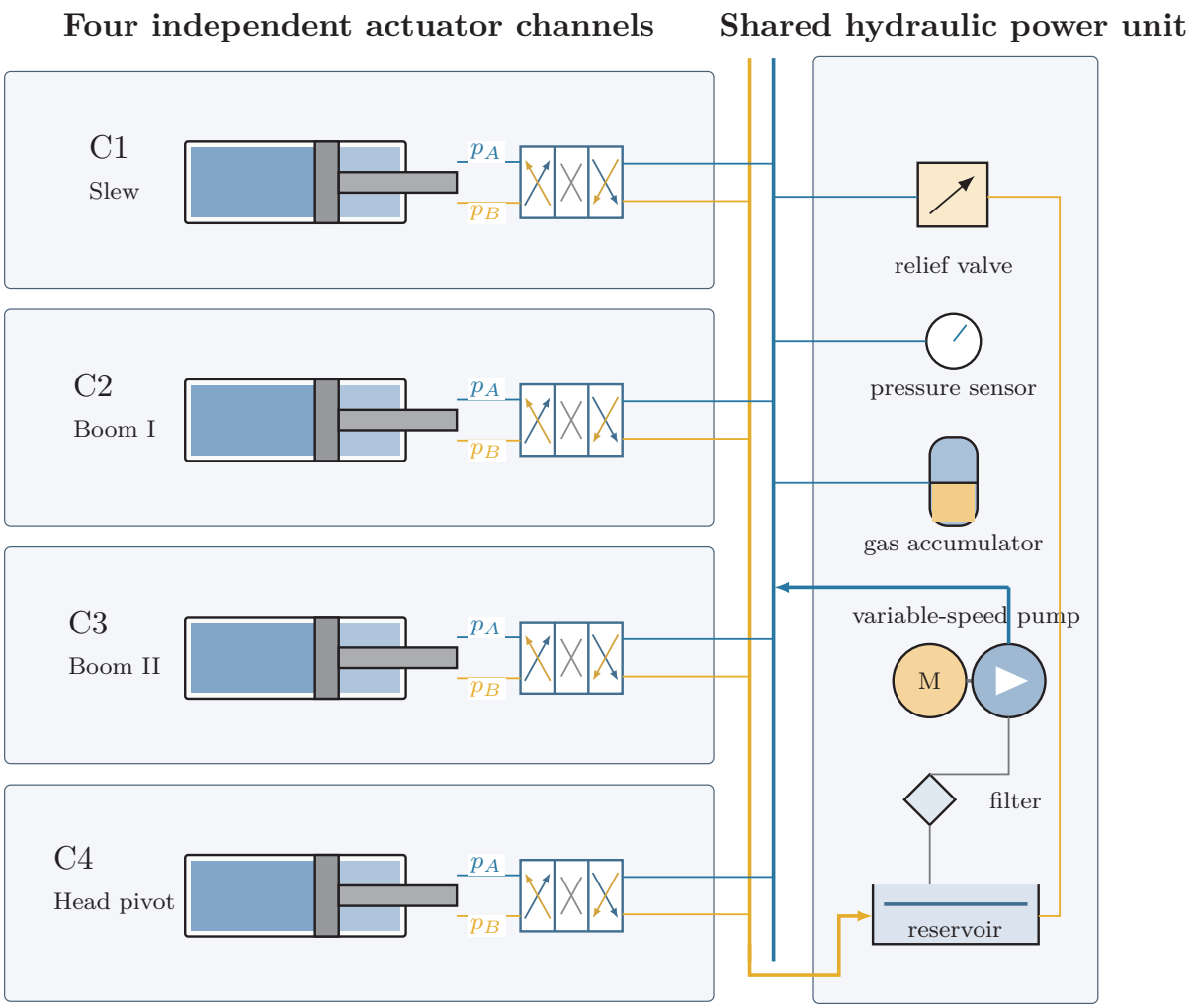

Figure 3.5: Hydraulic system schematic. Four proportional-valve cylinder channels share a variable-speed pump, reservoir, accumulator, sensor, and pressure-relief valve. Blue and orange rails denote supply and return.

where $V_{1,2}(x)$ include dead volumes, $A_1 \neq A_2$, and C_{leak} is internal leakage. Entrained air is represented by the usual effective bulk modulus [Jelali 03]

$$E_{\text{eff}}(p) = \left(\frac{1}{E_f} + \frac{\alpha}{p} \right)^{-1}, \tag{3.5}$$

Valve flow follows the orifice law of Section 2.1.2; spool dynamics are $\dot{x}_v = (u - x_v)/\tau_v$. The piston force is $F_h = p_1 A_1 - p_2 A_2 - F_{\text{fric}}(\dot{x})$. The coupled model is stiff and uses a variable-step implicit solver, which prevents RL-time simulation.

3.6.3 Scope of the Source Model

The source model provides command-dependent hydraulic behaviour within the recorded operating range. It is not validated against measurements from the physical FME 500. Its stroke coverage is limited by the assembled linkage, and the absolute pressure level is not validated. These limits apply to every model identified from its data. The behavioural checks and their implications follow below.

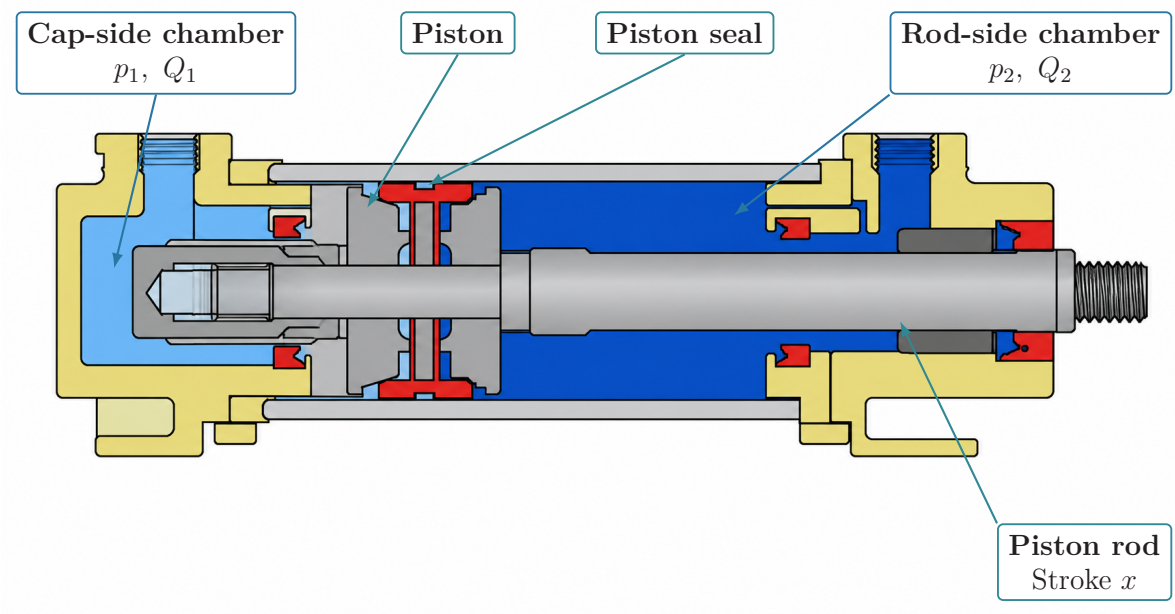

Figure 3.6: Annotated cutaway of an asymmetric double-acting hydraulic cylinder. The cap-side and rod-side chamber pressures generate the piston force and rod stroke. The labels are the states of (3.4), and the four cylinder channels of Figure 3.5 are of this type.

Logged rod force receives zero supervised weight in the ROM because it fails the piston-area consistency check and PhysX generates force internally at deployment. Cylinder displacement is the quantity passed to the position drives.

3.6.4 Source-Model Checks

The preceding master project tested bulk-modulus sensitivity under a fixed integral-backstepping controller at 0.01 s. Sweeping $\beta \in \{0.5, 1.0, 1.5, 2.0\}$ GPa reduced slew-axis delay from about 0.85 s to 0.65 s and the system delay index from 0.45 s to 0.35 s. Task-space tracking changed by about 11 %, with improvement saturating beyond 1.5 GPa, consistent with the compressibility term of Equation (3.5).

The present export audit checks the assembled model against the collector's declared settings, using four open-loop runs and 25 600 rows at 100 Hz, of which 93.7 % are valid (Table 3.4). Timing, motion direction and load dependence are consistent with hydraulic actuation. This is an internal behavioural check, not hardware validation.

Two limits remain. Recorded stroke covers only 13 %–34 % of the configured range, with the head pivot least covered. Differential pressure clips at 116 MPa–122 MPa despite the configured 27.6 MPa relief setting, so the absolute level is not validated. Standardisation permits the learned networks to use its variation, but does not make it a calibrated pressure measurement. The ROM also uses pressure in its explicit equations; the inconsistent level may contribute to the clamped parameter fit discussed in Section 4.4.2. No result establishes physical pressure accuracy or identification of the machine's hydraulic constants.

Table 3.4: Behavioural checks on the strict open-loop excitation export against the parameters the collector configures. Values are pooled over the four runs; ranges span the four cylinders. The last two rows are the limits of the validated scope, not passes.

Property	Measured	Expected
Motion onset after command onset	within two log samples, $\leq$ 20 ms	the nominal 8 ms spool constant plus transport
Rise to 63 % of peak velocity	30 ms–80 ms	spool constant plus chamber fill
Velocity sign follows command	100 % above 0.05 m/s, $\geq$ 95.8 % above 0.01 m/s	no inversion once the cylinder moves
Speed against rising load at $\lvert u\rvert > 0.5$	falls by two orders of magnitude across $\lvert\Delta p\rvert$ terciles	orifice law: flow follows the pressure margin
Stroke visited	13 %–34 % of the configured stroke	bounded by the assembled linkage, not by the valve
$\lvert\Delta p\rvert$ ceiling	116 MPa–122 MPa, repeatable per channel	relief valve set at 27.6 MPa

3.7 IsaacLab Environment

Policy training, adaptation-data collection, and evaluation run in IsaacLab on PhysX. IsaacLab is the successor to Orbit, and the Orbit paper [Mittal 23] remains its citable description; the underlying GPU-parallel simulator is Isaac Gym [Makoviychuk 21]. The USD articulation contains the Unimog carrier, five arm links, and four prismatic cylinder joints. Learned actuator models supply the hydraulic behaviour between the controller and PhysX cylinder targets.

3.7.1 The Deployment Plant

Section 3.1.3 gave the reason the command ROM plus PhysX is the deployment plant, not one of the alternatives. What follows from that choice for the three stacks is:

- IF drives that plant with the independently identified inverse and forward pair, which has never seen it. It is the stack under test.
- R uses the deployment plant itself as its actuator and is therefore the calibrated reference condition, not a guaranteed performance bound.
- AP drops the inverse model and has the policy emit the valve command directly. Its forward model is frozen like IF's; the adaptation enters at the actor input as an estimated payload, not on the forward model (Section 5.7.2).

The R–IF comparison measures the effect of actuator representation on two separately trained controllers. It is not a general ranking of model classes.

3.7.2 Simulation and Control Rates

Three rates govern the deployed loop. PhysX integrates at 120 Hz and the policy acts at 60 Hz under a decimation of two. The learned actuator models update every three policy steps, on the 20 Hz grid at which the identification data were sampled, and their prediction is held across the intervening PhysX updates. Closed-loop data collection uses that same 50 ms grid. "The control frequency" is therefore not a well-defined quantity for this system and no scalar rate is reported for it. Table A.3 summarises these rates and distinguishes the actuator-model timing from the deployed AP estimator.

3.7.3 Environment Configuration Details

The arm root is spawned at $z = 1.26\,\mathrm{m}$; ground-height spawning buries the vehicle. Cylinder targets are integrated in reference space so gravity does not erase commanded increments. Vehicle motion is scripted because the authored tyres and suspension do not provide reliable PhysX traction; a planar fixture holds lateral position and heading.

3.8 Experimental Task Specification

Section 3.1.1 gave the purpose of moving verge following. This section fixes only the task dimensions and the feasibility bound needed by the control and evaluation chapters; the campaign design and rendered environment belong to Chapter 7.

3.8.1 Task Definition

The truck drives a 26 m strip at a speed the experimental protocol sets per campaign (Section 7.1.1). Episodes allow 2808 steps (46.8 s), while scoring stops when the strip is completed. The arm must keep the cutter at the target height, on the verge centre line, in contact with the ground, and within its travel limits. Terrain height is randomised per environment.

Eleven reference-path families span straight, periodic, spline, sweep, and lane-change motion. A regression test verifies that all eleven families are distinct. Which of them and which terrain meshes a given campaign draws on, and how results are aggregated over them, belongs to the experimental design (Section 7.1.1).

3.8.2 Task Feasibility and Plant Capability

The cylinder-increment action space limits cylinders 1–3 to 4 mm per policy step, which bounds lateral cutter speed at

$$v_{\mathrm{lat}}^{\max} = \frac{4\,\mathrm{mm}}{1/60\,\mathrm{s}} = 0.240\,\mathrm{m/s}. \tag{3.6}$$

This uses the 1/60 s policy interval and bounds the commanded rate, which is what the reference paths are designed against. Early path settings exceeded this limit; the worst case (`boom_sweep`) required 3.7× the available rate. Path and terrain parameters are now derived from (3.6), and a preflight check rejects infeasible tasks.

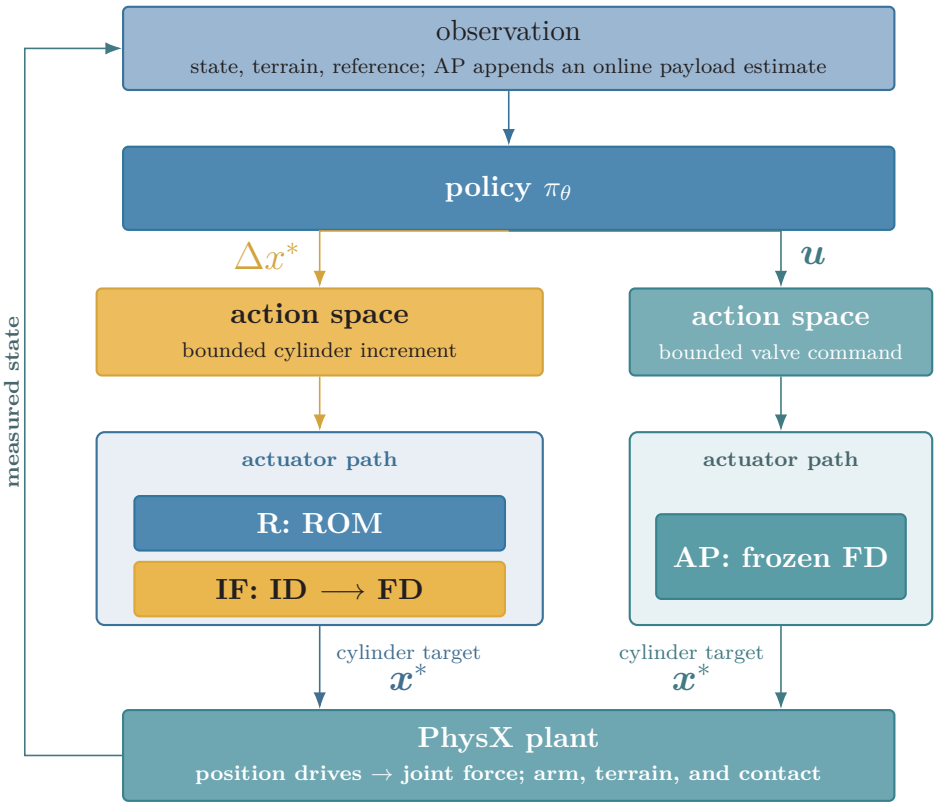

Figure 3.7: Deployed control stacks. R and IF share a cylinder-increment action space and differ only in how they realise it. AP instead uses a valve-command action space, which the frozen forward model maps to cylinder motion. AP's actuator path carries no adaptation; the adaptation is the payload estimate appended to the policy's input. All three supply the same cylinder-position target to PhysX; its position drives convert target error into joint force, so the policy does not command torque or force directly. Measured state closes the loop. The ROM block includes its frozen calibration head. AP is assessed against its own frozen base (Section 7.1.2).

Equation (3.6) is a property of the cylinder-increment control interface, so it describes the action chains of R and IF directly. AP uses a valve-command action space instead, and the same arithmetic does not apply to it. The paths and terrain derived from the ceiling are nonetheless shared by all three stacks, so what the task *demands* is identical in every case; what differs is only the space in which the request is bounded.

3.9 Control Architecture

Section 3.2 placed the three stacks inside the wider pipeline. Figure 3.7 resolves the deployment stage into the signal path each stack actually runs, which is where the one structural difference between them becomes visible: R and IF share an action space, whereas AP uses a different one.

The vehicle advances at the scripted speed and the reference generator supplies the verge centre line and cut height. The policy maps normalised observations to four bounded actions and has no runtime baseline controller. The DLS controller is used only for teaching and identification.

Only the extension increment crosses from a learned model into PhysX, integrated into a cylinder target: no force is applied, and the joint force is generated by the PhysX solver. A force interface was not available, because the logged rod force fails its consistency check (Section 6.10) and the ROM's force balance is degenerate at its fitted constants (Section 4.4); the cost is that a cylinder target discards pressure, which is where plant identity turns out to live.

3.10 Problem Formulation

The reference is the verge centre line $y_{\mathrm{ref}}(t)$ and cut height h_{ref}, expressed in the truck frame. With cutter-tip position $\boldsymbol{p}_c(t)$ and terrain height $g(\cdot)$, the scored errors are

$$e_{\mathrm{lat}}(t) = p_{c,y}(t) - y_{\mathrm{ref}}(t), \qquad e_{\mathrm{cut}}(t) = \Big(p_{c,z}(t) - g(\boldsymbol{p}_c(t))\Big) - h_{\mathrm{ref}}. \tag{3.7}$$

The objective is to minimize the mean and tail of both errors across path families and terrain while maintaining contact, head alignment, and smooth motion. Learned models are evaluated on one-step and recursive cylinder predictions. The stacks are tested across the randomised plant family. R and IF are compared with each other on held-out factors, and AP is compared with its own frozen base under a plant change, for the reasons given in Section 5.9.2.

Constraints are cylinder limits and headroom (Table 3.2), the control-interface bounds, checkpoint output limits, and the head-down condition.

3.11 System-Level Evaluation Criteria

Controllers and models are evaluated separately. Controller scores can hide actuator-model errors through policy compensation.

The system-level criteria cover three aspects of operation. Cut-height and lateral errors measure task accuracy. Contact, head alignment and impact rate describe the tool's interaction with the terrain, while cutter acceleration describes motion quality. Cylinder headroom and safety terminations check whether the task is completed without driving the mechanism into its limits. The metrics are evaluated over the scored part of each episode and are kept in physical units where possible. This makes a failed gate traceable to a specific limitation rather than to a single combined reward value.

3.11.1 The Eight System-Level Evaluation Gates

Table 3.5 defines the eight physical gates used throughout this thesis. Their role is to supply a criterion that is *not* the reward: a policy is optimised against the scalar return of Section 5.2.5 but selected and promoted against this vector, and Section 5.3.2 shows the two point in measurably different directions. They rank and promote candidates; they do not certify a stack as accepted, and no such claim is made anywhere here.

All stacks are evaluated under the same path, terrain and randomisation conditions. A candidate is preferred when it satisfies more gates at a wider margin, and strong mean tracking does not compensate for loss of contact, inadequate headroom, excessive impacts or safety terminations. Worst-case aggregation stops easy terrain or path cases from

Table 3.5: Eight system-level evaluation gates. Each gate uses at least three terrain meshes and the worst mesh; tracking-critical gates also use the worst path family. The last column states where each number comes from, because a threshold read off an earlier baseline does not carry the same weight as one fixed by the task or by the machine.

Gate	Metric	Threshold	Aggregation	Origin
Cut height	mean $\lvert e_{\mathrm{cut}} \rvert$	$\leq 0.040\,\mathrm{m}$	worst mesh	mowing spec
Lateral tracking	mean $\lvert e_{\mathrm{lat}} \rvert$	$\leq 0.300\,\mathrm{m}$	worst mesh, worst family	earlier baseline
Ground contact	contact fraction	≥ 0.700	worst mesh, worst family	mowing spec
Head alignment	p01 head-down alignment	≥ 0.900	worst mesh	tool geometry
Hard impacts	hard-impact step fraction	≤ 0.350	worst mesh	wear tolerance
Tool acceleration	p95 cutter acceleration	$\leq 120\,\mathrm{m/s^2}$	worst mesh	engineering tolerance
Cylinder headroom	p01 travel margin	$\geq 0.005\,\mathrm{m}$	worst mesh	machine limit
Safety terminations	termination rate	≤ 0.080	worst mesh	engineering tolerance

masking a local failure, the percentiles catch short alignment, clearance and acceleration events a mean hides, and contact and termination measures are fractions so episodes of different completed length stay comparable.

Cut height and ground contact follow the mowing task; head alignment and cylinder headroom follow tool geometry and travel limits. Acceleration and termination thresholds are engineering tolerances. The lateral threshold was set using an earlier trajectory library and is retained for consistent comparison, not as a task requirement. Section 7.2 reports the resulting per-family failures.

3.11.2 Cylinder and Model Checks

Cylinder checks use fresh $\pm 10\,\mathrm{mm}$ commands, coordinated $80\,\mathrm{mm}$ motions, return error, and saturation. Model checks use one-step and rollout accuracy, input ablation, and Jacobian conditioning (Chapter 4). Chapter 7 reports these with the evaluation contract. The small command checks local response and deadband; the coordinated motion checks coupling between actuator channels. Return error tests reversibility, and saturation checks whether the learned model respects the command and stroke limits. These diagnostics are interpreted alongside the system gates, not as substitutes for closed-loop task performance.

The system is now specified. The next chapter turns to the learned actuator models built on it. The datasets those models are identified from are described in Chapter 6, after the controllers that consume them, so that the purpose of each collection is known before its construction is set out.

4. Learned Models and Online Adaptation

This chapter defines the learned components used for control: the inverse and forward models in IF, the command-conditioned ROM in R, and the online payload estimator in AP. It specifies their inputs, outputs and training objectives. All actuator models are frozen during policy training and deployment, so every number reported later is produced by a fixed representation. Chapter 6 documents the collections they are identified from.

Each model is scored immediately after it is defined, so the evidence for a representation sits beside the representation itself. Those checks are offline. They use held-out runs and measure prediction, transfer and command sensitivity rather than task performance. One result recurs across all three models: a high supervised score does not by itself establish that a model responds to the command it is given, and each section therefore states what its numbers support and what they leave open. The closed-loop consequences are deferred to Chapter 7.

4.1 Modelling Overview and Common Notation

All learned models act on the four hydraulic cylinders of the MULAG arm, indexed $i \in \{1, \dots, 4\}$ in the fixed order slew cylinder (*Drehzapfen*), boom cylinder I (*Ausleger I*), boom cylinder II (*Ausleger II*) and head pivot cylinder (*Messerkopf-Schwenk*). The passive fifth joint is not actuated and enters no model input. At the learned-model sample time $T = 0.05\,\mathrm{s}$ the discrete index k refers to the cylinder extension $\boldsymbol{x}_k$, the extension velocity $\dot{\boldsymbol{x}}_k$, the chamber pressure difference $\Delta\boldsymbol{p}_k$ and the logged rod force $\boldsymbol{F}_k$, each in $\mathbb{R}^4$. The valve command $\boldsymbol{u}_k \in [-1, 1]^4$ is the normalised spool command of the four proportional valves. The desired cylinder command produced by the task controller is the pair

$$\boldsymbol{c}_k = \left[\Delta\boldsymbol{x}_k^*,\ \Delta\dot{\boldsymbol{x}}_k^*\right] \in \mathbb{R}^8, \tag{4.1}$$

the extension and extension-velocity increments to be realised over the next sample. Predictions carry a hat, and $\Delta(\cdot)_k = (\cdot)_{k+1} - (\cdot)_k$ denotes a within-run one-step increment throughout. Figure 4.1 shows the interface the two actuator paths share.

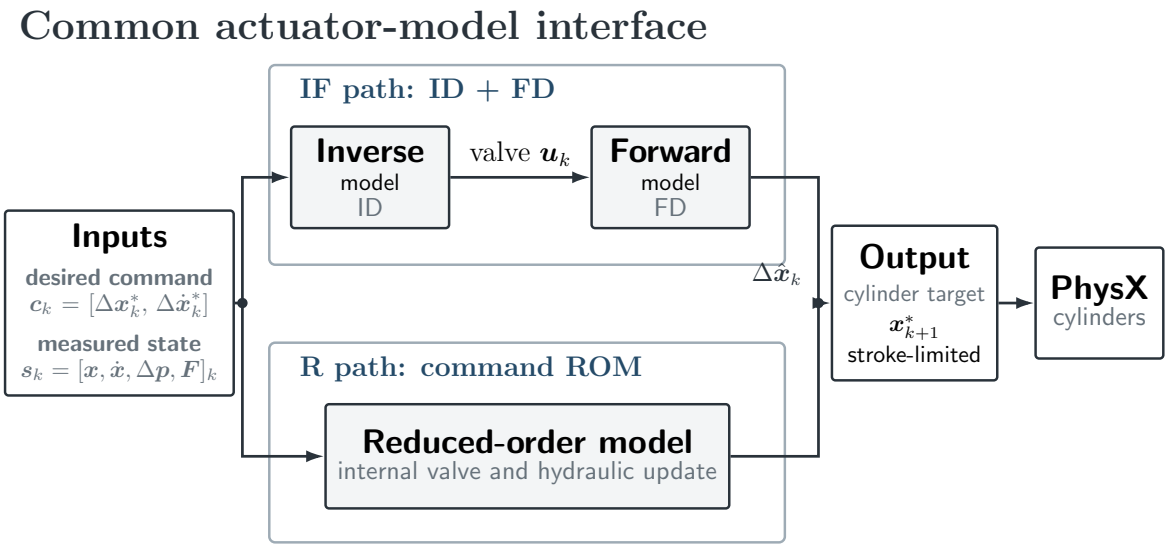

Figure 4.1: Common interface of the two alternative actuator paths. Both receive the desired cylinder command and measured hydraulic state, and both produce the same bounded PhysX cylinder target. The IF path composes the inverse and forward models; the R path uses one command-conditioned reduced-order model.

Table 4.1: Trainable parameters of the three actuator models, counted from the selected checkpoint state dictionaries.

Model	Role	Architecture	Parameters
ID	valve-command increment	512-wide, 4-block residual MLP	2 124 292
FD	extension, velocity, pressure	512-wide, 4-block residual MLP	2 124 300
Command ROM	command-conditioned actuator	GRU 128, 2 layers, hydraulic chain	196 220

Three conventions apply throughout. Dynamic targets are within-run increments, with $\Delta\hat{\boldsymbol{x}} = \mathbf{0}$ as the persistence baseline. Each checkpoint uses its own training-set normalisation. Inputs at step k are causal, and targets are measured transitions to step $k + 1$.

Simscape supplies the identification data of Chapter 6. The exported arrays are audited, frozen and used to train the models in PyTorch [Paszke 19]; Simscape itself is part of neither training nor deployment.

Table 4.1 lists the network sizes. The closed-loop Simscape export supplies the initial ID, FD and command-ROM training data. FD is subsequently fine-tuned on open-loop excitation and ROM receives a bounded IsaacLab calibration. Their separate provenance is recorded in Table 3.1.

4.2 Inverse Dynamics Model

The ID network maps the current hydraulic state and desired cylinder increment to a valve-command change:

$$\Delta \boldsymbol{u}_k = f_{\mathrm{ID}}\Big(\boldsymbol{x}_k,\, \dot{\boldsymbol{x}}_k,\, \Delta \boldsymbol{p}_k,\, \boldsymbol{F}_k,\, \Delta \boldsymbol{x}^*_k,\, \Delta \dot{\boldsymbol{x}}^*_k,\, \boldsymbol{u}_{k-1}\Big) \in \mathbb{R}^4, \tag{4.2}$$

a 28-dimensional input. The deployed command is $\boldsymbol{u}_k = \mathrm{clip}(\boldsymbol{u}_{k-1} + \Delta \boldsymbol{u}_k,\, -\boldsymbol{u}_{\max},\, \boldsymbol{u}_{\max})$ with per-axis limits $\boldsymbol{u}_{\max} = [0.95,\, 0.95,\, 0.8,\, 0.8]$ stored in the checkpoint. The change formulation retains the previous valve operating point near the direction-dependent dead zone (Section 2.1.2). Logged achieved transitions serve as desired increments because no independent cylinder-space reference was recorded.

The 512-wide residual MLP [He 16] uses SiLU activations, layer normalisation and dropout [Elfwing 18, Ba 16, Srivastava 14]. Adam minimises a smooth-L_1 loss on normalised targets, with saturated samples up-weighted [Kingma 15, Huber 64].

Figure 4.2 separates the three input groups and shows the four residual blocks explicitly. The network predicts a four-axis valve-command increment, which is accumulated and clipped before use.

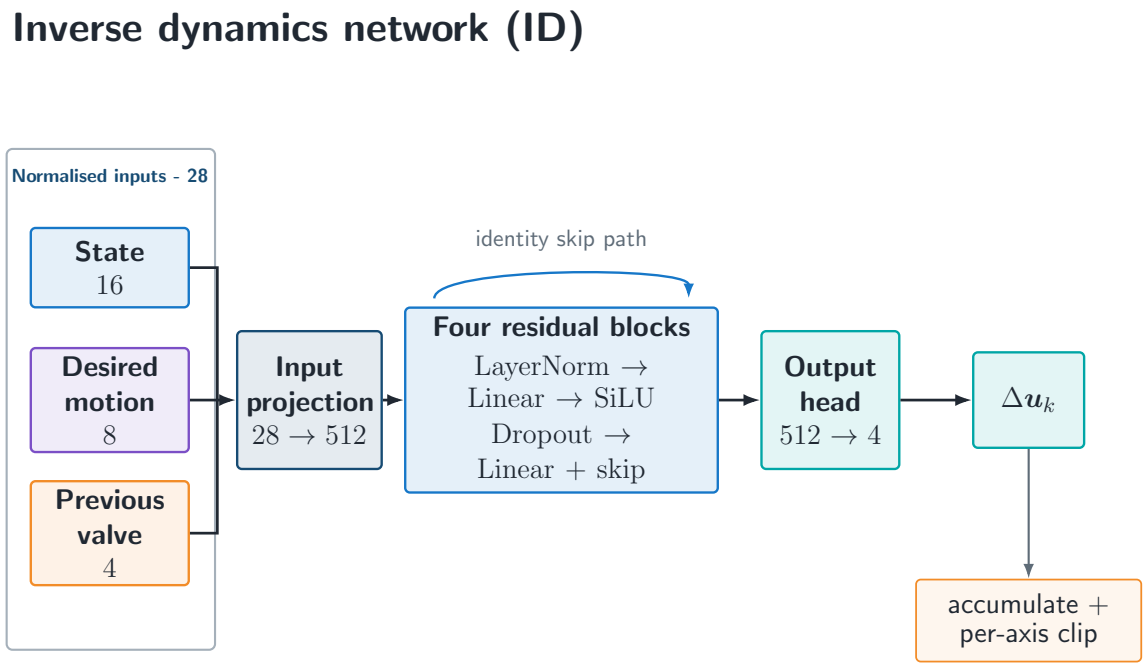

Figure 4.2: Inverse-model architecture. Current hydraulic state (16), desired motion (8) and the previous valve command (4) form the 28 inputs. A 512-wide, four-block residual MLP predicts $\Delta \boldsymbol{u}_k$, which is accumulated and clipped per axis to form the bounded valve command.

4.2.1 Paired Control Fine-Tuning

The composite is assessed by the *paired differential response*: the FD prediction under the ID command for the requested target minus the prediction under its command for a zero target,

$$\begin{aligned} \boldsymbol{u}_k^{\mathrm{tgt}} &= f_{\mathrm{ID}}(\cdot,\, \boldsymbol{c}_k,\, \boldsymbol{u}_{k-1}), \qquad \boldsymbol{u}_k^{0} = f_{\mathrm{ID}}(\cdot,\, \mathbf{0},\, \boldsymbol{u}_{k-1}), \\ \Delta\hat{\boldsymbol{x}}_k^{\mathrm{pair}} &= f_{\mathrm{FD}}\Big(\boldsymbol{u}_k^{\mathrm{tgt}}, \cdot\Big) - f_{\mathrm{FD}}\Big(\boldsymbol{u}_k^{0}, \cdot\Big). \end{aligned} \tag{4.3}$$

The networks are fine-tuned jointly for 30 epochs using $\mathcal{L}_{\mathrm{ctl}} = \|\Delta\hat{\boldsymbol{x}}^{\mathrm{pair}} - \Delta\boldsymbol{x}^{*}\|^2_{\mathrm{norm}} + \lambda_{\mathrm{ID}}\mathcal{L}_{\mathrm{ID}} + \lambda_{\mathrm{FD}}\mathcal{L}_{\mathrm{FD}}$. The supervised terms are retained, while target blocks are shuffled across valid states to break the logged state–target pairing. The resulting ID checkpoint is frozen for IF. AP has no inverse model. Section 4.2.2 below reports the measured effect.

4.2.2 Inverse Model and Paired Control Response

Per-valve validation R^2 of 0.9973–0.9996 shows that ID reproduces the logged commands, but does not establish control authority. Before paired fine-tuning, the composite normalised control error was 1.0011, essentially the zero-response baseline. After 30 epochs it fell to 0.1086, with every response slope above 0.8 and every correlation above 0.92 (Table 4.2).

Table 4.2: Effect of paired control fine-tuning on the composite differential response (4.3), per cylinder axis on validation data. Before fine-tuning the composite had essentially no control authority despite near-perfect supervised scores of both networks; the composite normalised control error fell from 1.0011 to 0.1086.

	Regression slope		Correlation	
Cylinder axis	before	after	before	after
1 (slew)	0.003	0.818	0.135	0.920
2 (boom I)	0.002	0.928	0.090	0.967
3 (boom II)	0.001	0.849	0.037	0.933
4 (head pivot)	0.004	0.912	0.249	0.958

4.2.3 Transfer of the Frozen Inverse Model

IF drives the deployment plant through this pair, so its transfer was measured on all 118 frozen runs of Section 6.7, comprising 2 nominal, 40 hydraulic-only, 12 mechanical-only and 64 combined runs, of which 16 are held-out combined-test. These runs lie outside the ID's Simscape-only training distribution and are recorded in the plant the adaptive controller is closed around. Two regimes were scored: a teacher-forced one-step transfer supplying the measured cylinder state at every tick, and a closed-loop transfer feeding back the pair's own prediction while supplying the recorded reference. Pressure and force are held at their training means, as the deployed feature contract requires.

Teacher-forced, the pair transfers well and evenly (Table 4.3). One-step extension error is millimetre-scale in every domain and nearly flat across them, from 2.36 mm on the

Table 4.3: Transfer of the frozen inverse and forward pair across deployment domains, all 118 runs of the deployment collection. The one-step columns are teacher-forced; the saturation column is the fraction of pre-clip valve elements at the limit during closed-loop transfer. The ROM valve recorded on these same runs never saturates.

		one-step Δx error		closed-loop
Domain	runs	RMSE	worst run	valve saturation
combined	64	2.36 mm	5.72 mm	0.888
hydraulic	40	2.43 mm	4.96 mm	0.885
mechanical	12	2.81 mm	4.14 mm	0.919
nominal	2	3.36 mm	4.02 mm	0.929
all	118	—	—	0.891

combined runs to 3.36 mm on nominal, and the held-out combined-test group is no worse than the domains used in training, at 2.34 mm. This is the regime in which IF uses the pair, because the policy closes the loop at every tick and never asks it to predict its own future.

Closed-loop, it does not transfer at all, and the failure is uniform rather than occasional. Every one of the 118 runs leaves the recorded reference by more than one metre, after about 32 s on the combined and hydraulic domains and sooner on the harder ones. The mechanism is saturation: the pair's pre-clip valve command sits at its limit on 89.1 % of elements over the collection, between 88.5 and 92.9 % by domain, while the ROM valve recorded on those same runs never reaches its bound at all. For scale, the measured plant tracks the recorded cylinder reference at about 0.807 mm on the held-out group, so the closed-loop divergence is a failure of model transfer and not a property of the collection.

Two consequences follow. The pair cannot be promoted as an unadapted autoregressive deployment path, because its own predicted state leaves the support its training data covered, though nothing in that result disturbs the one-step use IF makes of it. The transfer claim is therefore limited to the measured one-step regime.

4.3 Forward Dynamics Model

The FD network predicts the one-step cylinder response to a valve command,

$$\left[\Delta\hat{\boldsymbol{x}}_k,\ \Delta\hat{\dot{\boldsymbol{x}}}_k,\ \Delta\widehat{\Delta\boldsymbol{p}}_k\right] = f_{\mathrm{FD}}\left(\boldsymbol{u}_k,\ \boldsymbol{x}_k,\ \dot{\boldsymbol{x}}_k,\ \Delta\boldsymbol{p}_k,\ \boldsymbol{F}_k\right) \in \mathbb{R}^{12}, \tag{4.4}$$

using the same residual trunk as the ID network. Supervised pretraining on the closed-loop export uses an auxiliary recursive rollout loss [Nagabandi 18].

Figure 4.3 shows the forward model separately because its input and output contracts differ from the inverse model despite the shared residual trunk.

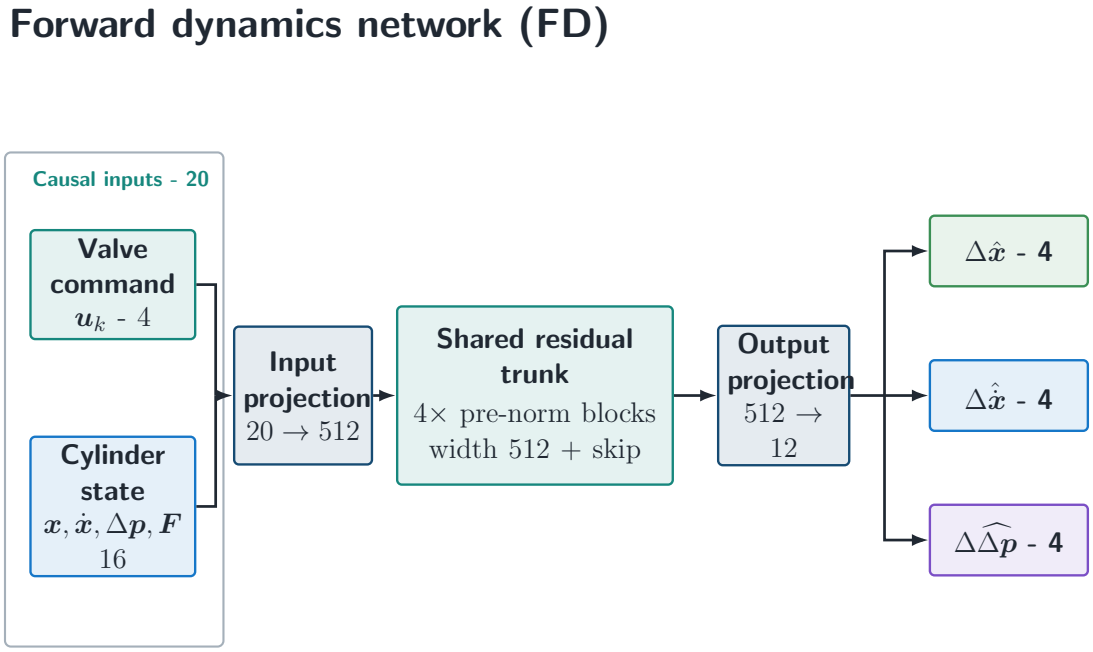

Figure 4.3: Forward-model architecture. The applied valve command and cylinder state condition a one-step hydraulic transition through a 512-wide, four-block residual MLP. Its 12 outputs are extension, velocity and pressure increments. Extension is the integrated controller output; velocity and pressure are diagnostic outputs.

4.3.1 Training Objective

Starting from the paired-control checkpoint, strict fine-tuning uses the open-loop excitation dataset of Chapter 6 and minimises

$$\mathcal{L}_{\mathrm{FD}} = \mathcal{L}^{\mathrm{strict}}_{\mathrm{Huber}} + 0.25\,\mathcal{L}^{\mathrm{MAT}}_{\mathrm{Huber}} + 2.0\,\mathcal{L}^{(10)}_{\mathrm{roll}} + 10.0\,\mathcal{L}_{\mathrm{kin}}, \tag{4.5}$$

combining the one-step loss on the strict data, a replay term on the original closed-loop data, a 0.5 s recursive rollout loss, and a kinematic consistency term enforcing the trapezoidal identity [Siciliano 09]

$$\Delta \boldsymbol{x}_k = T\,\dot{\boldsymbol{x}}_k + \tfrac{1}{2}\,T\,\Delta\dot{\boldsymbol{x}}_k, \tag{4.6}$$

with $T = 0.05$ s. Deployment integrates only the predicted extension increment. Velocity and pressure are retained as diagnostic outputs; their validation is reported in Section 4.3.2.

4.3.2 Forward-Model Command Sensitivity

The supervised score did not establish valve sensitivity. At a 0.5 s horizon, zeroing the valve input changed the rollout-error ratio to only 0.94. The FD had mainly extrapolated the smooth closed-loop kinematics because command and state history were nearly redundant. Closed-loop ablation confirmed that the valve was not identifiable, so repair required new data.

The repair dataset applies rank-4 open-loop valve plateaus to four plant draws. Its trajectories are distinct, valve history predicts pressure with $R^2 \approx 0.80$–0.85, valve-regressor variance-inflation factors [Kutner 05] are 7–10, and relief saturation occupies 9–22 % of samples.

Strict fine-tuning reduces extension rollout RMSE from 75.2 mm to 12.4 mm at 0.5 s and from 157.9 mm to 27.3 mm at 1.0 s. The valve-zero error ratio rises from 0.94 to 1.81, and per-axis one-step extension R^2 improves from $[-5.7, 0.60]$ to $[0.76, 0.98]$. Zero-order hold remains competitive below 0.25 s, and velocity increments remain weak on two axes. Deployment therefore integrates only $\Delta\hat{\boldsymbol{x}}$ and retains velocity as a diagnostic output.

4.4 Reduced-Order Model

The R path uses one command-conditioned model instead of the ID–FD composition. Its structure is grey-box: physical equations propagate pressure, force, velocity and extension. Neural heads estimate valve realisation, load and bounded residuals [Karniadakis 21, Lutter 19]. Valve authority is pressure dependent by construction through the orifice-flow and continuity equations [Merritt 67]. A GRU [Cho 14] reads six samples (0.25 s) of cylinder state, previous predicted valve command and desired command. The measured valve is only an internal-head training target [Hwangbo 19].

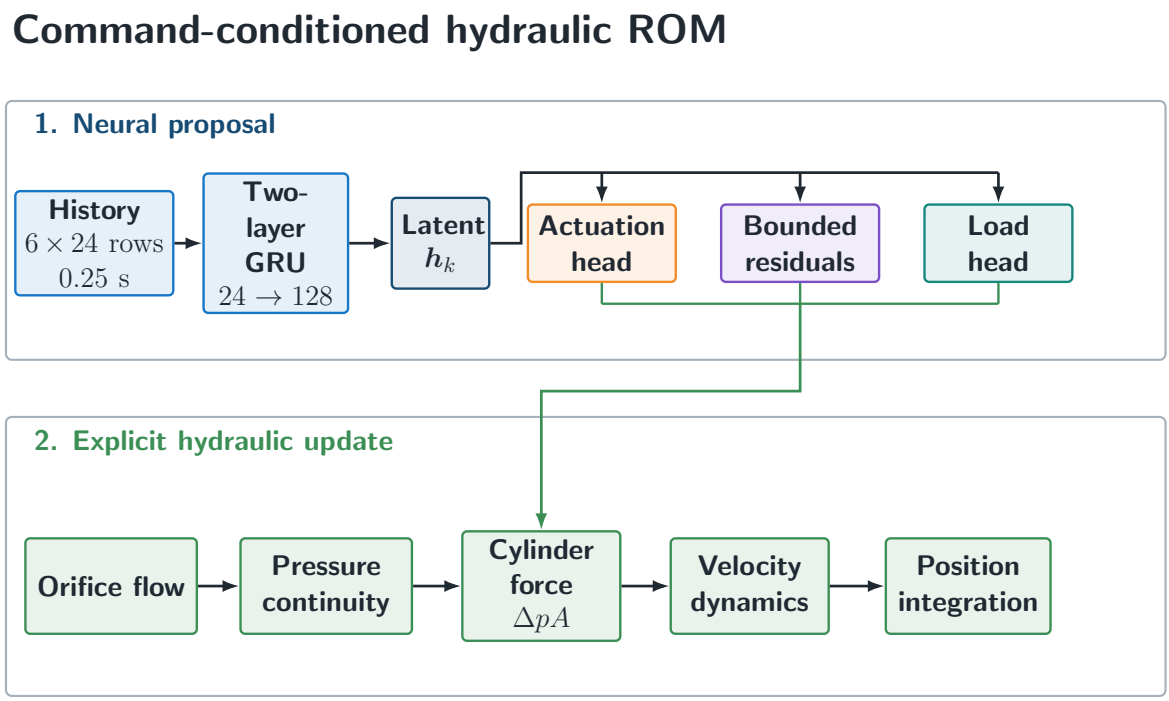

Figure 4.4: Command-conditioned ROM. A recurrent neural proposal encodes a six-step history. Its actuation, bounded-residual and load heads condition the explicit hydraulic update: orifice flow, pressure, force, velocity and position. The explicit chain constrains the predicted transition.

The actuation head of Figure 4.4 uses a learnable smooth dead zone with per-cylinder width and asymmetric gains [Yang 26]. Its output $\hat{a}_i \in [-1, 1]$ drives the orifice-flow and continuity update

$$q_i = \hat{a}_i \, k_{q,i} \sqrt{\tfrac{1}{2}\big(p_{s,i} - \operatorname{sgn}(\hat{a}_i)\, \Delta p_i\big)_+},$$
$$\Delta\hat{p}_{i,k+1} - \Delta p_{i,k} = \frac{4\beta_i}{V_i(x_i)}\big(q_i - A_i\dot{x}_i - k_{\ell,i}\,\Delta p_i\big)\,T \;+\; r_{p,i}, \tag{4.7}$$

where β_i, $p_{s,i}$, $k_{q,i}$, $k_{\ell,i}$, A_i and $V_i(x_i)$ are the effective bulk modulus, supply pressure, flow gain, leakage, piston area and chamber volume. Pressure then produces force and motion:

$$\hat{F}_i = \Delta\hat{p}_{i,k+1}A_i + r_{F,i}, \qquad \ddot{x}_i = \frac{\hat{F}_i - c_{v,i}\,\dot{x}_i - \ell_i(\boldsymbol{h}_k)}{m_i},$$
$$\dot{\hat{x}}_{i,k+1} = \dot{x}_{i,k} + \operatorname{sat}\big(T\,\ddot{x}_i + r_{v,i}\big), \qquad \hat{x}_{i,k+1} = x_{i,k} + \tfrac{T}{2}\big(\dot{x}_{i,k} + \dot{\hat{x}}_{i,k+1}\big) + r_{x,i}, \tag{4.8}$$

Here $c_{v,i}$ is viscous friction, m_i is effective modal mass and $\ell_i(\boldsymbol{h}_k)$ is the configuration-dependent load. Parameters are trained in log-space within fixed clamps and are treated as effective coefficients, not measurements of the machine. Pressure and velocity residuals are bounded to 10 % of their step limits and the position residual to 1 cm. Section 4.4.2 examines the resulting physical fit.

Training uses rollout horizons $H \in \{1, 5, 20, 50\}$ and scheduled sampling up to 75 % self-fed [Bengio 15]. Residual penalties retain the physical structure. Logged rod force has zero supervised weight because PhysX generates it at deployment and the export fails the piston-area consistency check.

4.4.1 Closed-Loop Calibration Against the Deployment Plant

The Simscape-trained ROM has a systematic extension offset when driving PhysX. The promoted checkpoint adds a bounded, command-gated correction:

$$\hat{x}_{i,k+1} \leftarrow \hat{x}_{i,k+1} + \delta_i(\boldsymbol{h}_k, \boldsymbol{c}_k), \qquad |\delta_i| \le 3\,\text{mm}, \qquad \delta_i\Big|_{\boldsymbol{c}_k=\boldsymbol{0}} = 0, \tag{4.9}$$

It is trained on 4 084 IsaacLab transitions from five reachable reference shapes, with Simscape replay weighted at 0.25. All hydraulic and integration parameters remain frozen. The gate sets the correction to zero for a commanded hold.

The calibrated ROM is frozen at deployment and has no latent input. Its correction head is internal to the ROM block in Figure 3.7; it is not an online adaptation module. ROM plus PhysX supplies the virtual reference plant (Section 3.7.1).

4.4.2 ROM Accuracy and Physical Interpretation

With the true command, mean one-step R^2 is 0.998 for the internal valve, 0.64 for velocity and 0.97 for extension. Zeroing or shuffling the command reduces velocity R^2 below zero and extension to about 0.5; the extension Jacobian is diagonally dominant with condition number 2.14. At 5 s, terminal extension R^2 is 0.998, 0.998, 0.84 and 0.91 across the cylinders. Pressure-increment R^2 is only 0.06–0.09, so the internal pressure is a dynamic

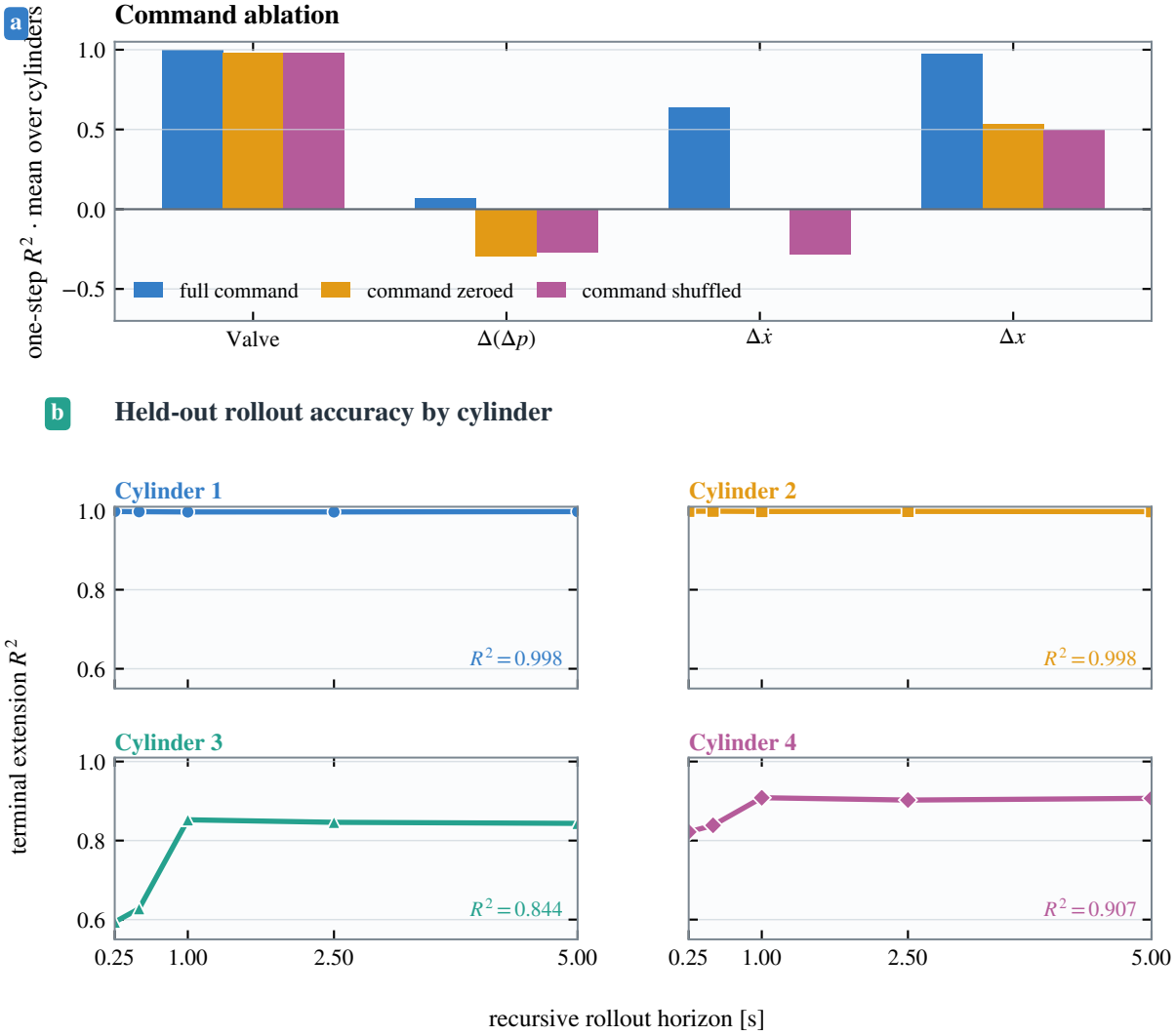

Figure 4.5: Offline validation of the command ROM on held-out runs. (a) One-step R^2 per output group, averaged over the four cylinders, with the full desired command, with the command zeroed, and with the command shuffled across the batch. The collapse of the velocity and extension scores without the true command demonstrates command authority. (b) Terminal extension R^2 of recursive rollouts up to 5 s, shown separately for each cylinder so coincident high-accuracy traces remain visible.

state, not a calibrated pressure estimate (Figure 4.5), consistent with the source Simscape pressure signal's own wrong-level ceiling (Section 3.6.3) and with the deployed pressure state carrying shape rather than level (Section 5.7.5). Any later use of this pressure channel inherits this limitation.

The ROM's fitted coefficients do not identify the physical machine. Modal mass and viscous friction reach their upper clamps of 10^7 in their respective units, piston area reaches its 1 cm^2 lower clamp from a 64 cm^2 initialisation, and supply pressure reaches either 1 MPa or 50 MPa. These values recur across the checkpoint family.

With the residual heads inactive, the explicit chain produces no net travel: velocity increments fall six orders of magnitude below nominal. At 0.1 m/s the fitted friction term reaches 1.0 MN, against about 10 kN of pressure force at 100 MPa. The learned residuals

therefore produce the displacement. Pressure remains responsive: halving leakage with the heads inactive changes it by 63 % of its range. Hydraulic parameters influence motion through the pressure state read by the recurrent encoder. The equations constrain the transition, but neither their presence nor the tracking score establishes physical parameter identification.

4.5 Offline AAM Diagnostic for the ID Path

The Arm Adaptation Module (AAM) is a diagnostic for the ID path, not a deployed component. It maps a one-second causal history of observable states and commands, arranged as 20×18 values, to a two-dimensional plant context estimate. The encoder is calibrated offline against a privileged context built from the known plant factors, and those factors and all future signals are excluded from its input. A bounded correction conditioned on the estimate was then added to the frozen inverse model's valve command.

It proved two things (Figure 4.6). Plant identity is recoverable from observable history: latent RMSE is 0.802 on validation and 0.734 on the reserved test set, and the adapter beats a zeroed latent on its own held-out objective. Delivery is what fails: in closed loop the correction improves nothing, because the command it corrects already rests on its clip for most of the scored window. That is why the deployed stack drops the inverse model from the adaptive path and conditions the frozen actor on a named payload instead (Section 4.6).

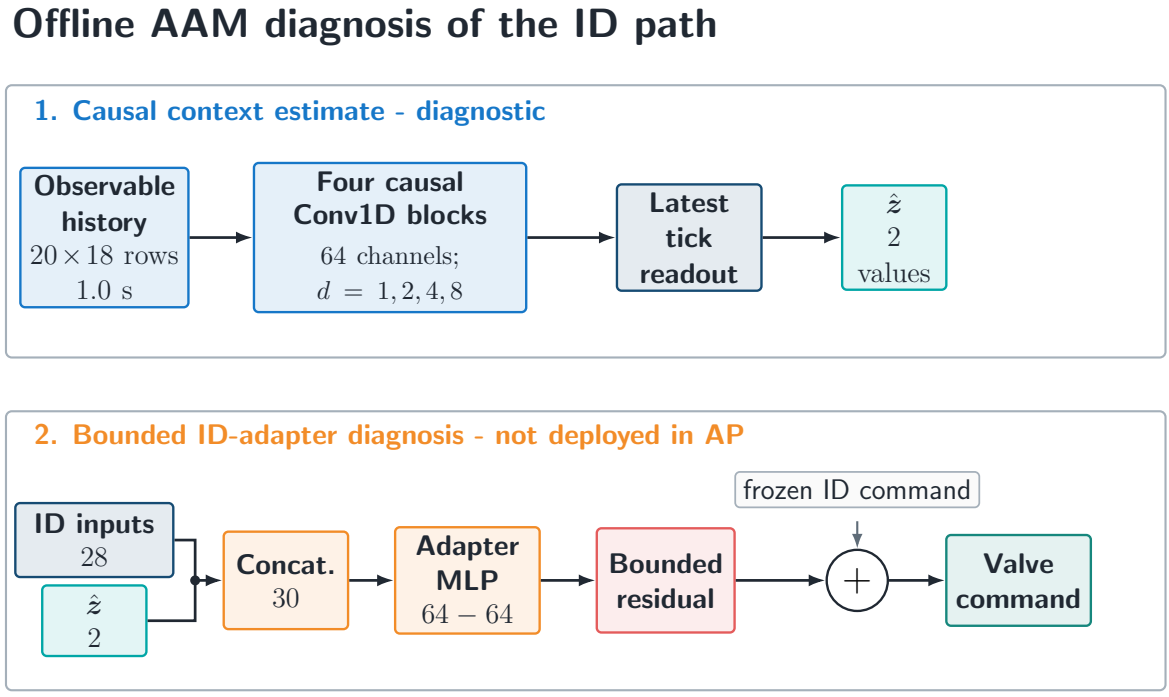

Figure 4.6: The AAM diagnosis of the ID path. Four causal convolutional blocks map a 20-step history of 18 observable features to the two-value context estimate $\hat{z}$, and the bounded adapter turns that estimate into a correction on the frozen ID valve command. Both parts are diagnostic: the estimate is recoverable, but the correction does not survive the command path, so neither is deployed in AP.

4.6 The Deployed Online Payload Estimator

The quantity the deployed controller conditions on is the cutter payload, normalised as $(m - 1)/0.5$ so that a nominal cutter maps to zero. It is estimated at runtime from the *current* observation by ridge regression on its 66 task channels, extended with their squares, standardised, and read out as a single scalar. There is no history window and no encoder. Nothing privileged enters it.

The estimator is fitted after policy training on a mixed-payload rollout of the policy it serves, with a separate fit for each training seed. The ridge regularisation is fixed across seeds and is not selected on held-out data. At runtime, an exponential moving average smooths the estimate and resets to nominal at each episode boundary. No deadband or threshold gate is applied. Zeroing the context recovers the frozen controller exactly (Section 5.7.2); the consequences of estimation error and smoothing are assessed in Section 7.5.7.

The actuator models are frozen before policy training. AP's context columns are trained under the reinforcement objective and its estimator is fitted afterwards for each seed. Chapter 5 assembles these components; Appendix A.5 records the selected artefacts.

5. Control Framework and Simulation Integration

This chapter assembles the learned components into three controllers. R and IF use the same cylinder-increment action space with different frozen actuator models. AP commands valves through the frozen forward model and conditions its actor on an online payload estimate.

The three stacks are built as one controlled experiment rather than as three independent designs. They share the observation layout, the reward, the training protocol and the evaluation gates, so that a difference measured later can be attributed to the actuator representation or to the payload conditioning, and not to tuning. The chapter specifies the privileged teacher and the interfaces the stacks hold in common, then the training protocol and the drift constraint it requires, and finally the three stacks themselves, the integration with IsaacLab and the matching rules that keep the comparison fair.

The training diagnostics belong here rather than with the results. The teacher's measured performance, a distillation run and the objective mismatch that unconstrained fine-tuning produces are what motivate the protocol, so they are reported beside it. Chapter 7 scores the finished stacks on the task.

5.1 The DLS Teacher and Its Two Roles

5.1.1 Scope and Role of the Controller

The DLS controller has two roles: it supplies privileged labels for policy distillation and drives the identification runs of Section 6.7. It is removed at deployment, when the policy alone issues commands (Figure 5.1). No nominal controller output is added to the policy action.

5.1.2 Privileged Information

The teacher reads the exact terrain height beneath the cutter and the true cutter–ground contact force from the simulator. These privileged signals are not available to the deployed student.

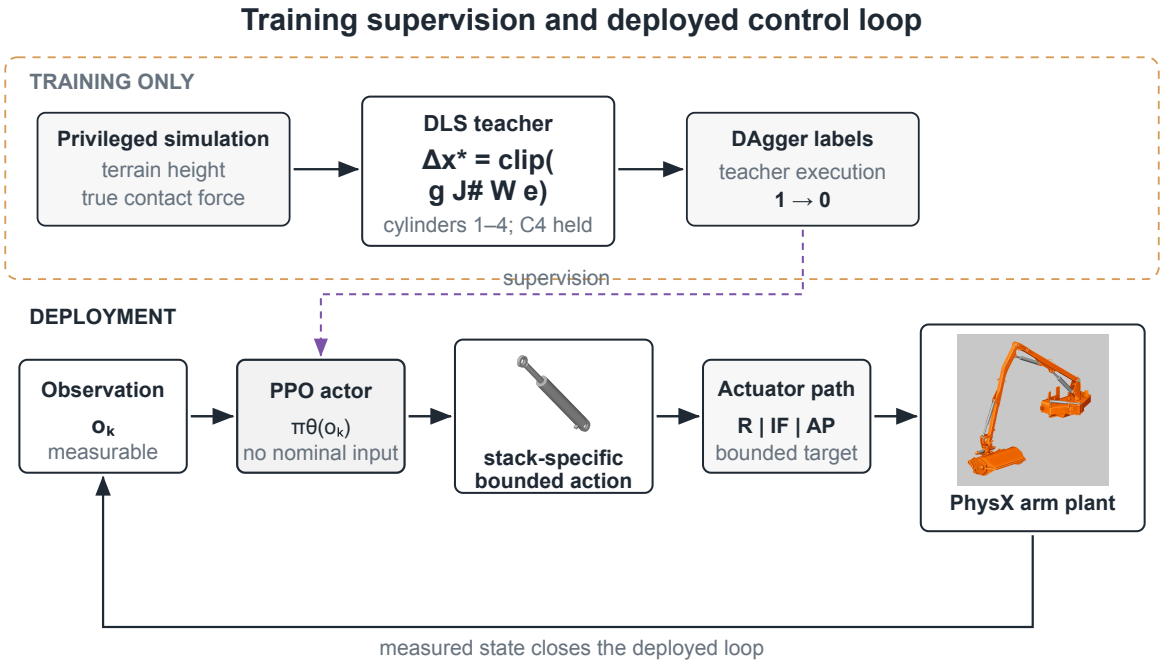

Figure 5.1: Training supervision and deployment. Simulator-only terrain and contact signals drive the DLS teacher, which supplies DAgger labels to the actor. At deployment only the lower feedback loop remains: measured observations pass through the PPO actor, the stack-specific action space and the selected R, IF or AP actuator path before PhysX updates the arm. R and IF issue cylinder increments; AP issues valve commands. No nominal command is added to the policy output.

It uses a contact regulator: it maintains a vertical offset relative to the ground and adjusts that offset to hold the measured contact force near 250 N within a 100 N deadband, pressing down below the band, backing off proportionally above it, searching downward when contact is lost and retreating immediately on a force spike. The accumulated offset is clamped to $[-0.30, -0.05]$ m so the regulator can neither drive the head into the ground nor lift it clear.

5.1.3 From Task Error to Cylinder Command

A goal point is maintained in the truck frame: laterally it tracks the verge centre line, vertically it follows the measured ground height plus the contact regulator's offset. The task error is the difference between that goal and the measured cutter position, with the lateral component weighted by a factor of three, because on this machine lateral error is what the acceptance gate measures and vertical error is largely handled by contact.

That error is mapped to cylinder space through the damped least-squares inverse of the measured cutter Jacobian $\boldsymbol{J}_c$ of Section 3.4.2 [Wampler 86, Nakamura 86],

$$\begin{aligned} \boldsymbol{J}_c^{\#}(\lambda) &= \boldsymbol{J}_c^{\mathsf{T}} \left(\boldsymbol{J}_c \boldsymbol{J}_c^{\mathsf{T}} + \lambda^2 \boldsymbol{I} \right)^{-1}, \\ \Delta \boldsymbol{x}_{1:3}^{*} &= \operatorname{clip}\Big(g\, \boldsymbol{J}_c^{\#}(\lambda)\, \boldsymbol{W} \boldsymbol{e},\ \pm \Delta x_{\max} \Big), \quad g = 0.30, \quad \Delta x_{\max} = 3\,\text{mm}. \end{aligned} \tag{5.1}$$

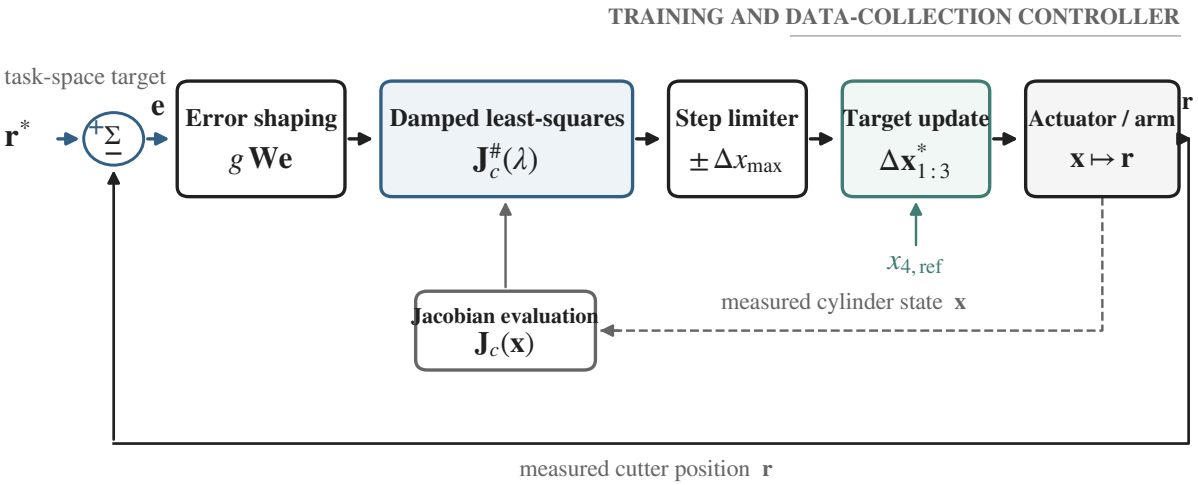

Figure 5.2: Feedback realisation of the damped least-squares teacher of (5.1). The weighted task error is scaled by g, mapped to the first three cylinder coordinates by the measured damped inverse $\boldsymbol{J}_c^{\#}(\lambda)$, and clipped to $\pm\Delta x_{\max}$ per tick. Cylinder 4 is held at its head-down reference. Cutter position closes the task-space loop and the measured cylinder state updates $\boldsymbol{J}_c$ at every step. The loop is used only for teaching and data collection, never at deployment.

with $\boldsymbol{W}$ the lateral weighting and the per-step clamp bounding how far any cylinder may be asked to move in one tick. Only cylinders 1–3 are solved. Cylinder 4 is held at the reference extension that keeps the head down, for the reason given in Section 3.3. The gain g is small on purpose: this is a stiff, heavy mechanism in contact, and a resolved-rate controller that closes quickly on the task error simply excites it. Figure 5.2 draws the resulting loop.

5.1.4 Execution Through the Student Interface

The teacher's cylinder command is converted into the same normalised action as the student's and executed through the complete student action chain. This keeps demonstration labels inside the student's reachable command set.

5.1.5 Teacher Performance

Table 5.1 reports the privileged DLS teacher on the stage-6 verge task through the ROM plant. It provides the performance reference for distillation, not a guaranteed upper bound on the student.

The teacher loses ground contact for about a fifth of the run and its cut-height tail is six times its mean. The evaluation thresholds are stated with their origins in Section 3.11; the teacher's numbers are the distillation reference, not the source of those thresholds.

5.2 Policy Formulation

5.2.1 Policy Definition

The deployed controller is a memoryless Gaussian multilayer perceptron mapping a 66-value observation to four normalised actions. Actor and critic are separate networks

Table 5.1: The privileged DLS teacher, measured on the stage-6 verge task through the command-ROM plant (128 environments, 480 steps, all eleven path families, seed 17). The 480-step window is the earlier selection window of Section 7.1.3, not the full-strip horizon of Chapter 7, so these values are not comparable with the tables there. It is competent, not perfect, and several evaluation thresholds sit only a little beyond what it achieves.

Quantity	Teacher	Gate
Mean $\lvert e_{\mathrm{cut}} \rvert$	0.0234 m	≤ 0.040 m
p95 $\lvert e_{\mathrm{cut}} \rvert$	0.1524 m	—
Mean $\lvert e_{\mathrm{lat}} \rvert$	0.1221 m	≤ 0.300 m
Ground-contact fraction	0.7807	≥ 0.700
p01 head alignment	0.9638	≥ 0.900
Minimum head alignment	0.8317	—
p95 tool acceleration	64.9 m/s^2	≤ 120 m/s^2

with hidden widths 256–128–64, ELU activations and empirically normalised observations. PPO [Schulman 17] is supplied by RSL-RL [Rudin 22]. This network *architecture* is shared by all three stacks, but the weights are not: R and IF are the matched pair trained at a common seed, while AP's base is a separate distilled checkpoint trained at a different seed and actor learning rate (Table 5.3). AP freezes that base and widens its actor input by four plant-context columns, of which only the payload channel varies in the reported campaign (Section 5.7.2).

The four actions denote desired cylinder increments in R and IF and valve commands in AP (Section 5.7.6). The base policy has no nominal controller beneath it, and the truck's forward motion remains scripted.

5.2.2 Observation Space

Figure 5.3 lays out the 66 channels by group. The grouping tells the story of what this controller has to reason about: nine terrain-height samples ahead of the cutter, three task-error channels, sixteen channels of cylinder state and remaining travel, sixteen describing the actuator interface, ten of pose and context, and twelve of model error.

The last twelve channels carry cylinder tracking residual, actuator prediction residual and normalised effort. A memoryless network cannot form these temporal errors from one observation, and without them the error to be corrected is absent from its input.

This shared observation space also limits attribution. The tracking residual is near zero under R but carries the ID–FD path's standing error under IF, giving IF a useful closed-loop correction signal. The R–IF comparison therefore cannot separate the actuator representation from the value of that signal.

5.2.3 Action Space and Bounds

The policy emits $\boldsymbol{a} \in \mathbb{R}^4$, sampled from its Gaussian at training time and taken at the mean for evaluation. Five bounding stages sit between that sample and the PhysX cylinder target (Figure 5.4). Each can only reduce authority, never introduce motion the policy did not request.

Figure 5.3: The policy observation space. The six coloured groups form the 66-channel task input shared by R, IF and AP. AP appends the four white plant-context inputs in the slice 66:70; only payload varies in the reported campaign.

The rate limit bounds $|\Delta \boldsymbol{a}|$ at 0.25 per control step and is held *constant* across every curriculum stage rather than used as a curriculum knob: the teacher's own command stream has a structural per-step bound near 0.20, so a student limit below that makes part of the label set unreachable. The exponential filter ($\beta = 0.70$ at the final stage) smooths the command. The limit shield scales the command down, and only in the direction that consumes headroom, over the last 20 mm of cylinder travel, replacing the teacher's hard clamp into the learned range. The command scale converts the filtered action into 4 mm of cylinder travel per unit action, and the target integrator accumulates the increments in reference space, permitted to lead the measured extension by at most 50 mm on cylinders 1 to 3 and 20 mm on cylinder 4.

This is the cylinder-increment reading of the action, used by R and IF. Under AP the same bounded action is a valve command scaled to span the admissible valve range, so the cylinder-space stages do not carry the same meaning there.

The chain also fixes the capability ceiling: four millimetres of cylinder travel per step is the entire authority budget, which produces the 0.240 m/s lateral rate limit of Section 3.8.2 from which the reference paths are derived.

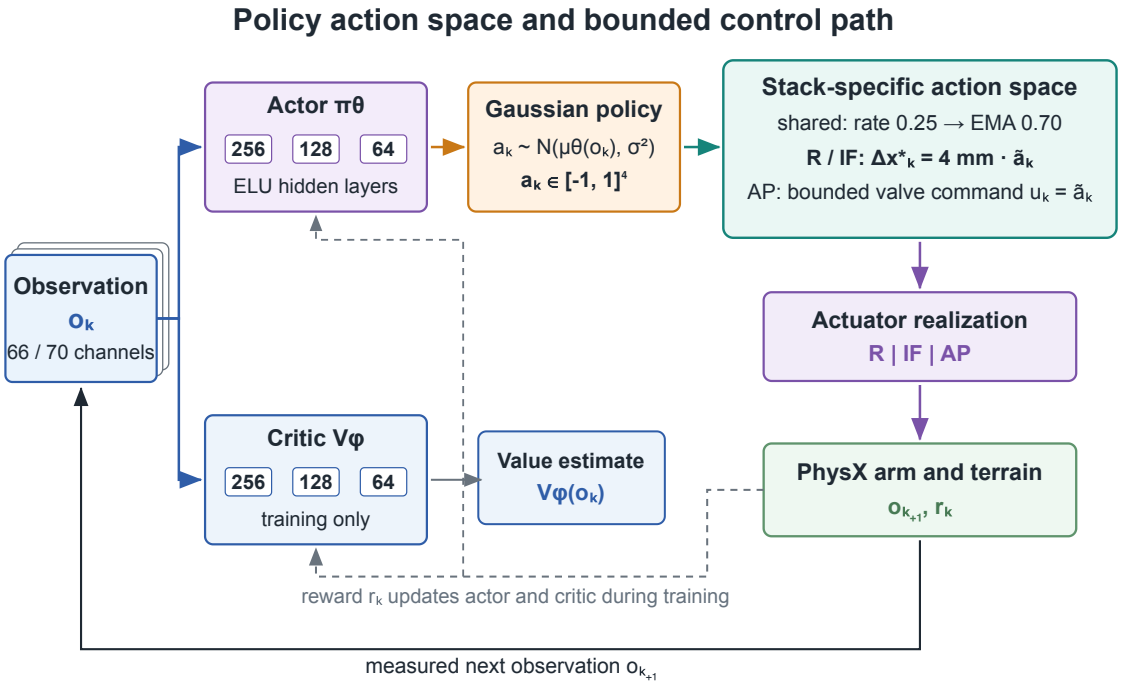

Figure 5.4: Policy action space and deployed control path. The actor maps the observation space to a four-dimensional Gaussian action. Rate limiting and exponential filtering are shared. R and IF then apply the cylinder travel shield, command scale and target-lead cap; AP instead interprets the bounded action as a valve command. The critic and reward update are training-only; measured state closes the deployed loop.

5.2.4 The Reward Function

The final R reward has ten bounded terms (Table 5.2): three task rewards and seven behaviour penalties. Their shaping remains responsive beyond the acceptance boundary until the stated clip. Three rules govern the terms.

No Gaussian Shaping

The earlier reward priced tracking as $w \exp[-(e/s)^2]$, which is effectively flat beyond about $2s$ and therefore supplies almost no gradient where the failing path families live. Every shaping term is instead linear or Huber in the error. Writing $x = |e|/s$, the two shapes used are

$$B(e; s, c) = \min(x,\, c), \qquad H(e; s, c) = \min\left(c,\, \begin{cases} \frac{1}{2}x^2, & x \le 1 \\ x - \frac{1}{2}, & x > 1 \end{cases}\right). \tag{5.2}$$

B has the same gradient at 0.5 m of error as at 0.05 m, so a policy far from the reference is still told which way to move; H buys quadratic precision near zero where cut height needs it, without a vanishing far field.

Bounded Terms

An unbounded term can dominate the sum and destabilise the critic. Each contribution is clipped, giving the reward a published budget: $6 + 12 + 4 = 22$ for the task terms against

Table 5.2: The ten reward terms of the final R task, with weights and shaping scales. B and H are the bounded shapes of Equation (5.2). The lateral and head terms each combine a coarse and a fine scale, so far-field correction and near-field precision are priced separately.

Term	Weight	Shape and purpose
Cut-height tracking	+6	H, scale 0.04 m, cap 2.5; the primary mowing-quality term
Lateral tracking	+12	B, coarse 1.50 m and fine 0.60 m mixed 0.6/0.4; the primary line-following term
Ground contact	+4	pays for the cutter being in contact
Head alignment	−12	B on the deficit below 0.98, fine scale 0.02 and coarse scale 1.00 mixed evenly
Contact loss	−3	charges directly for loss of contact
Action rate	−1	B on the mean absolute command change, scale 0.25; penalizes chatter
Tool acceleration	−1	B, scale 300 m/s^2; mechanical-wear proxy on the cutter
Contact force	−1	B on force above 30 kN, scale 150 kN
Cylinder headroom	−2	B on encroachment inside a 0.02 m margin, scale 0.10 m
Hard impact	−4	indicator on the impact event

−24 for the safety terms. A non-finite value is replaced by −25, just below that worst case, so a numerical fault can never outscore legitimate behaviour.

Fixed Sign Convention

The three task terms are non-negative and the seven safety terms non-positive. This makes worst-case (CVaR) weighting [Rockafellar 00] safe: scaling a non-positive term by a factor ≥ 1 can only lower return.

One apparent inconsistency is deliberate. The tool-acceleration term saturates at 300 m/s^2 while its gate sits at 120 m/s^2, because the reward scale is a saturation scale for a bounded penalty, not a target: the gate stays the binding criterion and the reward only keeps a gradient against gross excursions. The action-rate term is mild for a related reason, since cylinder 4 owns head pitch and a rate tax falls hardest on the fast corrections that keep the head down.

Every term is logged by name during training, because total reward can rise while the system gets worse. The reward is not the result criterion; promotion uses the eight gates of Section 3.11.

5.2.5 Optimisation

Policies are optimised with clipped-surrogate PPO over generalised advantage estimation, in the form of Section 2.7. The maintained configuration uses clip parameter $\epsilon = 0.2$, discount $\gamma = 0.99$, GAE parameter $\lambda = 0.95$, five learning epochs over four minibatches per

update, an adaptive learning-rate schedule targeting a KL divergence of 0.02 from a base rate of 3×10^{-4}, gradient-norm clipping at 1.0, and a clipped value loss with coefficient 1.0. The terrain task collects 32 steps per environment per update. At 512 environments that is 16 384 transitions per update.

The final fine-tuning blocks depart from that configuration because they run under the reference anchor of Section 5.4 rather than an adaptive learning-rate schedule: clip $\epsilon = 0.1$, discount $\gamma = 0.97$, three learning epochs, fixed learning rates of 10^{-5} for the actor and 10^{-4} for the critic, entropy coefficient 0.001, an exploration standard deviation fixed at 0.15 and excluded from the gradient, and frozen observation normalisers.

Observation normalisers and exploration scale remain frozen during the final fine-tuning blocks. Exact values are recorded in each run manifest.

5.3 Policy Training Protocol

5.3.1 Distillation by DAgger

Policies are initialised by imitation, not by reward. The privileged teacher is rolled out in the student's own environment and the student is fitted to the teacher's raw action by regression, in the DAgger scheme of Ross et al. [Ross 11]: the teacher's probability of *executing* the action, rather than merely labelling it, is annealed from one to zero over the first 70 % of iterations, so later training collects states the student itself visits. Labels are accumulated in a replay buffer of 262 144 transitions and refitted over three epochs per iteration in minibatches of 8 192.

The cloning loss in Figure 5.5 falls while the teacher drives and rises after the student takes over, exposing the state-distribution shift that DAgger addresses. Closed-loop metrics remain near the teacher's preflight values without matching them.

The final protocol uses the reference anchor below to limit drift from an accepted checkpoint.

5.3.2 Objective Mismatch in Unconstrained Fine-Tuning

Unconstrained PPO fine-tuning from distilled checkpoints increased total reward while degrading the primary tracking objective, in the worst case by a factor of 12.9. The critic remained healthy, with explained variance in the 0.85–0.95 range. This motivates the anchored formulation of Section 5.4, which is used for the final policies.

The reward includes ground contact and cut height as well as lateral tracking, so it can increase while a gate-relevant tracking measure degrades. This is a claim about two functionals over the same policy space rather than about the optimiser. Training maximizes the expected return $J(\vartheta)$, while acceptance is decided by the vector of physical gates $\boldsymbol{G}(\vartheta) \preceq \boldsymbol{g}^\star$ of Section 3.11. Writing $\mathcal{F} = \{\vartheta : \boldsymbol{G}(\vartheta) \preceq \boldsymbol{g}^\star\}$ for the feasible set, the observed outcome is

$$\vartheta_{\text{unconstrained}} \notin \mathcal{F}, \qquad \text{while} \qquad \vartheta_{\text{distilled}} \in \mathcal{F}. \tag{5.3}$$

The accepted distilled checkpoint satisfies the gates, while unconstrained fine-tuning may leave the feasible set. The protocol therefore reports both checkpoints, logs gate metrics

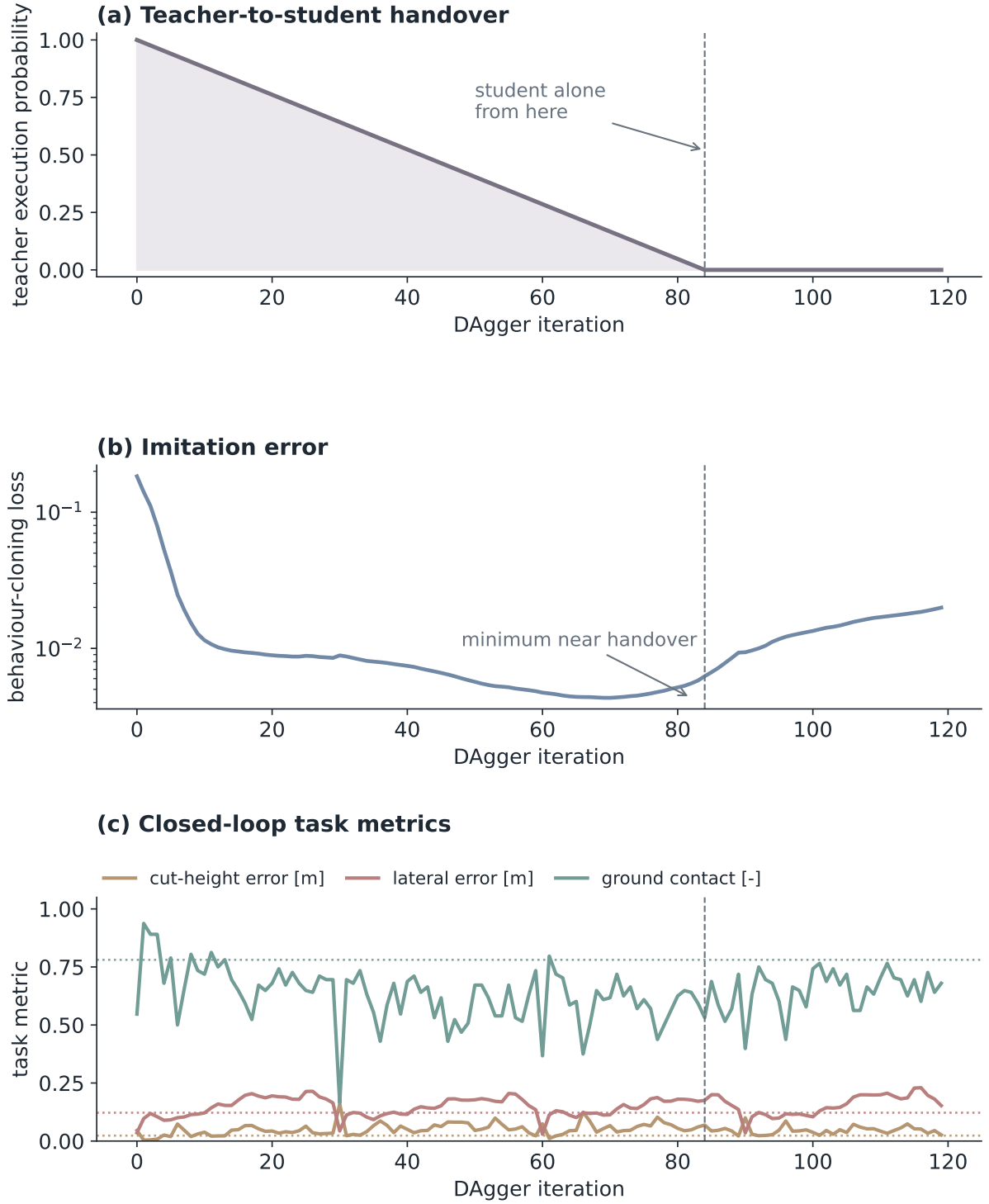

Figure 5.5: A measured distillation run for the R stack. (a) The teacher's execution probability falls to zero over the first 70 % of iterations. (b) Behaviour-cloning loss falls by over an order of magnitude under teacher execution, then rises as the student visits new states. (c) Cut-height error, lateral error and ground-contact fraction; dotted lines mark the teacher's preflight values from Table 5.1.

during training and constrains policy drift from the point where reward and evaluation

were known to agree.

In the final anchored block, five of one hundred updates were rejected, all on the reference bound and none on the update bound. This illustrates why the protocol checks cumulative drift as well as individual update size.

5.3.3 Action Limits

The lead cap is load-bearing. Raising it to 50 mm halved lateral error on `lane_change_hold` (0.4833 → 0.2310) and `boom_sweep` (0.3745 → 0.1602), but applying the same value to cylinder 4 reduced head alignment from 0.9507 to 0.4636. This measured split sets the per-axis limits above.

Across ten earlier training blocks, changing only the feedforward source or levelling gain left p01 head alignment at 0.79–0.80. The final observation therefore includes the tracking and model-error channels defined in Section 5.2.

5.4 Reference-Anchored PPO

Reference-anchored PPO limits drift from a frozen policy that has passed the evaluation gates. Let $\mathcal{F} = \{\vartheta : \boldsymbol{G}(\vartheta) \preceq \boldsymbol{g}^{\star}\}$ denote the accepted set, π_{ref} the reference policy and J the expected return. Each block aims to solve

$$\max_{\vartheta} J(\vartheta) \quad \text{s.t.} \quad \mathbb{E}_{\boldsymbol{o}\sim d_{\pi_\vartheta}}\Big[\mathrm{KL}\Big(\pi_{\mathrm{ref}}(\cdot \,|\, \boldsymbol{o}) \,\|\, \pi_{\vartheta}(\cdot \,|\, \boldsymbol{o})\Big)\Big] \le \delta_{\mathrm{ref}}, \tag{5.4}$$

with the expectation taken over the states the policy itself visits.

5.4.1 Critic Warm-Up

After distillation the actor already behaves sensibly, but the critic has never been fitted to the long terrain return, so the first PPO updates would move a useful actor using a value estimate that is not yet meaningful. The actor is frozen for 150 iterations while only the value loss is optimised, under the asserted invariant $\vartheta_{\mathrm{after}} = \vartheta_{\mathrm{before}}$. The observation normalisers are frozen at the same point, since otherwise the same raw observation is normalised differently from one iteration to the next and the actor's behaviour changes while its weights do not.

5.4.2 The Reference Anchor

Let π_{ref} be a frozen copy of the accepted warm actor. It is queried only on states the live policy visits and never supplies an executed action, so PPO remains the sole controller of the plant. The penalty is the *forward* divergence, which for diagonal Gaussians is available in closed form [Cover 06],

$$D_{\mathrm{ref}} = \mathrm{KL}\Big(\pi_{\mathrm{ref}} \,\|\, \pi_{\vartheta}\Big) = \sum_{i} \left[\log \frac{\sigma_{\vartheta,i}}{\sigma_{\mathrm{ref},i}} + \frac{\sigma_{\mathrm{ref},i}^{2} + (\mu_{\mathrm{ref},i} - \mu_{\vartheta,i})^{2}}{2\sigma_{\vartheta,i}^{2}} - \frac{1}{2}\right], \tag{5.5}$$

at the cost of one additional forward pass and no sampling. Because both scales are fixed at $\sigma = 0.15$, this collapses to a pure quadratic on the drift of the commanded action,

$$D_{\text{ref}} = \sum_i \frac{(\mu_{\text{ref},i} - \mu_{\vartheta,i})^2}{2(0.15)^2}, \tag{5.6}$$

so with fixed σ the divergence directly measures drift in the mean action, without changes in exploration scale affecting that measure.

Forward KL penalises abandoning any reference behaviour; reverse KL could collapse onto a subset, reproducing the observed trade between competencies. The term is therefore a behaviour-cloning anchor on the live policy's states, following demonstration-regularised policy gradient [Rajeswaran 18].

The actor objective is

$$\mathcal{L}(\vartheta) = \mathcal{L}_{\text{PPO}} + \mathcal{L}_V - \beta\,\mathcal{H}(\pi_\vartheta) + \lambda_{\text{ref}}\,\mathbb{E}_k\Big[\text{KL}(\pi_{\text{ref}}\|\pi_\vartheta)\Big], \qquad \lambda_{\text{ref}} = 0.10. \tag{5.7}$$

5.4.3 Trust Regions and Rollback

A fixed penalty alone does not enforce the divergence limits. Each update is therefore checked against both bounds:

$$D_{\text{update}} = \text{KL}(\pi_{\text{old}}\|\pi_\vartheta) \le 0.01, \qquad D_{\text{ref}} \le 0.05. \tag{5.8}$$

The first limits an individual update; the second limits accumulated drift from the fixed reference within the block.

Enforcement is in two stages. During minibatch optimisation the epoch is early-stopped as soon as D_{update} exceeds its bound, and after the update both divergences are recomputed over the full rollout. If either bound fails, or any quantity is non-finite, the actor and its optimiser state are restored *atomically*. On a divergence rejection the critic fit is retained when finite, since the bounds constrain the policy rather than the value function; on a non-finite update both networks are restored.

5.4.4 The Block Ratchet

The reference is not fixed forever. Within a block it is frozen. When a checkpoint is scored and passes the gates, it is promoted to reference,

$$\vartheta_{\text{ref}}^{(m+1)} = \vartheta^\star, \qquad \vartheta^\star \in \mathcal{F}, \quad \vartheta^\star \in B_{\delta_{\text{ref}}}\Big(\vartheta_{\text{ref}}^{(m)}\Big), \tag{5.9}$$

and the actor's optimiser moments are reset, because momentum accumulated relative to the old anchor is not meaningful relative to the new one. The critic's are retained: it is fitting the same return, and its calibration is expensive to rebuild.

The anchor advances only through checkpoints verified against $\boldsymbol{G}$, bringing the evaluation criterion into training. Distillation supplies the first anchor; the critic is warmed with actor and normalisers frozen; each later block trains within the fixed bound, rolls back rejected updates, scores checkpoints on at least three held-out meshes and promotes only a passing checkpoint. The last promotion is returned. Algorithm 5.1 states the procedure.

Algorithm 5.1: Reference-anchored PPO. The teacher is used only for distillation, every rollout is controlled by π_ϑ alone, and the frozen reference moves only when a checkpoint passes every gate.

input : teacher π_T; gate test $\boldsymbol{G}(\vartheta) \preceq \boldsymbol{g}^\star$; M blocks; bounds $\delta_{\text{upd}} = 0.01$, $\delta_{\text{ref}} = 0.05$
output : last promoted actor $\vartheta^\star$

distil ϑ from π_T on DAgger states, then remove π_T `// teacher-free thereafter`
freeze ϑ and the observation normalisers; fit the critic for 150 iterations
$\vartheta_{\text{ref}} \leftarrow \vartheta$; $\vartheta^\star \leftarrow \vartheta$
for *block* $m \leftarrow 0$ **to** $M - 1$ **do**
  freeze ϑ_{ref} for the whole block
  for *each PPO iteration* **do**
    roll out π_ϑ as sole controller; snapshot $\mathcal{S} \leftarrow (\vartheta, \text{optimiser})$
    update on Equation (5.7), stopping the epoch once $D_{\text{upd}} > \delta_{\text{upd}}$
    recompute $(D_{\text{upd}}, D_{\text{ref}})$ on the full rollout
    if *either bound is exceeded or non-finite* **then** restore $\mathcal{S}$ atomically
  end
  score every checkpoint on ≥ 3 held-out meshes, aggregating each gate to the worst mesh
  if *some checkpoint satisfies* $\boldsymbol{G} \preceq \boldsymbol{g}^\star$ **then**
    $\vartheta^\star \leftarrow$ that checkpoint by the fixed selection rule; $\vartheta \leftarrow \vartheta_{\text{ref}} \leftarrow \vartheta^\star$
    reset the actor optimiser moments; retain the critic state
  end
end
return $\vartheta^\star$

5.4.5 Scope of the Constraint

The update bound follows trust-region policy optimisation [Kakade 02, Schulman 15]; the fixed-reference bound limits cumulative drift within a block. Neither certifies task performance. A KL bound does not provide a useful gate guarantee at the full evaluation horizon, so promotion still requires measured gate checks.

5.5 IF: The Transferred Actuator

IF is the stack the thesis is about. Its actuator is the pair identified in Simscape and transferred frozen: the policy emits desired cylinder motion, the inverse model of Section 4.2 converts it to a valve command, the strict forward model of Section 4.3 converts that command to a predicted extension increment, and PhysX receives the bounded cylinder target. Neither model has seen the deployment plant. If a policy can be trained through this path and can hold the task, then an actuator identified from a high-fidelity model is usable for robot learning, which is the question Section 3.1 posed.

The scripted truck crosses a 26 m strip at a speed drawn per episode from 2 to 7 km/h; path families and terrain meshes are distributed across environments. The policy follows the chapter's three-stage construction: distillation, frozen-actor critic warm-up and reference-anchored PPO.

Robust here means what Section 2.6 means by it: trained across the randomised plant family, applying one fixed response to whichever member it meets, with no online estimation. The policy is not blind to plant variation, since the twelve model-error channels carry a standing residual that is itself plant-dependent. What it lacks is any representation of *which* plant produced them.

Two properties of the composed path shape its behaviour, both established offline. The valve command is accumulated rather than issued absolutely and is clipped to the per-axis limits $[0.95, 0.95, 0.80, 0.80]$, so the actuator path carries its own saturation boundary independently of the action chain. Only the predicted extension increment is integrated; the velocity output is a diagnostic (Section 4.3).

5.6 R: The Reference Condition

R is the calibrated reference condition. Its policy issues the same desired cylinder motion as IF, but the command ROM realises the transition. ROM plus PhysX defines the virtual reference plant, so R does not face the same actuator-transfer mismatch as IF. This does not make its tracking score an upper bound: separately trained policies can trade off lateral and cut-height error differently.

IF inherits the task, terrain families, meshes, observation space, action chain, reward, gates and PPO update unchanged, and the stored run manifest matches R field for field on actor learning rate, entropy coefficient, seed, terrain seed, environment count, curriculum stage, frozen normaliser and warm-up. The actuator representation is the changed training condition; the resulting policy weights are separate. The experiment does not transplant an IF-trained policy unchanged into the ROM plant. Figure 5.6 draws both pipelines.

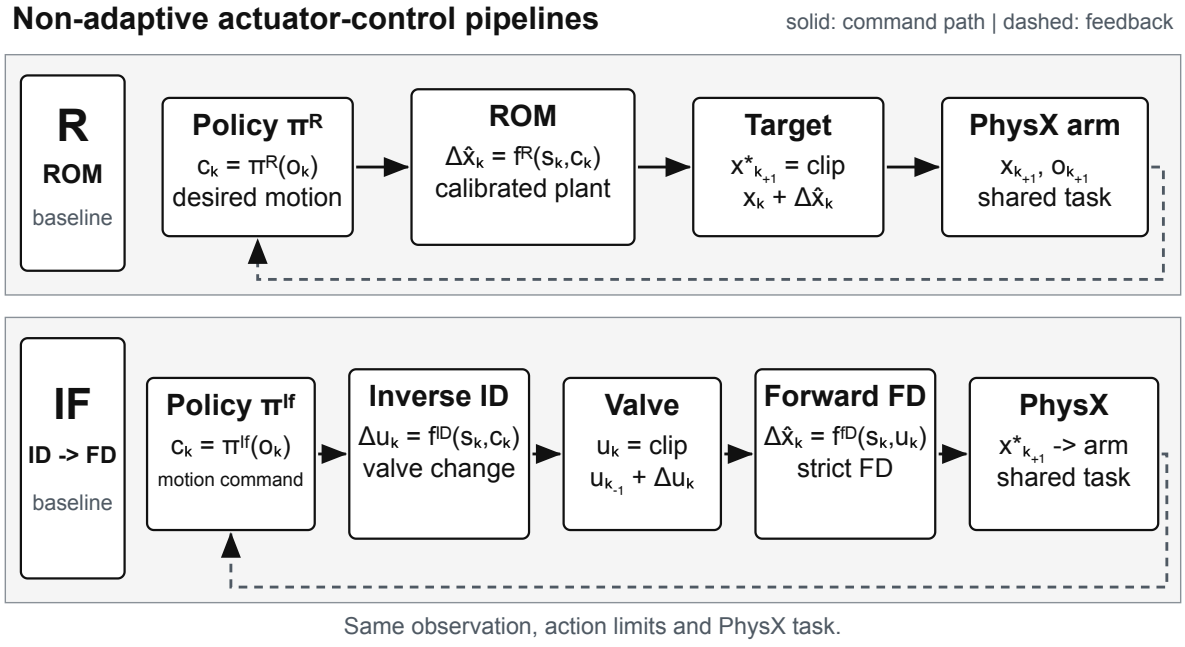

Figure 5.6: The non-adaptive R and IF control pipelines. R sends the policy's desired cylinder motion through the calibrated command ROM. IF composes the independently identified inverse and strict forward models before applying the same bounded cylinder target in PhysX. Both paths use the same observation, action limits and task.

5.7 AP: Adaptation on the Policy

AP is the stack that adapts online. The mechanism is deliberately small. A frozen actor emits the valve command directly. Four normalised plant-context channels are appended

to its observation, only the weights on those channels are trained, and a payload estimate computed at runtime drives the one channel that varies. The forward model is frozen and the inverse model is absent from the command path, so the controller adapts and the actuator does not. Section 7.5 reports the measurements behind this arrangement and evaluates it.

5.7.1 Why AP Does Not Use the Inverse Model

ID is needed when the policy requests cylinder motion: it converts that request into a valve command. AP already emits the valve command, so inserting ID would require a different action space and an additional conversion. The frozen FD is still needed to predict the cylinder response.

There is also a control reason for this choice. ID accumulates valve-command increments before clipping. Sustained demand can hold that command at its limit, leaving little authority for a further correction. The transfer test in Section 4.2.3 measures this saturation. In AP, payload context changes the direct-valve actor's command before it passes through the existing bounds. This removes the inverse-command accumulator, not the valve limits. The omission applies only to AP; IF retains the ID–FD chain.

Table 5.3 states what AP shares with R and IF and what it does not. AP is not a matched replacement for IF: it answers whether conditioning on an online payload estimate can improve a frozen direct-valve controller. The matched actuator-representation comparison remains R against IF, and adaptation is evaluated inside AP against its own frozen and wrong-information controls.

Table 5.3: What is matched between AP and the R/IF pair, and what is not. Section 5.9.2 gives the full matching rules.

Dimension	Matched?	Detail
Task, terrain, reward, gates	yes	identical across all three stacks
Base 66-channel observation	yes	AP appends four context columns derived from these channels
Action space	no	AP emits the valve command directly, not a bounded cylinder increment
Training seed	no	AP trains at a different seed
Actor learning rate	no	lower than the R/IF value
Frozen-actor warm-up	no	AP's base is the already-distilled policy, frozen; there is no separate warm-up stage

The command path is therefore shorter than IF's by one model. The frozen strict forward model of Section 4.3 converts the policy's valve command to a predicted extension increment, and PhysX receives the same bounded cylinder target as the other two stacks:

$$\pi_{\text{frozen}}(\boldsymbol{o}, \hat{\boldsymbol{c}}) \longrightarrow \boldsymbol{u} \longrightarrow f_{\text{FD}} \longrightarrow \text{PhysX}. \tag{5.10}$$

The inverse model is absent from the command path. The final AP stack is the frozen direct-valve base actor, four trained context columns carrying the estimate $\hat{\boldsymbol{c}}$, and the

frozen strict forward model. Zeroing the estimate exactly recovers the frozen base, and permuting it across environments preserves its distribution while destroying its pairing, which are the controlled comparisons used in Chapter 7. Figure 5.7 draws the training and deployment paths.

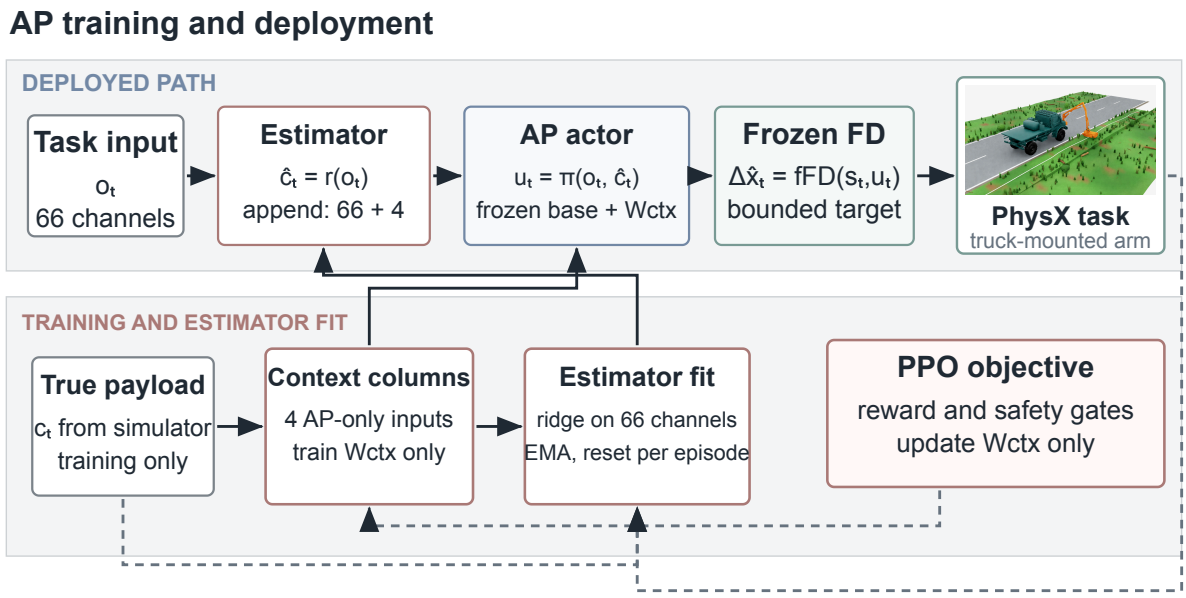

Figure 5.7: AP training and deployment. A ridge estimator maps the 66-channel task input to a single payload scalar, smooths it with an EMA that resets each episode, and writes it into the payload channel of the four AP-only context inputs; the other three are held constant. The actor combines its frozen base with $\boldsymbol{W}_{\text{ctx}}$ and sends valve commands through Frozen FD to PhysX. During training, true payload supplies the context inputs and PPO updates only the 1 024 weights of $\boldsymbol{W}_{\text{ctx}}$; the estimator is fitted afterwards on trained-policy rollouts. Dashed paths show updates and fitting data.

5.7.2 Conditioning on the Payload Estimate

Adaptation enters through the actor input, not through its output. The 66-value observation is extended by four normalised plant-context channels, and the actor's first layer is widened to receive them:

$$\begin{aligned} \boldsymbol{h}_1 &= \mathrm{ELU}\big(\boldsymbol{W}_o\boldsymbol{o} + \boldsymbol{b}_1 + \boldsymbol{W}_{\text{ctx}}\hat{\boldsymbol{c}}\big), \\ \boldsymbol{u} &= f_{\text{frozen}}(\boldsymbol{h}_1), \qquad \hat{\boldsymbol{c}} = \boldsymbol{0} \;\Rightarrow\; \boldsymbol{u} = \mathrm{base}(\boldsymbol{o}). \end{aligned} \tag{5.11}$$

Here f_{frozen} denotes the remaining actor layers and $\boldsymbol{o}$ is the normalised task observation. The added term acts in the first hidden layer, not as a residual valve command. Only $\boldsymbol{W}_{\text{ctx}} \in \mathbb{R}^{256\times 4}$ is trained: 1 024 parameters of the 59 588-parameter actor, with the input-layer gradient masked to the four context columns and every other parameter left untouched.

The identity on the right is what makes the comparison exact rather than approximate. Nominal is the zero vector, so the frozen arm is not a separately trained control but the same network evaluated at $\hat{\boldsymbol{c}} = \boldsymbol{0}$, and it is recovered bit for bit. The base task columns

are verified identical across training seeds after adaptation training, which confirms the mask held.

The four channels are (α, τ, delay, payload), and only the last of them varies in any experiment reported in this thesis: the first three are held at their constant nominal values in every run, so the trained $\boldsymbol{W}_{\text{ctx}}$ is exercised through one column. The payload channel is normalised as $(m - 1)/0.5$, so a nominal cutter maps to zero and a fivefold payload to eight.

Section 4.6 specifies the estimator and its per-seed fit. Its smoothed output enters the payload channel without a deadband.

5.7.3 Freezing the Base Policy

All base weights are frozen and gradients are restricted to the context columns. This preserves the zero-context controller while allowing payload conditioning to be learned independently of the base weights.

5.7.4 Training of the Context Columns

PPO trains the context columns using true payload. The ridge estimator is fitted afterwards on rollouts from that trained policy, separately for each seed. The other three context channels remain at nominal throughout the reported campaign.

5.7.5 Consistency With the Plant Dynamics

Two consistency requirements follow from the system model of Chapter 3, and both are met by construction.

The first is causality. The estimate is formed from the 66-value policy observation alone, every channel of which the deployed stack already produces at runtime. The privileged factor vector, the true payload and any hidden simulator state are excluded, so nothing enters the deployed path that would be unavailable on the physical machine. The estimate also starts each episode at nominal, which is the sense in which the machine works out what it is carrying while driving, not being told.

The task reference is not excluded, so the regression could in principle read terrain and task state, not load. Section 7.5.8 tests that directly by permuting the payload across environments, which leaves the distribution of the context untouched and destroys only its correspondence to the plant it describes.

The pressure differences among those channels are recursive states of the frozen forward model, not plant measurements. The estimate is therefore faithful only to the extent bounded in Section 5.7.6, and carrying it to the physical FME 500 would need differential-pressure instrumentation the machine does not have. Every result here uses the pressure-inclusive observation row; a strictly deployable row excluding measured pressure is defined but not scored.

5.7.6 Sufficiency of the Forward Model

The direct-valve path requires the frozen FD to produce a command-sensitive extension increment. Only that increment is integrated; velocity and pressure remain diagnostic outputs. Section 4.3.2 reports the valve ablation and rollout tests, including the weaker velocity channels and operating-point dependence.

5.8 IsaacLab Integration and Timing

Table A.3 lists the concurrent rates: PhysX integrates at 120 Hz, the policy and its payload estimate run at 60 Hz, and the learned actuator models advance at 20 Hz. No single value describes the whole control loop.

The relationship between the 60 Hz and 20 Hz layers is where integration errors hide, so it is fixed by construction rather than by interpolation. A learned actuator model is evaluated on every third control step and its predicted increment is held across the two intervening steps. The action presented to the model is the action of the tick on which it fires, not an average, and the target integrator accumulates in reference space, because integrating against the measured extension turns a static hold into a slow ratchet (Chapter 3).

The estimator needs no history buffer, although its exponential smoothing introduces a response lag. Within a tick the estimator takes that tick's 66 task channels, forms the raw scalar, updates the per-environment exponential moving average and writes it into the context column; the actor and then the actuator path run after that. Each environment's average resets to nominal at its own episode boundary, detected from its episode step counter, so asynchronous resets need no separate hook. Checkpoint loading rejects mismatched normalisation or feature width, and the polynomial degree is stored in the fit file so a deployed estimator cannot disagree with the one validated offline.

5.9 Constraints and Fair-Comparison Rules

5.9.1 Limits and Safeguards

Authority is bounded by the action-chain rate, travel shield and lead cap, then by per-axis valve limits. The environment terminates on the safety conditions of Section 3.11, and every learned component has a defined fallback for non-finite output.

Preflight also asserts that each reference remains within the measured 0.240 m/s lateral authority ceiling of Section 3.8.2; a faster path would be infeasible regardless of the reward or optimiser.

5.9.2 Matching Rules and Comparison Controls

Two comparisons carry distinct claims; the thesis does not rank all three stacks.

The Matched Pair: R Against IF

The representation claim rests on R and IF, which differ only in the actuator path. Their task, paths, terrain and held-out meshes, reward, action bounds, gates, episode length, networks and training construction are identical. Only this pair supports an across-stack comparison.

The policies are trained separately through their respective actuator paths. This tests control with transferred actuator models, not zero-shot transfer of one unchanged policy between models. ROM's deployment calibration also precludes interpreting R as an independent estimate of the real machine. The results establish neither policy transfer to hardware nor a universal ranking of model classes.

The Paired Comparison: AP Against Its Own Frozen Base

The adaptation claim is made within AP. Its arms share the frozen base, training budget, reward and evaluation conditions, and are the same network evaluated at different context values (Section 5.7.2). Estimated, zeroed, true and permuted context are compared within seed and paired environment by environment; the permutation control is run against the true payload rather than the estimate (Section 7.5.8). The primary checkpoint rule uses training reward without consulting held-out performance; the fixed final checkpoint provides a robustness analysis (Sections 7.5.8 and 7.5).

6. Data Generation and Preparation

This chapter documents the collections used by the models and controllers: their excitation, recorded signals, supervision targets and data splits. Table 6.1 maps each collection to its use.

Simscape campaigns. Closed-loop runs provide realistic operating data for the inverse and reduced-order models. Separate open-loop valve excitation provides the causal command information needed by the strict forward model (Section 6.4).

ROM calibration. After Simscape training, the command ROM was fine-tuned on five closed-loop IsaacLab reference shapes. Only its bounded correction head remained trainable, with Simscape replay retained (Section 6.6). The ID–FD pair was not calibrated in IsaacLab.

IsaacLab deployment data. A frozen 118-run task-controller collection tests ID–FD transfer. Separate on-policy AP rollouts fit the payload estimator after each policy-training run. Figure 6.1 summarises preparation of the training data.

Table 6.1 lists every collection this chapter builds, what each one records and what each one is scored against. The rest of the chapter takes them in turn; the table is the map, and no collection appears later that is not on it.

6.1 Dataset Objectives and Design Requirements

Four requirements governed collection and preparation.

R1: realistic operating distribution. The learned models are deployed inside a tracking controller, so they must be accurate in the states that controller visits, namely coordinated multi-axis motion, gravity-loaded poses and moderate speeds.

R2: causal identifiability of the valve. A forward model is only useful to a policy if it responds to the command. This requires that the valve signal in the training data carry information that the state history does not already carry, which is a property of the *experiment*, not of the network [Ljung 99, Isermann 11].

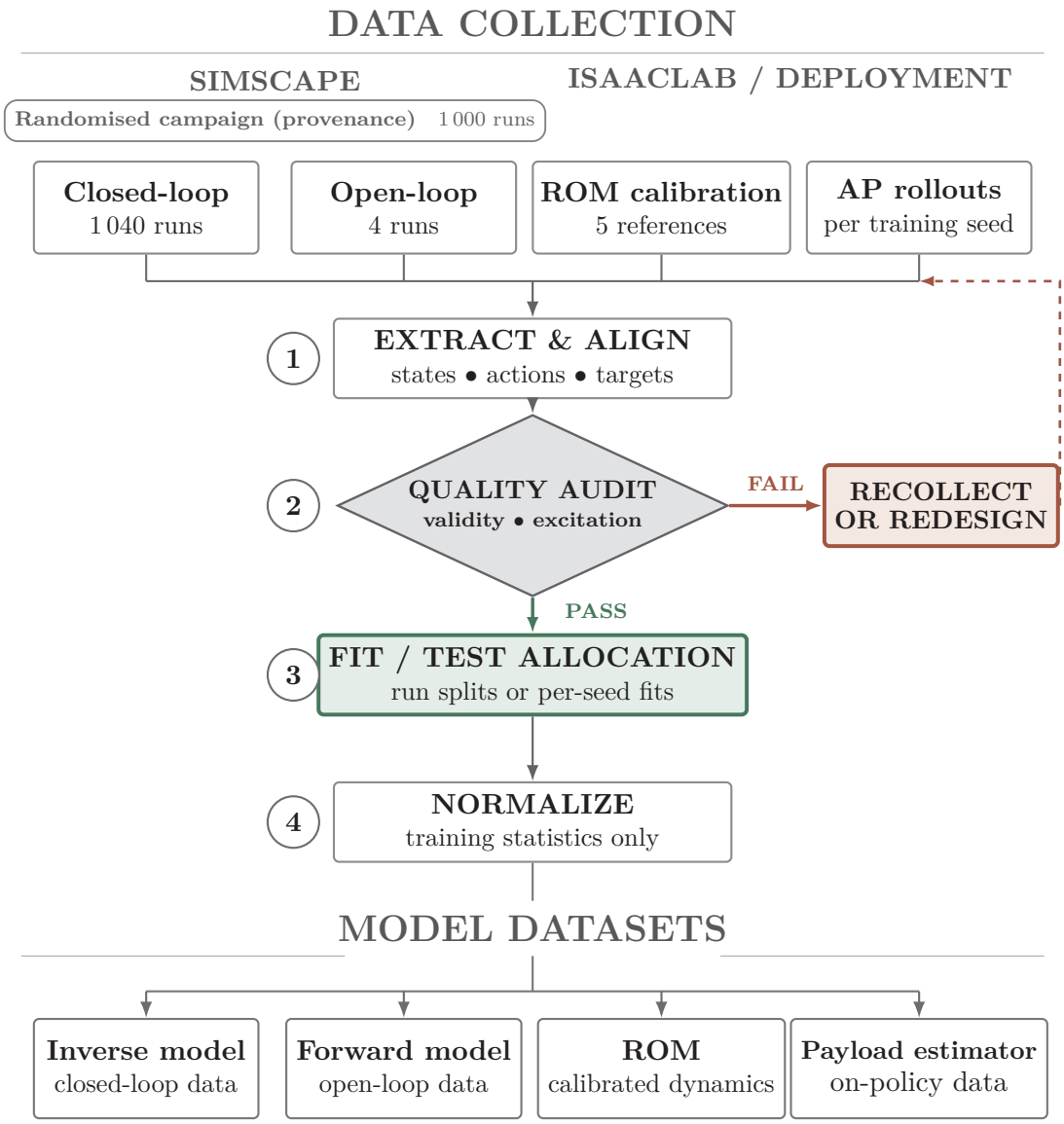

Figure 6.1: Data-preparation workflow. Two Simscape collections and two IsaacLab collections pass through extraction, audit, allocation and training-statistics normalisation. Actuator datasets use run-level splits; AP estimator data are fitted separately per training seed. The randomised Simscape campaign is provenance metadata, not a pipeline stage. A failed audit returns the experiment to collection; accepted material forms the inverse, forward, ROM and payload-estimator datasets. The separate task-controller collection in Table 6.1 tests transfer.

R3: variation across a plant family. Robust training and online adaptation both need many plants, not one. The variation must be physically meaningful, must demonstrably change the measured response, and must be recorded per run so results can be attributed to the tested plant.

R4: clean separation. Train, validation and test material must be separable at the level of complete runs, so that no transition window straddles a split boundary and no run contributes to two splits.

R1 and R2 require different experiments. Closed-loop control gives realistic motion but

Table 6.1: The recorded collections, what each contains and what supervises it. The first two are produced in Simscape and identify the frozen actuator models; the rest are recorded in the deployment plant, the calibrated command ROM driving PhysX. Ground truth means the quantity a model is fitted or scored against, never a measurement from the physical machine.

Collection	Size	Recorded	Ground truth	Identifies
Simscape				
Closed-loop	1 040 runs, 452 400 rows at 20 Hz	x, $\dot{x}$, Δp, rod force, valve command	Simscape transition	ID, ROM
Open-loop excitation	4 runs, 25 600 rows at 100 Hz	same state, commanded excitation	Simscape transition	strict FD
Deployment plant				
ROM calibration	4 084 rows at 20 Hz	six-step history, desired command	PhysX transition	ROM correction head
Task-controller	118 runs	state under the task controller	PhysX transition	scores ID, FD
On-policy AP	refitted per seed	66 task channels	true payload	ridge estimator

correlates command with state; open-loop excitation breaks that correlation but is less representative of deployment. The two campaigns are therefore retained and combined as described in Section 6.5.

6.2 Randomised Parameter Space of the Simscape Campaign

This section records the Simscape randomisation design as collection provenance. No reported model is trained on this curriculum. The export the actuator models are actually fitted on is a separate closed-loop collection on the nominal plant (Section 6.5). The curriculum is kept because its compliance analysis informed the IsaacLab factor design of Section 6.7.1.

Plants are drawn from the parameter space of Table A.1 by Latin hypercube sampling [McKay 79], fixed once before any simulation runs, so the parameter table can be audited independently of the data it produced. The batch is a curriculum, not a homogeneous pool: run 1 is the nominal plant, runs 2–251 randomise only the hydraulics, runs 252–501 only the mechanics, and runs 502–1000 both (Figure 6.2(a)), so a modelling failure can be attributed to one axis by retraining on one block. Appendix A.1 lists the axes.

Two design choices carry into the deployment factors. Seal wear is one sampled number driving three parameters together, so a worn cylinder becomes leakier and slower at once: it scales piston clearance linearly, valve leakage area with the cube of the same factor, and

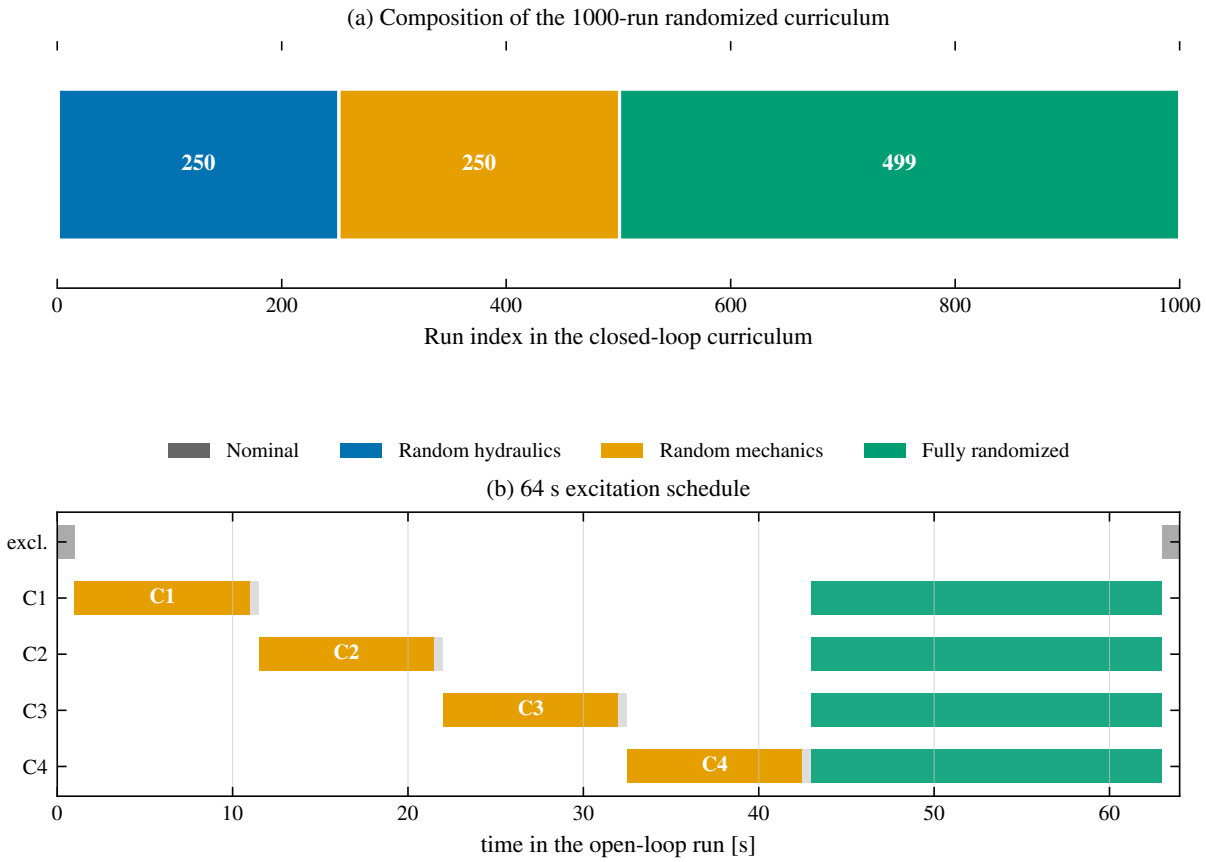

Figure 6.2: Design of the two collection campaigns. (a) Composition of the closed-loop curriculum: one nominal run, then 250 hydraulics-only, 250 mechanics-only and 499 fully randomised draws. (b) Timeline of one open-loop excitation run.

the spool time constant from 8 ms to about 13 ms at the top of the range. Sampling the three independently would have produced machines that are very leaky and yet perfectly fast. The entrained-air and dead-volume coordinates randomise chamber dead volume over 505 to 579 mL, which reaches the pressure dynamics through the chamber volume in Equation (3.4), so compressibility enters the batch as capacitance variation. The mechanical axes perturb all fourteen rigid bodies: mass by up to $\pm 20\,\%$, centre of mass by up to $\pm 5\,\%$ of its nominal offset, and the inertia tensor consistently with the mass change, so each body remains physically realisable. Joint stiffness and damping model compliance in the bearings and linkage, not in the fluid, and set how much of a pressure transient appears as boom oscillation instead of clean motion.

6.3 Closed-Loop Data Collection

The parameter table that produced the curriculum above is fixed before any simulation runs, and each run receives a new reference path.

6.3.1 The Controller as a Data-Collection Instrument

Every closed-loop run uses the same integral-backstepping controller at 0.005 s. Its gains remain fixed, so response differences are caused by the plant draw, not controller retuning.

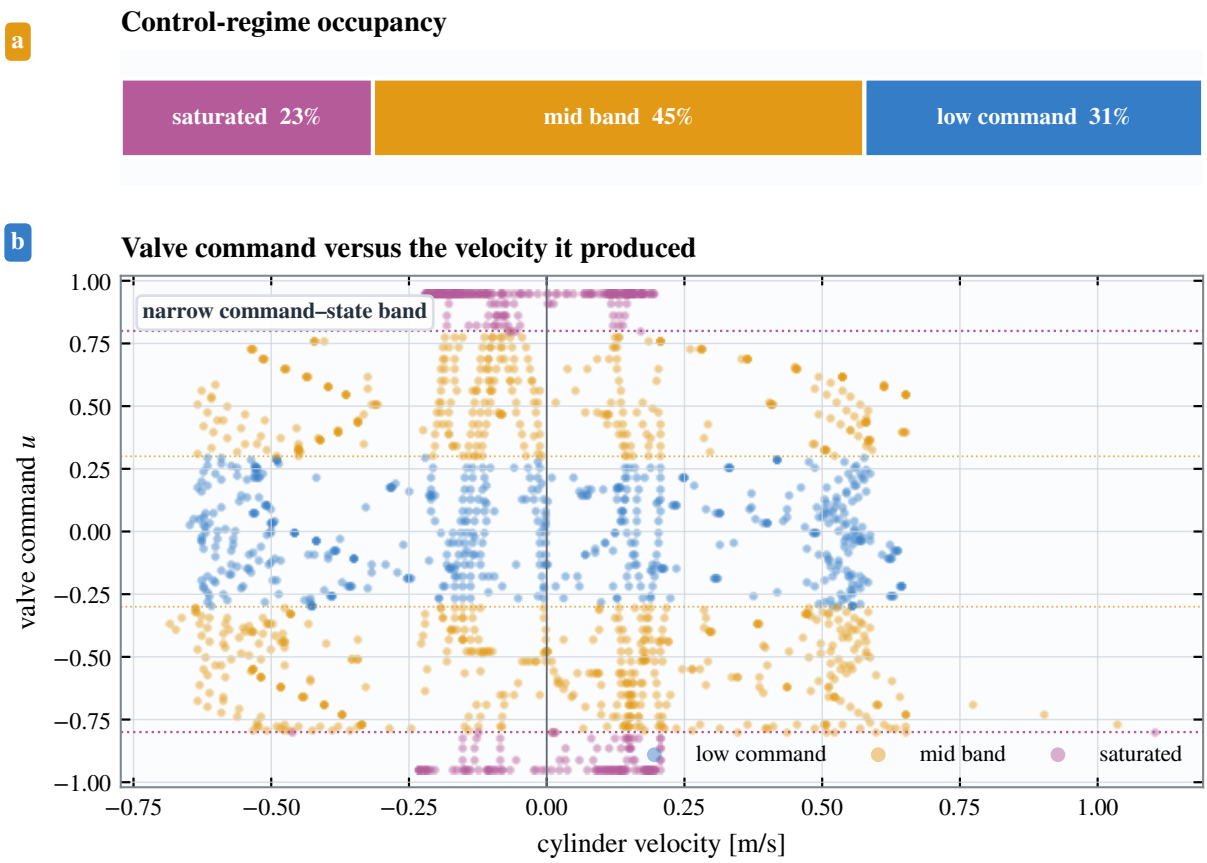

Figure 6.3: The closed-loop batch is diverse in control effort but not identifiable. (a) Fraction of time the valve spends saturated, in the mid band and in low-command creep. (b) The same run in the command–state plane, where the correlation that blocks identification is visible.

Joint 2 uses PI control with velocity damping and gravity bias because it carries the boom load. A barrier term enforces the URDF limits. Per-axis command limits are [0.95, 0.95, 0.80, 0.80, 0.90]; the first four are also stored in the inverse-dynamics checkpoint (Section 4.2).

6.4 Open-Loop Excitation Experiments

6.4.1 Limitations of the Closed-Loop Batch

The first forward model achieved one-step $R^2 \approx 0.999$ on closed-loop data but was almost insensitive to valve input: zeroing the input changed long-horizon error by only a few percent (Section 4.3.2). In closed loop, command is largely determined by the state history already given to the model, so the data do not separate command causality from state correlation [Ljung 99].

Figure 6.3 confirms that coverage alone is not the issue. The valve is saturated for about one quarter of a pooled run, operates at mid-authority for roughly half, and otherwise remains in low-command creep. Despite this spread, command and cylinder velocity remain concentrated along a narrow band. The data are diverse in control effort but not identifiable, motivating a separate open-loop experiment.

6.4.2 Design of the Excitation Signal

The excitation run lasts 64 s and follows the fixed schedule in Figure 6.2(b). One second of startup is discarded, because those samples describe the initialisation, not the dynamics. Four sequential slots follow, each giving one cylinder ten seconds of excitation followed by a half-second zero-command gap that makes the segment boundaries unambiguous. A twenty-second phase then drives all four channels at once, and a final second of zero command closes the run. Commands are limited to ±0.90 of full authority.

An initial rapidly switching signal met the formal persistence condition but kept each piston near one stroke position. The final design instead uses long full-authority plateaus in the sequential phase and slow, high-authority motion in the simultaneous phase. A slow near-square cycle is combined with distinct medium-frequency channel components, with raised-cosine entry and exit ramps. This provides both input excitation and useful stroke coverage.

6.4.3 Plant Draws and Acceptance

The same excitation is applied to four plant draws from the parameter space of the closed-loop campaign: nominal, randomised hydraulics, randomised mechanics and fully randomised. Holding the input fixed and varying only the plant makes the four runs directly comparable.

Each run is checked for stroke coverage before acceptance, the collector requiring at least 10 % of available stroke on cylinders 1–3 and 8 % on cylinder 4. The retained runs cover 33 % of the stroke on cylinder 1 and 12 % on cylinder 4, the narrowest of the four. Every channel clears the floor, and Section 6.10 confirms the material is informative at these coverages.

The acceptance checks in Section 6.10 verify that every export contains independent state trajectories.

6.5 Combining Closed-Loop and Open-Loop Datasets

The inverse model uses only closed-loop data because its target is the controller command for a measured state. The forward model is fine-tuned on the strict open-loop export while retaining closed-loop replay at weight 0.25. Without replay, four excitation runs do not provide enough pose coverage.

Figure 6.4 shows the difference: closed-loop data covers the reachable range, while open-loop data concentrates at identifiable valve plateaus. The forward model therefore uses both.

For ROM training, the inverse and forward exports are joined by run identifier, time index and sample time. The join is accepted only when their sixteen-dimensional cylinder states match sample for sample. It contains 452 400 transitions at 0.05 s from 1 040 runs, split into 936/52/52 complete runs. Valve command is used only to supervise the ROM's internal valve head; it is not a runtime input (Section 4.4).

This aligned nominal-plant export trains the inverse, forward and ROM models and ships with the project, so every model-level result can be recomputed from the file that produced it. Exact alignment, increment consistency and the identifiability failure were checked directly on that export.

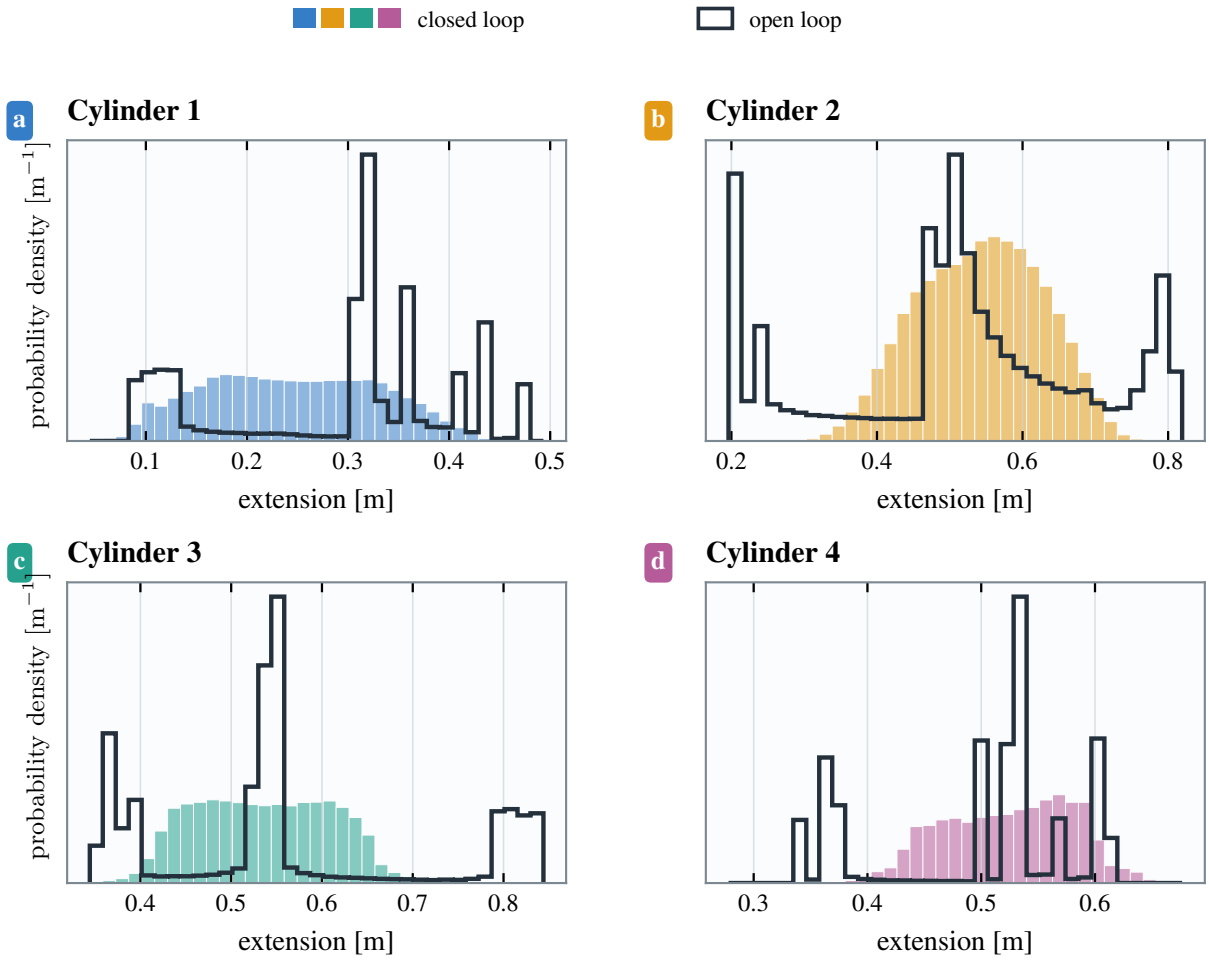

Figure 6.4: Extension coverage of the two campaigns, per cylinder and pooled over runs (closed-loop filled, open-loop outline). Each histogram is normalized to unit area, so the ordinate is the probability density of cylinder extension in m^{-1}: bin count divided by the total sample count and bin width. The closed-loop data fills the reachable range smoothly; the open-loop data concentrates into the plateaus where the valve is identifiable.

6.6 Closed-Loop Calibration Data for the Command ROM

The ROM drives a PhysX articulation whose compliance and contact response are not present in Simscape. Further Simscape training cannot remove the resulting deployment offset.

The calibration set therefore comes from closed-loop ROM–PhysX operation on the native 0.05 s grid. Its 4 084 transitions store the six-step runtime history and measured PhysX transition.

Only the command-gated correction head is trainable; all other parameters are frozen and the correction is capped at 3 mm. Original-data replay at weight 0.25 preserves wider pose coverage. The gate makes the correction zero at zero command, retaining stationary hold.

The trajectories predate the moving-verge benchmark and exclude cutter–ground contact. The bounded head limits, but does not remove, this coverage gap. Section 3.7.1 explains the consequence for the three-way comparison.

6.7 The IsaacLab Transfer Collection

The task-controller collection contains 118 runs recorded in the calibrated ROM–PhysX deployment plant. It measures transfer of the frozen ID–FD pair outside its Simscape identification distribution (Section 4.2.3). The following factor design and splits fix the conditions of that comparison.

6.7.1 The Physical Factor Vector

The randomised quantity is a six-dimensional physical factor vector

$$\mathbf{e} = \left[e_{\text{flow}},\ e_{\text{compliance}},\ e_{\text{seal}},\ e_{\text{pressure}},\ e_{\text{mass}},\ e_{\text{COM}_z}\right], \tag{6.1}$$

with the ranges of Table A.2. Four coordinates are hydraulic and two mechanical, which is what makes the hydraulic-only, mechanical-only and combined domains of Section 6.7.2 separable. Each candidate was admitted only after a one-factor sensitivity screen confirmed that moving it alone produces a measurable change in cylinder extension, velocity, pressure or end-effector motion.

One omission is deliberate. Actuation delay is not a randomised coordinate, because delay here is an *emergent* property of flow, compliance and load rather than a free parameter. It is measured as a response property of each run (Chapter 7) rather than supplied to any model as a known factor: handing an estimator a factor the real machine would never publish is the same mistake as handing it the true payload.

6.7.2 Domains and Factor Design

Runs are organised into five domains (nominal, hydraulic-only, mechanical-only, combined-training, and an *unseen* combined-test domain), and one factor vector is fixed for a complete run, so that no run mixes plants. Factor configurations are drawn with deterministic scrambled Sobol sequences [Sobol' 67] rather than independent uniform sampling, with explicit boundary anchors so that the corners of the space are represented and not merely approached, and with fixed per-domain seeds so the design is reproducible independently of the data it produced. Combined draws that compound into an excessive response gain are rejected at draw time rather than being collected and discarded later.

The resulting design is auditable: 98.3 % of the factor keys are unique, the active-factor matrix is full rank, every factor quartile is populated, and the factor keys of the training, validation, and test splits are disjoint. That last property is the one that matters most for the claims of Chapter 7: a test run does not merely come from a different episode, it comes from a plant the models have never seen.

6.7.3 Collection Procedure: The Task-Controller Runs

Every run is collected through the same interface the policy uses. The DLS controller closes the loop on measured cylinder state and issues its command through the four-dimensional action port into the command ROM and PhysX. No joint-target bypass is used and no PPO policy collects identification data, because a policy trained on the same plant would reintroduce the closed-loop correlation that Section 6.4 spent a campaign removing.

Independent excitation is supplied by dedicated four-channel PRBS windows through that same action port, at small amplitude and a hold of ten control steps, so the excitation is causally independent of the state without leaving the safe envelope. Probe actions are logged separately from controller output. Each run follows a fixed eleven-segment structure (settle, three postures, three PRBS windows, DLS task motion, reach-and-return and stow). As recorded, each run is 11 520 control steps and 192.0 s of simulated time.

Figure 6.5 summarises the IsaacLab collection configuration and its frozen run-level outputs.

6.7.4 The Frozen Dataset

The collection contains 118 complete runs: 2 nominal, 40 hydraulic-only, 12 mechanical-only, 48 combined-training and 16 unseen combined-test. Collection ran in four vectorized IsaacLab environments with per-environment factors and logging, and slots that tripped a safety or validity check were discarded and retried individually. All 118 runs passed the final collection gate, which checks finite physics, exact model-grid alignment, command rank, factor-response authority, state coverage and saturation limits.

The dataset is then *frozen*: the raw runs are checksummed and the run-level split is fixed at 82 training, 20 validation and 16 reserved combined-test runs, with zero factor-key leakage between them. The reserved sixteen do not influence history length, checkpoint, hyperparameter or architecture selection; they are scored only after each model is fixed, and test results are reported with validation results, not in place of them.

6.7.5 Learning Timebase and Hydraulic State

The learning timebase is the actuator model grid, not the control clock. Samples are exact 50 ms ROM transitions taken at control indices $0, 3, 6, \ldots$: state row t and command row t predict state row $t + 3$. Nothing is interpolated. This is the same alignment rule as Section 3.7.2, and it is enforced rather than assumed, because a model trained across a mismatched interval is wrong by a fixed factor in the quantity it exists to supply.

Differential pressure is the learned actuator's recursive state, not a plant measurement. It is recorded with cylinder extension and velocity and inherits the actuator model's prediction error.

6.8 Feature and Target Construction

6.8.1 The Actuator Models

The three learned actuator models take cylinder-space features, in the ordering and notation fixed in Section 4.1. Their input contracts differ because their questions differ.

Figure 6.5: IsaacLab transfer-data collection. Each run uses one fixed six-factor plant draw and combines DLS task motion with independent PRBS probes. State, action and factor keys are logged, audited and frozen into disjoint train, validation and held-out test splits.

The inverse model receives the measured cylinder state, the logged rod forces, the desired increments, and the previously applied command, twenty-eight numbers in all. The forward model receives the applied valve command and the measured state, twenty numbers. The reduced-order model receives a six-sample history of state, previously predicted valve, and desired command, twenty-four numbers per step.

Three conventions apply throughout, and they are the same ones the models enforce at inference time.

Increment Targets

Every dynamic target is a within-run one-step increment, $\Delta \boldsymbol{x}_k = \boldsymbol{x}_{k+1} - \boldsymbol{x}_k$, never an absolute state. This removes the large static component of the signals and turns the trivial persistence solution into an explicit baseline, not a hiding place.

Causal Alignment

Features at step k contain only quantities available at step k. The extraction stage verifies this by recomputing every logged target from the within-run difference of the states and requiring exact agreement, a check reported in Section 6.10.

Time-Base Conversion

The open-loop campaign is logged at 0.01 s but the models run at 0.05 s, so five consecutive raw samples are combined into one transition. The valve input attributed to that transition is the command held at the start of the window, which is the causally correct choice; the window mean is recorded for analysis only. Cylinder extension velocity is not logged directly in the closed-loop archive and is obtained by differentiating extension on the output grid, consistent with the trapezoidal identity that the strict data satisfies to $R^2 = 0.994$–0.9997 (Equation 4.6).

6.8.2 On-Policy AP Adaptation Rollouts

The context columns that carry the adaptation result in AP are not trained from the frozen collection above. They have no dataset and no run-level splits, because they are optimised under the reinforcement objective, so their training data is the on-policy experience the stack generates as it learns.

What replaces the dataset is a plant design, since on-policy training makes the training distribution a property of the environment, not of a file. The cutter mass is resampled per episode across a range spanning one to five times nominal. One fixed compromise is then unlikely to suit the whole family, which is what gives a controller something to gain from knowing which member it is driving, without assuming in advance that it will.

The estimator that supplies that knowledge at deployment does have a dataset, and it is collected after policy training, not before: a mixed-payload rollout of the trained policy, with the 66 task channels regressed onto the payload that produced them. It is refitted for each training seed, because the regression is on the observation distribution that policy induces and a different policy induces a different one. Whether the correct payload contributes beyond conditioning on any plausible one is tested against a permuted-payload control in Section 7.5.

The estimator reads the current task observation, including its reference channels. The permutation control in Section 7.5.8 therefore tests whether correct load information contributes beyond conditioning on a plausible value.

6.9 Preprocessing and Data Splits

Splitting is done at the level of complete runs, never of individual samples, because a random sample-level split would place transitions from the same trajectory on both sides of the boundary and report a score that mostly measures interpolation within a run. For the command-conditioned dataset the 1 040 runs are divided into 936 training, 52 validation and 52 test runs, giving 407 160, 22 620 and 22 620 transitions, with no run identifier in more than one split. The IsaacLab deployment collection is split by run *and* by factor configuration (82/20/16), the stronger condition: separating runs alone would let two runs of the same plant fall on opposite sides of the boundary and turn a claim about unseen plants into a claim about unseen episodes.

The strict open-loop export needs different treatment because it contains only four runs, and holding one out would remove an entire plant draw from training. Each run is instead divided into twenty temporal blocks, every fifth block is assigned to validation, and a five-sample guard interval is discarded at each boundary so that no 0.05 s transition window spans the split. This preserves comparable target distributions in training and validation.

Normalisation statistics are computed on the training split only and stored inside each checkpoint, so a model always normalises with the statistics it was fitted with; the loader enforces the stored contract, not recomputing it. Samples are carried with an explicit validity flag, not silently dropped, 93.7 % of the strict export being marked valid and the remainder being the excluded startup and shutdown seconds and the zero-command gaps. Keeping them with a flag means the time base stays contiguous and the within-run difference checks remain meaningful.

6.10 Dataset Auditing

No dataset here is used before it passes an explicit audit, for the reason given at the end of Section 6.4. The audits differ by collection because the failure modes do.

The Simscape material is checked for extraction correctness and for excitation. Extraction checks recompute the sample grid, the within-run increments and the finiteness of every channel, which catches misalignment, off-by-one indexing and rows concatenated across run boundaries. They test the extraction stage, not the simulation, whose fidelity is the separate question of Section 3.6.3. Excitation is checked by the rank of the instantaneous command matrix and of the lagged regressor built from the command history, which is what requirement R2 asks for. The strict open-loop export passes both.

The IsaacLab transfer collection is checked for physical validity, factor coverage and separation of factor configurations across splits. Its 118 runs sample a six-factor design sparsely, so transfer findings are limited to the recorded domains.

Two properties these checks record are referred to later. Stroke coverage is narrowest on the head-pivot cylinder, which is also the channel the strict forward model predicts least well. The rod-force channel carries zero supervised weight in the reduced-order model, a design choice, not a data defect: force is generated internally by PhysX at deployment, and the logged channel fails a piston-area consistency check on this export. No result rests on it. Appendix A.2 gives the measured values behind every check above.

Chapter 7 reports the model checks, controller performance and adaptation results using these collections.

7. Experimental Evaluation: Representation, Robustness, and the Value of Adaptation

This chapter reports the closed-loop evidence. Everything it scores is fixed before it begins: the actuator representations and their offline results in Chapter 4; the policies, their training protocol and their training diagnostics in Chapter 5. What remains is how those fixed stacks behave on the task itself.

The order follows the questions. The matched R–IF pair is compared first on trajectory tracking, then on command behaviour and dynamic response, and then on how much of the randomised factor draw reaches each plant, which fixes how much room robustness leaves for adaptation. The AP study follows and tests whether online payload conditioning improves its own frozen controller, and whether the correct payload contributes beyond a plausible one. Section 7.6 brings these results together.

7.1 Experimental Protocol

7.1.1 Test Conditions and Comparisons

Two campaigns run under different conditions.

Both run the moving verge-following task defined in Section 3.8. Figure 7.1 shows the environment the stacks are scored in.

The matched comparison between R and IF, reported in Sections 7.2 and 7.3, uses held-out terrain meshes, eleven trajectory families, a fixed truck speed of 4 km/h, the full 26 m strip, and the sixteen reserved plant configurations of Appendix A.1.

The payload campaign of Section 7.5 assesses adaptation only inside AP, comparing a frozen base, the online payload estimate and the true payload, with a permuted-payload control. It keeps the held-out meshes and the eleven families but differs in three respects. Truck speed is drawn uniformly from 2 to 7 km/h per environment, not fixed. The scored

Figure 7.1: IsaacLab realisation of the moving verge-following task. The scripted truck travels on the level road while the four-cylinder arm places the cutter above the procedurally generated verge and follows the white reference path. Randomised grass, bushes, soil patches, twigs, and logs form a non-colliding visual layer; the underlying verge mesh supplies the terrain disturbance and height-sensing geometry used by the controller. Terrain meshes and trajectory families are held out from training, and every stack meets the same ones.

horizon is shorter (Section 7.1.3). And the sixteen reserved plant configurations are not applied: the stressed variable is cutter payload, set per run, and the remaining factors are held at nominal. The generalisation claims of Section 7.4 therefore rest on the matched-pair campaign, not on the payload one.

The results are paired by environment or by mesh and trajectory family. Error metrics are reported separately for lateral tracking and cut height because the two errors describe different aspects of the task. Confidence intervals are percentile bootstrap intervals over the paired units [Efron 93], at $B = 20\,000$ resamples. Where one channel is required only not to get worse, the bound is a non-inferiority margin declared before the comparison [Piaggio 12] rather than a failure to reject. Success rules and checkpoint rules are fixed in writing before the arms are trained, in the sense of a pre-registered analysis [Nosek 18]; the registrations are listed in Appendix A.5. Checkpoints are selected using the validation or training criterion specified for each experiment and are then evaluated once on the held-out set.

7.1.2 Comparative Scope and Scoring

R and IF form the cross-stack comparison. AP has a different action space and is therefore not ranked against that pair. Its results are interpreted only through within-stack comparisons.

Results are reported on the worst path family and the worst terrain mesh rather than as a pooled score, so that an easy family or an easy mesh cannot mask a local failure. The eight gates of Section 3.11.1 state which aggregation each metric uses.

The matched training configurations change the actuator path, but produce separate policy weights. This is not a test of transferring an unchanged IF-trained policy into R. One training run exists per stack, so that claim is about the two realised controllers, and the 33 paired cells quantify terrain and trajectory variation, not retraining variation. Where seed-level evidence is available it is reported as such: the adaptation statements of Section 7.5 rest on three context-column training runs over one shared frozen base, paired within seed.

7.1.3 Scoring Horizon

The matched-pair results use the full 26 m trajectory: 1 404 control steps at 60 Hz and 4 km/h. The earlier 8.9 m selection window covered only about one third of an episode and is treated as an evaluation-protocol limitation, not as evidence about adaptation.

The payload campaign and the authority sweep score 900 steps, 15 s of episode time. At the 4 km/h of the authority sweep that is 16.7 m of the strip; in the payload campaign, where speed is drawn per environment, it ranges from 8.3 m at 2 km/h to the full strip at 7 km/h. The horizon was set before those arms were trained and is identical for every arm, control and A/A run in both, so it costs resolution rather than validity: the comparisons are paired per environment inside the same window and the shorter horizon cannot favour one arm over another. It does mean those two sections are not directly comparable with the matched-pair tables, and no such comparison is made.

7.2 Closed-Loop Task Performance

The first comparison reports closed-loop tracking of R and IF on the held-out meshes and trajectory families.

The task is scored on two independent error channels, drawn side by side in Figure 7.2: lateral error is deviation from the planned cutting line in the horizontal plane, and cut-height error is deviation from the intended vertical cut. They are kept separate throughout because a controller can improve one while worsening the other. Ground contact, impacts and tool acceleration are reported alongside them as checks on motion quality.

Table 7.1 compares the matched pair over 33 held-out (mesh, family) conditions. Median errors are low for both stacks, and the p95 values move in the same direction as the means, so the improvement also reduces the larger tracking deviations, not only the average.

Figure 7.3 resolves the aggregate by trajectory family. IF reduces mean lateral error from 0.103 m to 0.070 m and mean cut-height error from 8.4 mm to 6.3 mm over the 33 paired conditions. The advantage is not uniform: IF is better on six of the eleven families and R on five, with the largest difference on the double S-curve. Both stacks follow the simpler paths closely.

That distribution is uneven enough that the mean is the wrong summary, so Table 7.2 reports the paired difference three ways. Three readings follow, and they do not agree. The mean favours IF on both channels, but neither interval excludes zero, so the headline mean

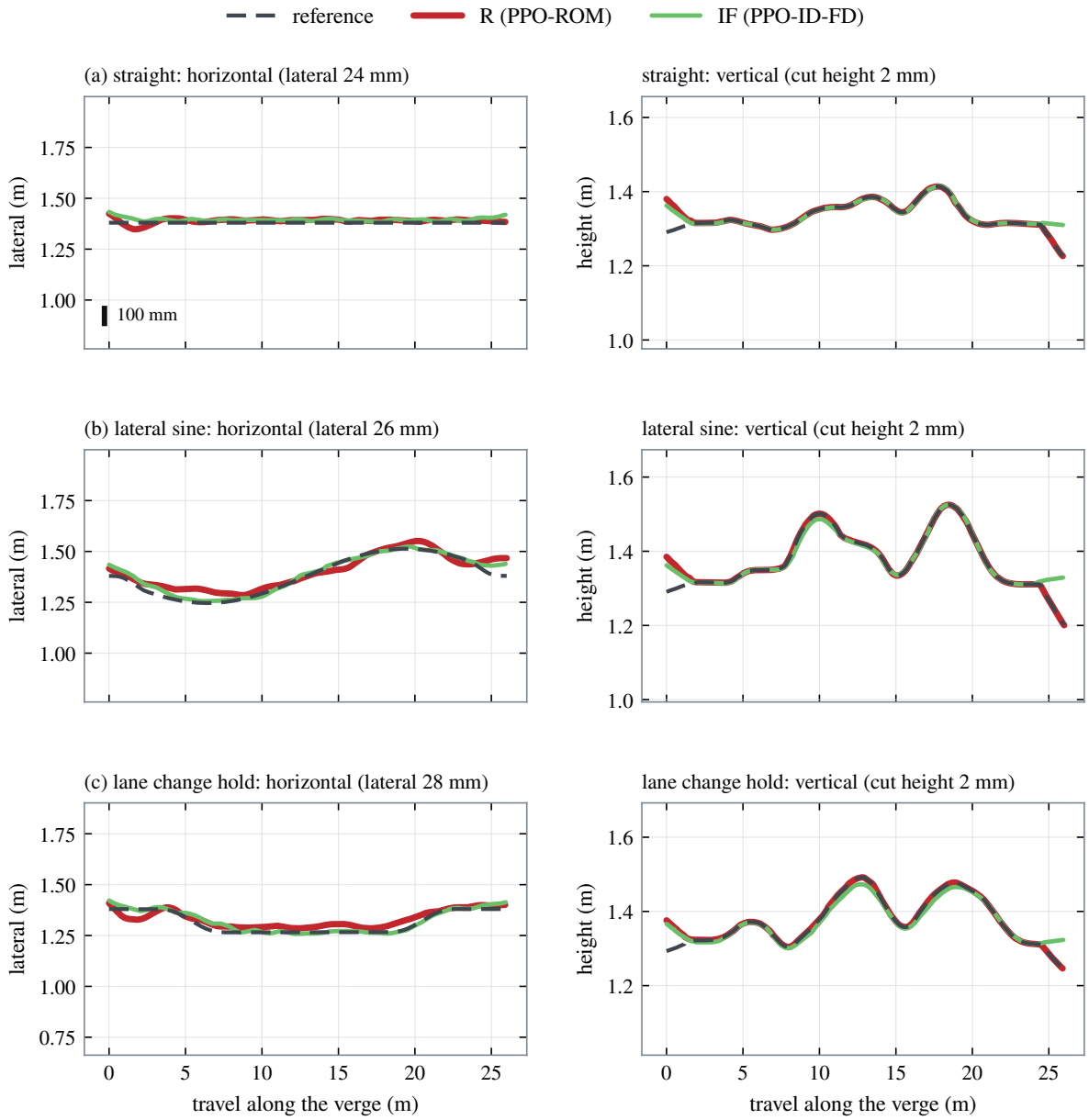

Figure 7.2: Cutter path against the reference on held-out mesh 1337, for three structurally different trajectory families: a flat line, a smooth periodic sweep and a step-and-hold. The left column is the horizontal plane and the right column the vertical plane over the same drive. The vertical reference is not recorded directly and is recovered as $z_{\text{ref}} = z_{\text{cutter}} - e_{\text{cut}}$, so both columns compare reference against actual. In the vertical panels the reference is largely hidden beneath both traces, which is what a millimetre-scale cut-height error looks like drawn at path scale.

Table 7.1: Closed-loop performance over 33 paired held-out conditions. The matched stacks differ only in the actuator representation.

	R	IF
lateral error, mean (m)	0.1026	**0.0703**
lateral error, median (m)	0.0392	**0.0281**
lateral error, p95 (m)	0.4664	**0.2665**
cut-height error, mean (m)	0.0084	**0.0063**
cut-height error, median (m)	**0.0013**	0.0031
ground contact	**0.9623**	0.9529
hard impacts	0.0947	**0.0905**
tool acceleration p95 (m/s^2)	39.90	**39.02**

Table 7.2: Paired difference between the matched stacks over the 33 held-out cells, in millimetres. A positive value means IF has the lower error. Intervals are percentile bootstrap over the paired cells, $B = 20\,000$. The leave-one-family-out row gives the aggregate that is furthest from the full-set mean.

	lateral	cut height
mean paired difference	+32.3 [−25.8, +98.8]	+2.1 [−3.8, +9.2]
median paired difference	**+9.3** [+7.0, +12.1]	**−1.7** [−2.3, −1.3]
cells with IF lower	26/33	6/33
families with IF lower	6/11	2/11
leave-one-family-out mean	−16.7 [−52.8, +16.2]	−3.2 [−6.4, −0.2]

is not a resolved difference. The median paired cell favours IF on lateral error by 9.3 mm with an interval excluding zero, and favours R on cut height by 1.7 mm, also excluding zero. The aggregate is carried by one family: the double S-curve contributes 522 mm of lateral difference against ±75 mm everywhere else, and removing it turns the mean lateral difference negative, at −16.7 with an interval crossing zero. A symmetric trim of the two most extreme families leaves −11.5 on lateral and −2.4 on cut height, both crossing zero.

The claim the data supports is therefore narrower than the means suggest. IF has the lower lateral error in the median cell and in 26 of 33 cells, and the advantage is concentrated on the demanding families rather than spread across them. R has the lower cut-height error in the median cell. No aggregate statement about the mean advantage survives removal of the double S-curve, and none is made here.

Under worst-mesh, worst-family aggregation both stacks miss the lateral gate and IF additionally misses head alignment, so neither satisfies the full eight-gate contract of Section 3.11.1 at that aggregation. The gates are met on the median cell, and it is the worst-case reading that separates them.

7.3 Actuator Representation and Dynamic Response

The same matched pair is now examined in the actuation channel: transport delay, closed-loop step response, and command effort.

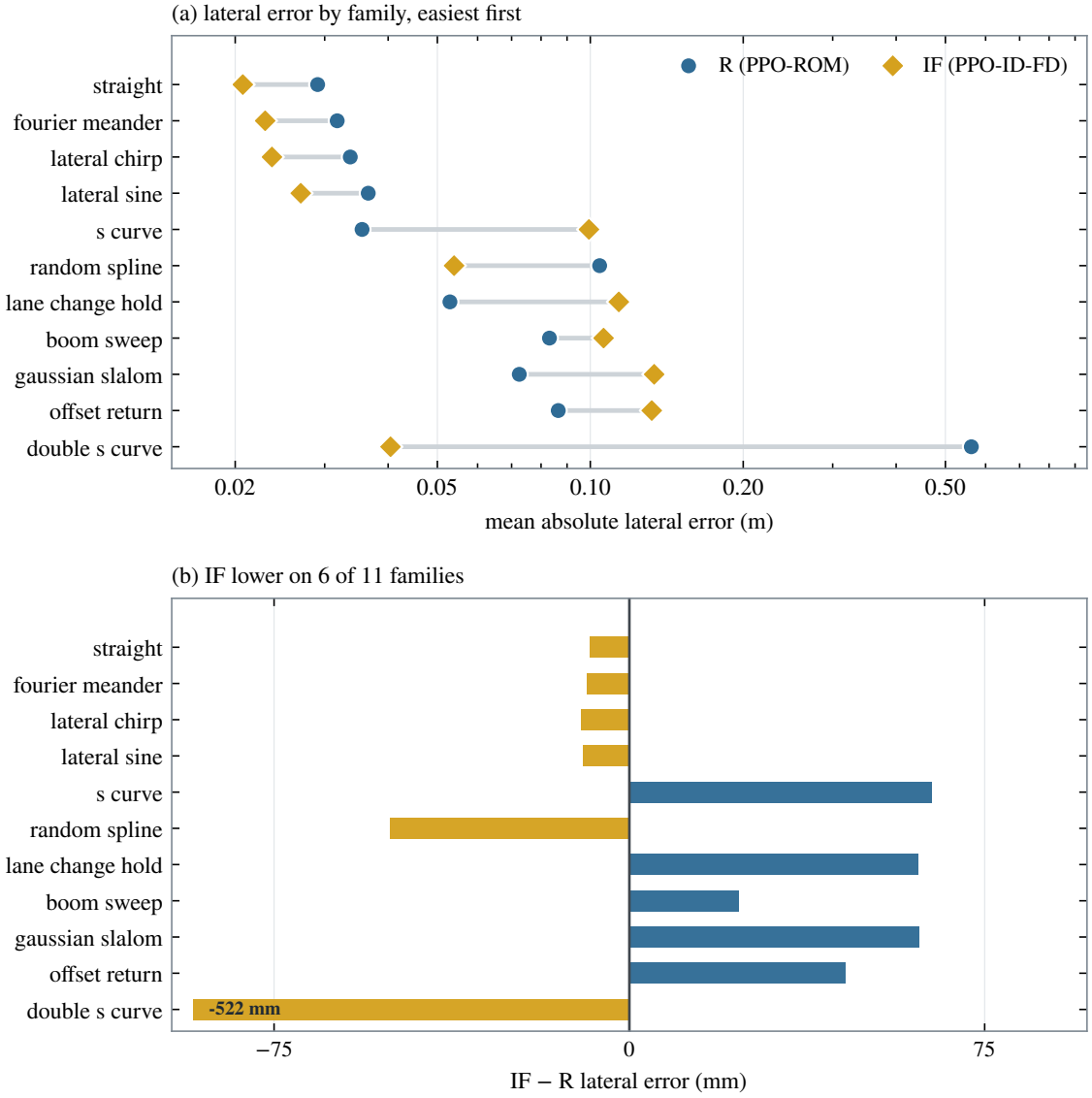

Figure 7.3: Lateral error by trajectory family for the matched pair. The aggregate difference is concentrated in the more difficult families.

7.3.1 Plant Delay and Step Response

Open-loop excitation of the deployed actuator path in IsaacLab measured a command-to-pressure delay of approximately 117 ms. Cylinder extension and joint motion began at the same time, which confirms that the dominant delay is hydraulic, not kinematic. The similar values across excitation amplitudes indicate that this delay is a stable property of the simulated actuation chain (Table 7.3). These are delays of the deployed chain, quantised to the 60 Hz control step, not the Simscape spool-and-fill onset of Section 3.6.4, which is resolved on the 100 Hz export.

The commanded-step experiment measures the closed-loop response to a 100 mm change in the reference line. Rise time is measured from 10 to 90 % of the step. Settling time is the first point at which the error remains within 20 mm for 0.5 s.

Table 7.3: Transport latency along the actuation chain, in milliseconds. Values are medians over open-loop excitation runs recorded in the deployment plant, at the 60 Hz control step.

	excitation amplitude			
Link	0.05	0.15	0.35	0.70
command → differential pressure	117	117	117	150
command → cylinder velocity	183	167	133	117
command → cylinder extension	117	117	100	100
command → joint position	117	117	100	100

Table 7.4: Response to a 100 mm commanded step. Values are median times over sixteen environments.

Stack	detected response	rise	settled	overshoot
R	333	617	2633	57 %
IF	350	717	2183	73 %

Times are in milliseconds.

Table 7.5: Control effort on held-out mesh 1337, over 16 environments and 1404 steps. The command and its rate are in normalised command units, so only their ratio between stacks is meaningful. Bold marks the better value of the pair. The lateral errors in the last column are this mesh's, not the 33-cell aggregate of Table 7.1.

Stack	RMS command (norm.)	command-rate RMS (norm. s^{-1})	reversals (s^{-1})	lateral error (mm)
R	0.586	7.55	40.0	120.1
IF	0.827	10.68	**10.6**	**26.8**

Table 7.4 shows IF settling about 0.45 s earlier, whereas R has lower overshoot. The command rate limit alone requires at least 417 ms to complete this step, and both stacks require more than 2 s to settle. Reducing hydraulic delay alone would therefore not remove the observed settling time; the command rate, the policy response and the reference handling would have to change with it.

Closed-loop settling is much slower than the hydraulic delay. Command-rate limits and policy dynamics dominate the response time. The actuator path changes the shape of the step, not the main settling time.

7.3.2 Control Effort

The reversal count identifies rapid changes in command direction that the root-mean-square magnitude does not reveal, and Figure 7.4 pairs each stack's effort with the tracking error it buys. A lower RMS command is not preferable when it comes with frequent reversals and larger path error.

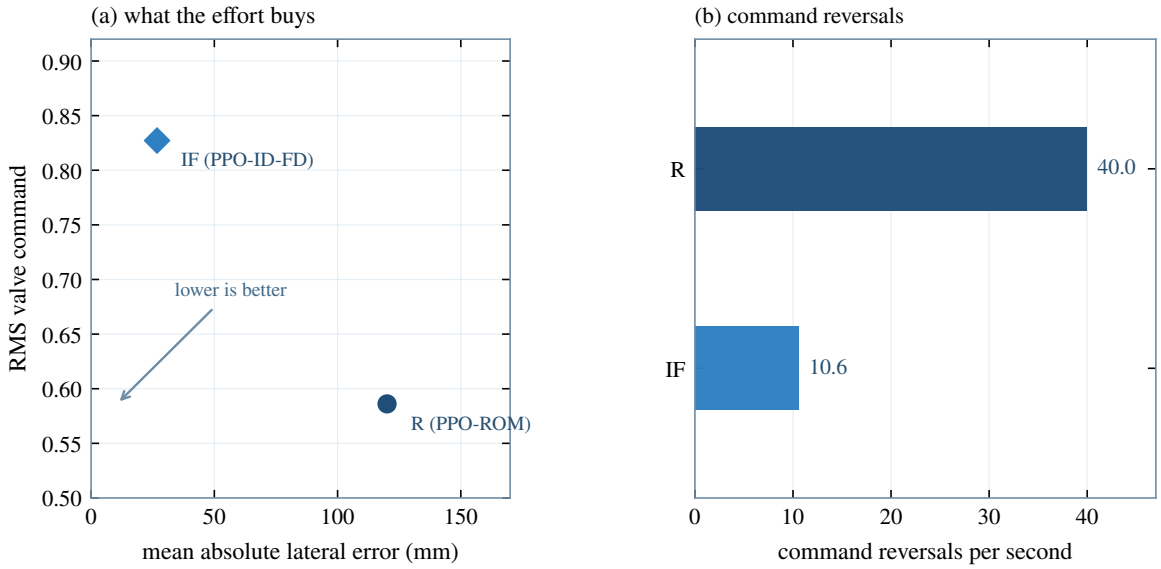

Figure 7.4: (a) Control effort against the tracking error it buys for the matched pair; the lower left is better. (b) Command reversals per second, which the root-mean-square command does not reveal.

Table 7.5 shows R using a smaller command magnitude but reversing almost four times as often. IF uses more actuation, reverses less, and tracks more closely. Effort is therefore reported together with the tracking it buys.

This is one mesh, so it is an observation about the recorded traces and not a 33-cell result. The lateral errors it reports, 120.1 mm and 26.8 mm, are further apart than the paired aggregate of Table 7.2, which is what a single condition can do. The reversal ratio is the part worth carrying forward, because it is scale-free and an order-of-magnitude effect, not a few per cent.

7.4 Mechanical Robustness and the Authority Margin

The preceding sections use the held-out evaluation set. This section establishes the robustness already present before adaptation is considered, and then applies a common authority reduction to all stacks.

One limit bounds every number below. Four of the six randomised factors are hydraulic (Table A.2). On the inverse and forward path the hydraulic factors are not applied at all, so the variation that reaches IF is mechanical: cutter mass and centre-of-mass height. On R the hydraulic factors are applied but constrained to the model's identification range. The authority sweep that follows is a generic reduction of realised actuator motion, not plant variation. Nothing in this section therefore supports a claim of robustness to hydraulic variation, and the summary in Chapter 8 is scoped to match.

Within that scope the matched stacks generalise to the held-out meshes, trajectory families and reserved plant configurations used in this chapter, and the randomisation sweep shows little change in tracking over the tested mechanical variation.

Table 7.6: Authority at which median lateral error first reaches 1.5 times its full-authority value, at nominal payload. The statistic is undefined under load, where lateral error never rises to that threshold as authority falls.

Stack	nominal error (mm)	threshold (mm)	onset κ
R	40.69	61.04	0.10
IF	23.66	35.49	0.20
AP frozen	45.18	67.78	0.10

The held-out evaluation therefore establishes transfer within the tested simulation population. It does not establish performance on arbitrary hydraulic conditions or physical hardware. A common authority perturbation was therefore used to compare all stacks more directly, which separates a reduction in realised actuator motion from the specific randomisation factors used during deployment.

7.4.1 Actuator Authority

Actuator authority was reduced by scaling the realised actuator displacement:

$$\Delta x_k^{\text{applied}} = \kappa \, \Delta x_k^{\text{actuator}}, \qquad \kappa \in (0, 1]. \tag{7.1}$$

Here, $\kappa = 1$ represents nominal authority and smaller values reduce the motion delivered to the plant. The same perturbation is applied after each stack's actuator representation, which makes the comparison independent of the individual factor parameterisations.

The learned controllers retain a broad margin under authority loss (Figure 7.5 and Table 7.6). A hard failure boundary was not reached within the tested range, so the experiment establishes an authority margin, not a breakdown threshold. IF has the lowest lateral error at every rung of the cross-stack comparison.

No stack reached the contact or safety gate, so at nominal payload the sweep bounds an authority margin, not locating a breakdown.

The reason errors stay bounded at $\kappa = 0.01$ is that the arm stops tracking rather than diverging. With almost no delivered motion it holds a nearly fixed pose while the truck carries it along the strip, so the lateral error becomes the reference's own excursion about that pose, not a growing divergence. The frozen AP arm shows this directly: as the reference-path amplitude is scaled its lateral error reads 69, 96, 130 and 168 mm, close to proportional to the amplitude, while ground contact is flat at 0.78, 0.77, 0.76 and 0.77. Contact holds because the cutter's height is then set by the terrain and the held pose, not by the withdrawn authority.

Under load the same axis behaves differently, and the difference is the useful part. At 4.0× payload the frozen controller is worst at *full* authority, at 330.9 mm lateral, 184.9 mm cut and 0.475 ground contact, and improves monotonically as authority is withdrawn, reaching 105.9 mm, 11.7 mm and 0.829 at $\kappa = 0.1$. Withdrawing authority is not a stress under load; it is a remedy. The failure mode is impact and rebound: the heavy tool is driven into the terrain faster than the controller corrects, bounces clear, and the cycle

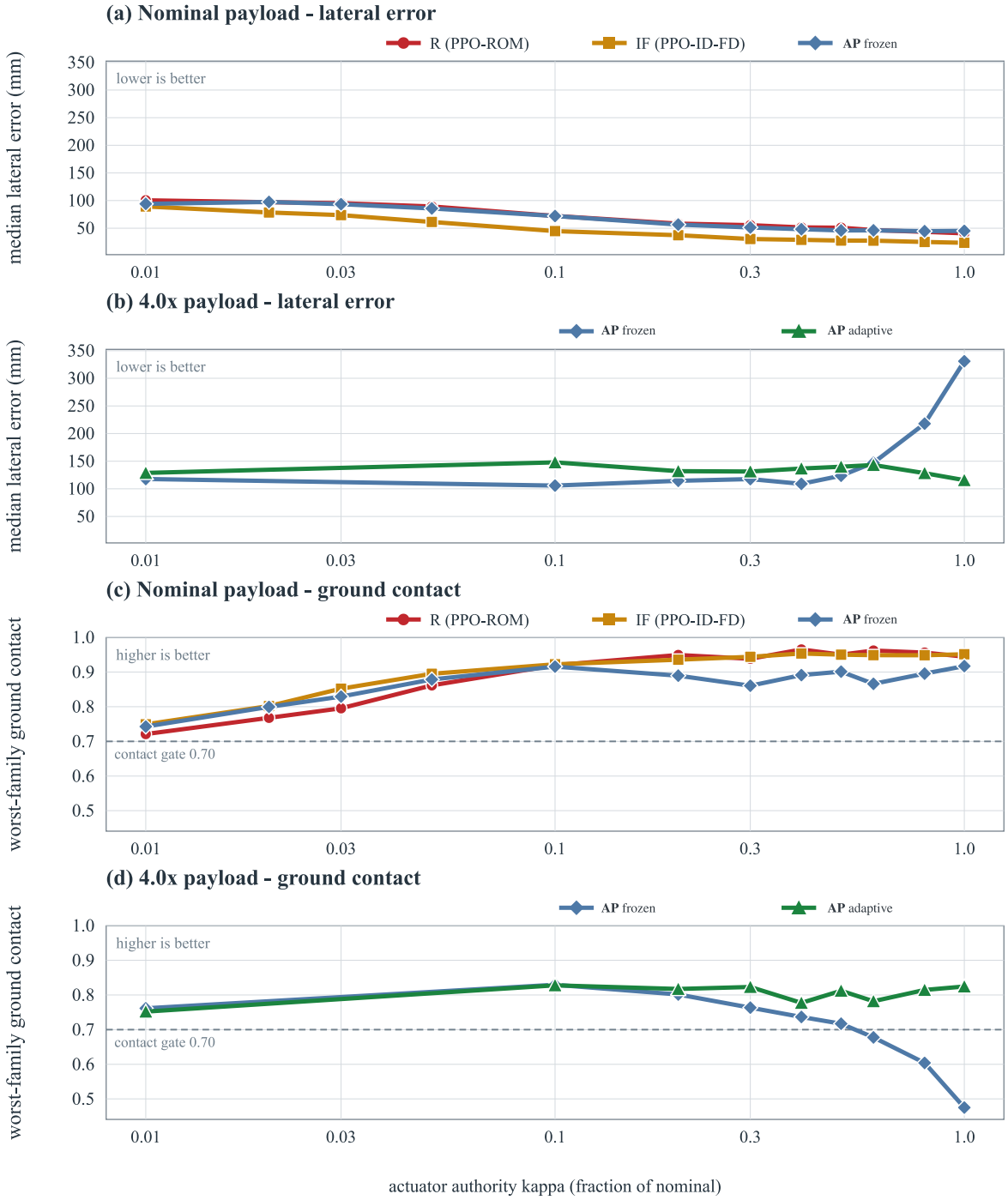

Figure 7.5: Lateral tracking and ground contact as actuator authority κ is reduced, on mesh 1337 at 4.0 km/h. Panels (a) and (c) show nominal payload, where every stack degrades as authority is withdrawn. Panels (b) and (d) show 4.0× payload, where the frozen controller is *worst at full authority* and improves as authority is cut, while the payload-conditioned arm is flat and already near the frozen controller's best point. Corresponding metric panels share a vertical scale.

repeats, which is why cut height, lateral error and ground contact all degrade together. Throttling the delivered motion damps the overshoot.

The payload-conditioned arm reaches the same place without giving anything up. At full authority it reads 115.7 mm lateral and 0.824 contact, within 10 mm and 0.005 of the best the frozen controller achieves anywhere on the authority axis, and it stays flat as κ falls. The two curves cross near $\kappa \approx 0.6$: below that the frozen controller is already damped, has stopped losing ground, and the conditioning costs a little lateral accuracy, which is the same regime dependence the payload sweep shows at 3.0×.

The distinction that matters for deployment is that authority is a fixed setting and payload is not. De-rating the actuator to $\kappa = 0.1$ buys the loaded improvement and charges for it permanently at nominal load, where the same stack degrades from 45 mm to 72 mm lateral. Conditioning on an estimated payload buys the loaded improvement at full authority and leaves the actuator alone. It is not free either: the conditioning is active at every payload, and what it costs at light load is measured in Section 7.5, not here. This sweep runs at 4.0× only, so it says nothing about the conditioned arm at nominal payload.

7.4.2 The Margin Left for Adaptation

The robustness result sets the scale for the following experiment. At nominal payload the frozen controllers absorb substantial variation and the authority sweep reaches no contact or safety failure, so there is little there for adaptation to recover. That is precisely why the adaptation study sweeps payload instead: it locates the regime where the frozen controller does fail, and measures the mechanism where a deficit actually exists.

7.5 Evaluation of Online Adaptation

The deployed mechanism conditions the frozen policy on a payload it estimates online from the task observation it already reads (Section 4.6). This section measures what that conditioning is worth, in which payload regime it holds, and how much of it depends on the estimate being correct, not merely present.

Three arms are compared, all of them the same network at a different context value. *Frozen* zeroes the estimate, which by Equation (5.11) recovers the base policy exactly rather than approximating it. *Adaptive* supplies the online estimate and is the deployable arm. *Oracle* supplies the true payload and is included as a reference point, not as a bound.

7.5.1 Experiment and Measurement

Scoring uses held-out mesh 3141, 176 environments and 900 control steps per arm, 15 s at 60 Hz, with truck speed drawn uniformly from 2 to 7 km/h per environment. Arms are paired on the environment index, which holds terrain, trajectory, speed and payload fixed across the pair, so the drawn speed is a matched nuisance variable, not a confound. Each arm runs in its own process, because an earlier layout that swept arms inside one process failed its own A/A check. The statistic is the paired per-environment median difference with a percentile bootstrap at $B = 20000$, reported with the fraction of environments improved.

The median log-ratio is not used as the primary statistic here. It disagrees in sign with the marginal medians in several cells, because with an even sample and a two-lobed paired distribution median$(x - y)$ and median(x/y) need not fall on the same side of their null

values. The difference and the win fraction are the honest summaries of an effect this heterogeneous.

Three payloads are scored, at 3.0×, 4.0× and 5.0× nominal cutter mass, and three independently trained context-column seeds. The seeds share one frozen base, so they test the adaptation mechanism's sensitivity to initialisation, not the whole controller's, and are described that way throughout.

7.5.2 Resolution of the Instrument

Before reading any effect, the instrument is measured against itself. Two arms are run at 4.0×, both the frozen policy, differing only in the fifth significant figure of the payload, a 0.0025 % mass change that is physically negligible but forces a different random draw. Any difference reported is noise, and it bounds what this campaign may call an effect.

Table 7.7: A/A control at 4.0× payload, mesh 3141, paired per environment. Both arms are the same frozen policy.

Channel	Median difference	95 % CI	Verdict
cut (mm)	−0.65	[−9.67, +6.96]	not significant
lateral (mm)	−5.14	[−35.08, +13.41]	not significant
contact	−0.01	[−0.04, +0.02]	not significant

No channel of Table 7.7 separates, which is the required outcome. The pooled effects reported below sit well outside these bounds: at 4.0× the lower end of the cut interval is +45.34 mm against an A/A upper bound of +6.96 mm, lateral +46.12 mm against +13.41 mm, and contact +0.09 against +0.02. The median effects are between five and ten times the noise the instrument generates on its own.

One detail of this table justifies the choice of statistic. The two arms' marginal medians differ substantially on cut, 134.83 against 153.71 mm, while their paired difference is −0.65 mm. Marginal medians move under resampling in a way the paired difference does not, which is why the paired difference is the primary statistic throughout.

A caveat travels with it. This floor is measured at 4.0×, where the frozen controller is failing and its variance is correspondingly large. It therefore bounds the claims made at 4.0× and 5.0× but does not bound the smaller lateral regression reported at 3.0×, which would need its own A/A on a stable plant.

7.5.3 The Frozen Controller Has a Payload-Dependent Deficit

The comparison is only meaningful where the frozen policy is actually failing. Table 7.8 establishes that it is, and where.

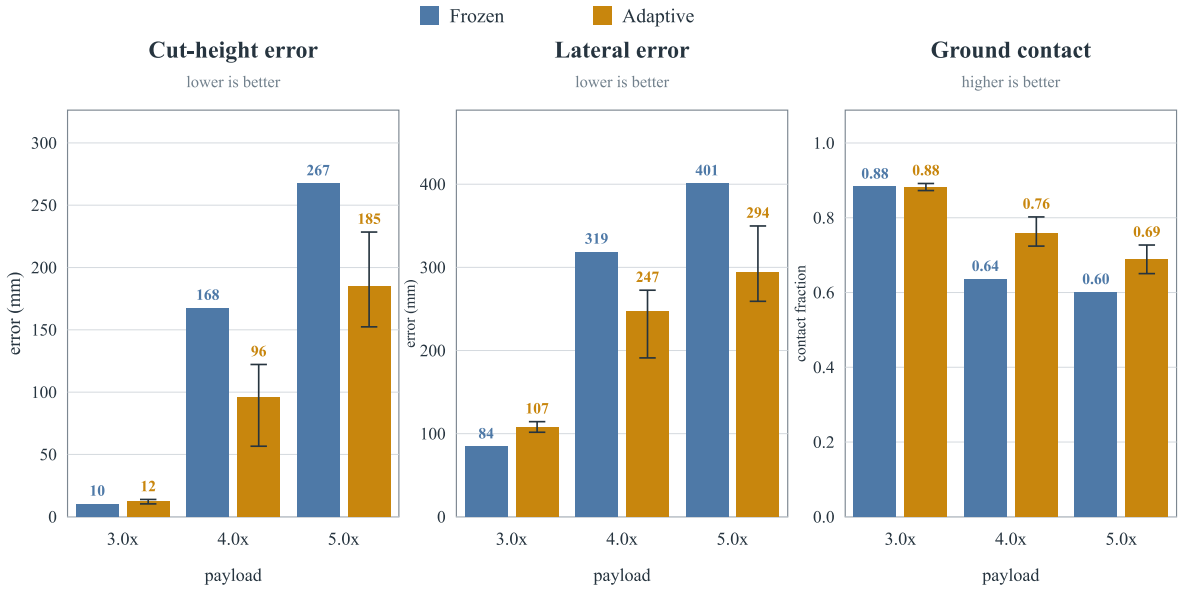

Figure 7.6: Frozen and adaptive performance on held-out mesh 3141. Each payload group places the frozen baseline beside the adaptive result; values are shown directly above the bars. Adaptive bars apply the across-seed pooled paired median change to the frozen baseline, and their whiskers show the corresponding 95 % percentile-bootstrap interval over $B = 20000$. Lower is better for the two error panels and higher is better for ground contact. Adaptation costs at 3.0×, but at 4.0× and 5.0× it improves all three metrics.

Table 7.8: The frozen policy across payload, held-out mesh 3141. The collapse between 3.0× and 4.0× is the robustness deficit the adaptive arm is asked to repair.

Payload	Cut (mm)	Lateral (mm)	Ground contact	Safety violations
3.0×	10.37	84.37	0.883	0.096
4.0×	167.52	318.63	0.636	0.281
5.0×	267.44	400.89	0.601	0.303

At 3.0× the controller still holds the ground for 88 % of the window and has little to give back. Beyond it, cut-height error rises by more than an order of magnitude and ground contact falls away.

7.5.4 Adaptation Against the Frozen Policy

Figure 7.6 shows every cell and Table 7.9 gives the pooled estimates.

Online payload conditioning improves cut height, lateral tracking and ground contact together at 4.0× and 5.0×, every channel with its interval excluding zero. Pooled cut-height error falls by 71.5 mm against a frozen baseline of 167.5 mm and by 82.2 mm against 267.4 mm, a reduction of 43 % and 31 % of the frozen error, while ground contact rises by 0.12 and 0.09. Both task channels improve at once, which a single trade-off along the cut–lateral curve cannot produce.

Table 7.9: Pooled paired median difference, adaptive against frozen, across the three seeds. Positive is better. Intervals are 95 % percentile bootstrap.

Payload	Channel	Frozen	Median difference	95 % CI
3.0×	cut (mm)	10.37	−2.03	[−3.51, +0.04]
	lateral (mm)	84.37	−22.75	[−30.04, −17.27]
	contact	0.883	−0.00	[−0.01, +0.01]
4.0×	cut (mm)	167.52	+71.52	[+45.34, +110.88]
	lateral (mm)	318.63	+71.58	[+46.12, +127.55]
	contact	0.636	+0.12	[+0.09, +0.17]
5.0×	cut (mm)	267.44	+82.18	[+39.00, +115.03]
	lateral (mm)	400.89	+107.05	[+51.11, +141.68]
	contact	0.601	+0.09	[+0.05, +0.13]

The mechanism is regime-selective, and the sweep resolves the regime exactly. At 3.0× the frozen controller still holds the ground and conditioning costs 22.8 mm of lateral accuracy; from 4.0× it gains on every channel. The discriminating variable is ground contact.

Nothing in the mechanism confines it to the loaded side. No deadband or gate is applied in any run reported here (Section 4.6): the estimate is non-zero and the conditioning is active at 3.0× as much as at 5.0×, and the sign of its effect turns because the frozen controller has a deficit at one payload and not at the other. A deployed machine would therefore have to choose when to enable the conditioning rather than rely on it switching itself off, and the evidence here does not locate that boundary more finely than between 3 and 4×.

7.5.5 Repetition on a Second Held-Out Mesh

Mesh 8191 was reserved and has not been used to design, diagnose or read this campaign. It is harder terrain than 3141: the frozen policy there reads 221.8 mm cut and 309.2 mm lateral at 4.0×, and 394.5 mm and 505.2 mm at 5.0×. Table 7.10 repeats the comparison on it with the same statistic.

Table 7.10: Independent repetition on reserved mesh 8191. Pooled paired median difference across the three seeds; positive is better, intervals are 95 % percentile bootstrap.

Payload	Channel	Frozen	Median difference	95 % CI
4.0×	cut (mm)	221.79	+96.80	[+64.68, +143.23]
	lateral (mm)	309.16	+109.70	[+86.04, +145.22]
	contact	0.591	+0.15	[+0.12, +0.19]
5.0×	cut (mm)	394.53	+169.40	[+107.29, +214.10]
	lateral (mm)	505.16	+240.05	[+202.92, +284.83]
	contact	0.474	+0.17	[+0.13, +0.20]

Every pooled effect repeats with its interval excluding zero, and every one is larger than on 3141, which follows from the larger deficit available on harder terrain. Lateral error

improves in all six seed-by-payload cells individually, each with its own interval excluding zero.

That last point qualifies the seed dependence discussed next. Seed 23's lateral regression on mesh 3141 does not repeat here: it improves lateral by 89.5 mm at 4.0× and 182.6 mm at 5.0×, both significant. Its shortfall on the vertical channels does repeat, neutral on cut at 4.0× and −56.2 mm at 5.0×. The seed-dependence is therefore specific to cut height and ground contact, and the lateral improvement holds across both meshes and all three seeds.

7.5.6 Seed Dependence and Its Source

On mesh 3141, seeds 17 and 101 carry the pooled effect on the vertical channels, improving cut height by 187.6 mm and 110.1 mm at 5.0×. Seed 23 does not: on the same mesh it is neutral on cut at 4.0× and −46.68 mm at 5.0×, the −56.2 mm of the previous section being the corresponding figure on 8191. Contact follows the same pattern on both meshes. Its lateral behaviour is not a seed effect, since it improves lateral significantly at both payloads on mesh 8191 (Table 7.10).

The cause is locatable, not mysterious. At 4.0× all three oracles sit near 750 mm of lateral error, at 772, 747 and 760 mm, so every seed has the same trade-off curve available, and what differs is where the online estimate places the policy on it at that payload: 113 mm for seed 17, 150 mm for seed 101, 368 mm for seed 23. The variation is in the fitted estimator, not in the policy it serves, which makes it a tractable target for future work rather than a property of the mechanism.

7.5.7 The True Payload Is Not an Upper Bound

The privileged arm can be worse than the deployable one, and the failure is specific, not general. At 5.0×, seed 101's oracle reads 652.29 mm of cut error at a ground contact of 0.220, while its adaptive arm on the same checkpoint reads 40.46 mm at 0.803. That is the one cell in which the oracle collapses. In the other eight seed-by-payload cells the oracle holds lower cut-height error and higher ground contact than the estimate, as a privileged arm should.

The claim is therefore narrow. On lateral error the deployable arm is ahead of the oracle in all nine cells, because the oracle buys its vertical accuracy by flying the tool. On cut height and contact the oracle is ahead everywhere except seed 101 at 5.0×. What the result refutes is the assumption that the true value is an upper bound on what conditioning can achieve, not the weaker claim that privileged information usually helps.

The context enters as $\boldsymbol{W}_{\text{ctx}}\hat{\boldsymbol{c}}$ with the payload channel reaching 8 at 5.0×, and seed 101 carries the largest context weight of the three. The largest weight meeting the largest input over-drives the policy off the ground. The norm does not order the trade-off at 3.0×, where the smallest-weight seed sits in the middle, so this is a statement about the extreme, not a monotone law.

The deployable arm escapes it because the estimate is damped: it starts each episode at nominal and arrives through an exponential moving average, so it never reaches the region the true value walks straight into. The gradual arrival is part of the mechanism, not an implementation detail.

7.5.8 Correct Payload Against Wrong Payload

The remaining question is whether the benefit comes from the payload *information* or merely from the presence of a plausible context. The control permutes the true payload across environments on a mixed-payload plant, which preserves its distribution and magnitude exactly and destroys only its pairing with the environment it describes. The permutation is redrawn every step and is a derangement, so no environment keeps its own value. At a fixed payload this control would be vacuous, because every environment shares one mass and a permutation is the identity; the mixed plant is what gives it content.

Two conditions differ from the fixed-payload tables above. Cutter mass is resampled per episode over 1.0 to 5.0×, not held at one value, and the arms compared are the true payload and a permuted one. The online estimate is not among them, so this test is made on the privileged channel. The run conditions otherwise match Section 7.5.1: mesh 3141, 176 environments, 900 steps, speed drawn from 2 to 7 km/h, paired per environment.

Table 7.11: Wrong-information control, mixed payload 1.0–5.0× resampled per episode, paired per environment. Positive is better.

Comparison	Cut (mm)	Median difference	Environments better
frozen → true	17.75 → 1.63	+12.66	94.9 %
frozen → permuted	17.75 → 6.43	+11.10	84.7 %
permuted → true	6.43 → 1.63	+3.55	83.0 %

The decisive row of Table 7.11 is the third: the correct payload beats a wrong one drawn from the same distribution by 3.55 mm, interval $[+3.01, +4.14]$, improving 83.0 % of environments, and by 0.03 on ground contact with $[+0.02, +0.03]$. The policy therefore responds to *which* load it is given, not only to being given a plausible one. That is shown for the policy supplied the true value. Whether the online estimate is accurate enough to hold the same margin is a separate question, which this table does not answer and Tables 7.9 and 7.10 answer only through the closed-loop effect.

The same table sizes the two components. A permuted payload recovers 11.10 of the 12.66 mm median gain, so conditioning on a plausible load supplies most of the effect and correct identification supplies the remainder. Both are real and the experiment separates them, which is more than a single aggregate number would have shown.

7.5.9 Scope of the Claims

The smoothing rate of the estimate is a tuned hyperparameter, and its effect is not neutral: the gradual arrival of the estimate is part of the mechanism rather than an implementation detail, to the point that supplying the true payload immediately is not an upper bound on it in the one cell where the privileged arm collapses (Section 7.5.7). The adaptation claim is made inside AP, on the identified forward model: frozen base, online estimate, and true payload. The sections above separate architecture-level evidence that the conditioning can reach the plant from method-level evidence about whether the improvement replicates across training seeds, and they locate the payload regime in which it holds rather than assuming it.

7.6 Conditions for Effective Online Adaptation

Three conditions govern whether online adaptation adds value beyond a robust learned controller, and the experiments above separate them.

A control-relevant deficit must remain. This is the condition the payload sweep isolates most cleanly. The same mechanism, on the same checkpoints, improves every channel at 4.0× and costs lateral accuracy at 3.0×, and the discriminating variable is whether the frozen controller is losing ground contact. Adaptation is not uniformly good; it is good where robustness has run out.

The context must affect a useful command. AP's payload input changes the direct-valve policy while its base weights and FD remain fixed. The paired improvement establishes a closed-loop effect for this conditioning path; it does not establish that every possible adaptation interface would behave the same way.

The estimate must be delivered at a rate the plant tolerates. The oracle result shows that correct information supplied instantaneously can be worse than an approximation supplied gradually. It does so in one cell of nine, at the largest load, where the true value over-drives the policy off the ground and the damped estimate does not (Section 7.5.7). Damping is therefore part of the mechanism, not a margin added afterwards, but the oracle is not generally the weaker arm.

The result is that a controller estimating its own payload from signals it already carries improves cut height, lateral tracking and ground contact together, in the loaded regime where a robust frozen controller is failing. The conditions above say where it works, and the controls separate what correct identification contributes from what conditioning on any plausible load contributes.

The improvement is large and still not sufficient. On mesh 3141 the adapted cut height is about 96 mm at 4.0× and 185 mm at 5.0×, against a cut-height gate of 40 mm (Table 3.5). Adaptation recovers a third to a half of the frozen controller's deficit at those payloads; it does not return the machine to specification, and no claim that it does is made here.

8. Conclusion and Future Work

This thesis asked how hydraulic actuator dynamics learned from high-fidelity simulation can be reduced, transferred into a computationally efficient robotic simulation and used for reinforcement-learning control, and under what conditions online adaptation adds a further increment to the robustness those learned models already provide. The six research questions of Section 1.4 are answered below, grouped by the finding that answers them and ordered by the dependence among those findings.

8.1 Summary of Findings

The findings are stated below, with the measurements left in Chapter 7, not repeated here. Figure 8.1 makes the dependency between the experiments explicit.

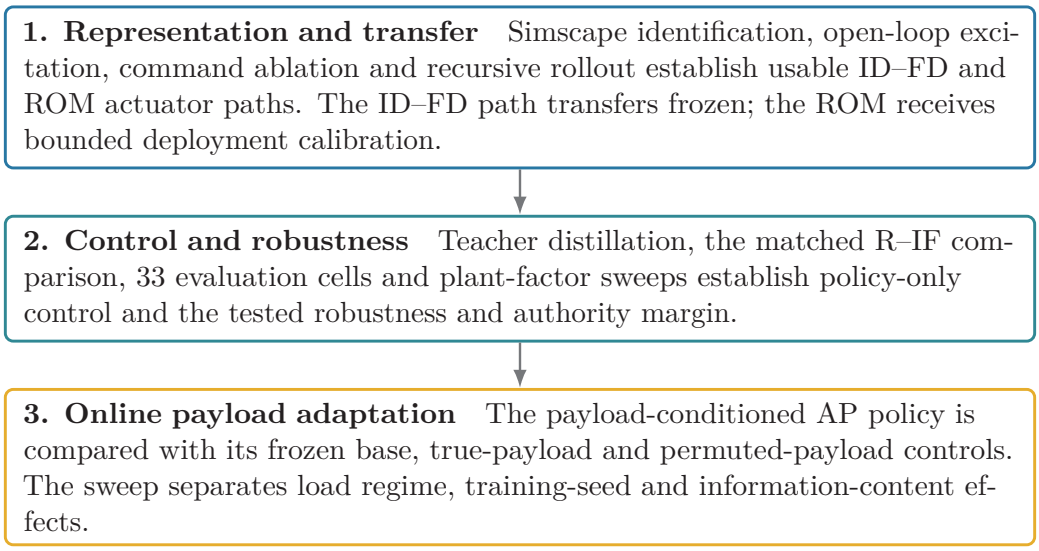

Figure 8.1: Experimental evidence chain used to answer the research questions. Each stage constrains the claim made at the next stage; the final adaptation claim is interpreted only after transfer and robustness have been established.

The inverse and forward representation identified in Simscape transfers to the deployment simulation without retraining (RQ1, RQ2). Trained on Simscape data alone and evaluated on all 118 IsaacLab deployment runs, outside that training distribution, the inverse model holds millimetre-scale teacher-forced one-step extension error and is nearly flat across domains. The claim holds only in that one-step regime: closing the loop on its own predictions diverges in every run (Section 4.2.3). The reduced-order model received a small bounded calibration against the deployment plant, and that asymmetry is carried through the comparison.

Predictive accuracy must be accompanied by a control-authority test (RQ1). A learned actuator must be tested for dependence on the command, not only for prediction error. Under the accepted excitation design, removing the valve input raises rollout error by a factor of 1.81; under the earlier design it lowered it. This is a general validation requirement for learned models used in control.

Two actuator representations both support closed-loop control, and they do not produce the same controller (RQ3, RQ5). Both are accurate enough to train against and cheap enough to sit inside GPU-parallel training, and the inverse–forward chain composes two separately learned models in series without compounding their errors. They induce different strategies from the same policy interface: R spends the least command and reverses far more often, while IF converts greater actuator effort into lower lateral error. Over the 33 paired cells the median favours IF on lateral error and R on cut height, neither mean difference is resolved, and the mean changes sign if one trajectory family is removed (Section 7.2). The choice of representation affects the resulting controller and is not an implementation detail, but the size of that effect is not established. With one training run per stack, their seed variance remains unknown.

A learned policy can replace the classical controller at runtime (RQ4). The damped-least-squares controller supplies privileged demonstrations during training and is then removed. The distilled policy emits desired cylinder motion directly into the learned actuator and is the entire deployed controller, holding the task across the evaluation cells at high ground contact (Section 7.2).

The robust stacks retain a broad margin against mechanical variation and authority loss without online adaptation (RQ2, RQ5). Across held-out meshes, eleven trajectory families and sixteen reserved factor configurations, IF meets the lateral criterion on the median cell and misses it only under worst-mesh, worst-family aggregation, and reducing realised actuator motion to one per cent of nominal triggers neither the safety nor the ground-contact gate. The sweep supports the tested mechanical variation rather than unbounded hydraulic variation, so what it establishes is an authority margin and not a robustness boundary (Section 7.4.1).

A controller that estimates its own payload online improves task tracking where robustness runs out (RQ6). Conditioning the frozen policy on a payload regressed at runtime from the observation it already carries improves cut height, lateral tracking and ground contact together at 4.0× and 5.0× nominal cutter mass, paired per environment on two held-out meshes, every channel with its bootstrap interval excluding zero. The effect is regime-selective: at 3.0×, where the frozen controller still holds the ground, the same conditioning costs lateral accuracy. Two of three training seeds carry the pooled effect on the vertical channels. The true payload beats a payload permuted across environments, so

the policy responds to which load it is given, not to being given one, though that control runs on the privileged channel, not on the estimate.

The gain is partial. Adapted cut-height error at the payloads where adaptation helps remains above the mowing gate, so it improves loaded performance without restoring specification. One result was not anticipated: supplying the true payload is not an upper bound, since on one seed at 5.0× it drives the tool off the ground while the damped online estimate on the same checkpoint tracks well. In the remaining seed-by-payload cells the privileged arm is stronger, so this bounds the assumption, not reversing it. The gradual arrival of the estimate is part of the mechanism.

The adaptation claim rests on paired comparisons within AP, three context-column training seeds and repetition on a reserved mesh. An A/A comparison of the identical frozen policy resolves no difference, while the reported adaptation effects exceed that measurement floor. The R–IF comparison uses separate policy weights and does not establish unchanged-policy transfer between actuator models.

8.2 Future Work

The evidence in this thesis is simulation-to-simulation. Every result rests on a plant identified from Simscape and deployed in PhysX, so the work that matters next is transfer to the machine and the validation that transfer requires.

Instrument the FME 500 and identify it from measured data. Hardware evaluation begins by adding chamber-pressure and extension sensing and a rod load pin. The open-loop excitation schedule of Section 6.4 and the excitation audit of Section 6.10 then apply to machine data, and the command-ablation test of Section 4.3.2 is repeated on the model fitted to it. An actuator model that depends on the valve command in Simscape has not been shown to depend on it on the machine. Any correction fitted to measured data should stay bounded, retain the identified hydraulic parameters and the original data for replay, and pass ablation before it is trusted in the loop.

Drive the arm through force, not through a cylinder target. Every result here reaches PhysX as a position target, so predicted pressure affects the arm only through the learned transition, not through an explicit force interface. The force interface was closed off, not rejected, by the failed force channel and the degenerate force balance, and reopening it needs a rod load pin, identified constants that stay off their clamps, and a load term not double-counted against PhysX contact.

Measure what the identification leaves out. Hose compliance, fluid temperature, supply-pressure sag when several actuators draw at once and valve dead-band drift with wear are all fixed or absent in the deployment plant. Recording them on the machine and comparing them against the factor ranges in Table A.2 would show whether the authority margin reported here covers the real operating envelope or only part of it. This is the measurement that decides how much of the robustness claim survives contact with hardware.

Fine-tune the learned models and the policy on machine data. Neither actuator representation has to be relearned from scratch. The inverse, forward and reduced-order models are trained on Simscape and can be fine-tuned on the measured records, keeping the identified structure and the clamped constants of Section 4.4 and bounding the update

so that it stays a correction, not a refit. Command ablation and recursive rollout are re-run after each update, since a model fine-tuned on closed-loop machine data is exactly the case in which command dependence can be lost again. The policy then follows the route it already took in simulation: the distilled, reference-anchored checkpoint is fine-tuned in a simulator rebuilt on the machine-identified actuators, anchored to the verified checkpoint so the update cannot drift away from behaviour that has already been accepted, and the payload context column is refitted against the measured load. Repeating identification, fine-tuning and evaluation on each new batch of machine data makes the transfer iterative, not a single crossing.

Validate the payload estimate against a measured load. The adaptation result uses a cutter-mass multiplier as its ground truth, applied uniformly for a whole episode. A rod load pin supplies a measured load instead, and real cut material loads the head unevenly and continuously, not by a fixed factor. The test is whether the online estimate tracks a measured load under that excitation, and whether the regime boundary found between 3 and 4× nominal has a counterpart on the machine.

Widen the validation campaign. The 33 evaluation cells cover held-out meshes, eleven trajectory families and sixteen reserved factor configurations, but they run at one forward speed on synthetic verge geometry for a fixed episode length. Recorded verge profiles, varied vehicle speed and longer continuous runs would test whether the tracking and contact margins hold outside the conditions in which they were measured. Hydraulic factors are not applied at all on the inverse and forward path; extending the sweep to that path, and past the identification range on R, would convert the reported authority margin into a robustness boundary.

Then closed-loop operation on the machine. The reference-anchored policy provides a verified starting checkpoint, and the eight promotion gates of Section 3.11.1 carry over unchanged as the acceptance criteria for a hardware trial. Field validation on real verges, with an operator able to intervene, is the point at which the controller is judged on the task it was built for.

The central design lesson is conditional. Robust training absorbs a large part of the plant variation before online adaptation is introduced. A remaining correction is useful only when the variation is observable, the fixed controller leaves a deficit to recover, and the command path retains authority. Apparent adaptation gains must also be separated from gains available by conditioning on any plausible load. Evaluating these conditions together provides a defensible basis for deciding when online adaptation is justified on a hydraulic manipulator.

A. Reference Data and Reproducibility Record

This appendix retains only the reference numbers, digests and entry points that the main chapters cannot carry.

A.1 Plant Ranges and Frozen Collections

Table A.1: Randomised parameters of the Simscape curriculum. Seal wear is drawn per cylinder and drives clearance, leakage and spool lag together; mechanical perturbations apply to each of the fourteen bodies.

Parameter	Nominal	Randomised range
Entrained air fraction $\varepsilon_{\mathrm{air}}$	0.005	0.001 – 0.080
Dead-volume fraction $\varepsilon_{\mathrm{vol}}$	0.020	0.005 – 0.080
Discharge coefficient c_d	0.70	0.56 – 0.84
Seal wear $w_{\mathrm{seal},i}$	0	0 – 0.25 per cylinder
Relief setting p_{set}	27.6 MPa	24.8 MPa – 30.4 MPa
Body mass	CAD value	±20 %
Centre-of-mass offset	CAD value	±5 %
Inertia tensor	CAD value	consistent with mass draw
Joint stiffness, J1–J4	60 kN/m	40 – 120 kN/m
Joint stiffness, J5	25 kN/m	15 – 60 kN/m
Joint damping	250	100 – 600

The task-controller collection contains 118 runs with one fixed factor vector per run: 64 with all six factors varied, 40 hydraulic-only, 12 mechanical-only and 2 nominal, split into 82 fitting, 20 validation and 16 held-out test runs. Splitting is at run level, since the factor vector is constant within a run and a window-level split would place near-duplicates of the same plant in both sets; the manifests store the run identifier, factor key and split label.

Table A.2: Six physical factors in the IsaacLab deployment collections, one vector drawn per run and held fixed throughout it.

Coordinate	Range	Domain	Acts on
e_{flow}	$[-0.25, 0.25]$	hydraulic	valve flow gain
$e_{\text{compliance}}$	$[-0.25, 0.10]$	hydraulic	effective chamber compliance
e_{seal}	$[0, 1]$	hydraulic	seal wear, leakage, spool lag
e_{pressure}	$[-0.15, 0.15]$	hydraulic	supply/relief pressure level
e_{mass}	$[0.8, 1.5]$	mechanical	cutter mass scale
e_{COM_z}	$[-0.10, 0.05]$	mechanical	cutter centre-of-mass height

A.2 Dataset Audit Values

The audit protocol is stated in Section 6.10. This section records the measured values it produced.

Strict open-loop Simscape export.

The 0.01 s sample grid is exact to 9.1×10^{-15} s, all three increment checks return a maximum error of exactly zero, no value is non-finite, and 93.7 % of the 25 600 rows over four runs are valid. The zero increment error is expected: the targets are computed from the logged states by the same within-run difference the check recomputes.

Excitation rank.

The instantaneous command matrix has rank four and the lagged regressor built from the command history has rank eighty, so the excitation is persistently exciting to the order the models need. The condition number of the lagged matrix is about 4.6×10^3, workable but not comfortable, which is the price of the stroke coverage the long plateaus buy.

Stroke coverage.

Coverage is 12 % on the head-pivot cylinder, the narrowest of the four channels.

A.3 Timing Contract

Table A.3: Rates of the simulation, control and learned components.

Layer	Rate	Period
PhysX physics substep	120 Hz	8.33 ms
Policy and control step	60 Hz	16.67 ms
AP payload estimate and context	60 Hz	16.67 ms
Inverse, forward and ROM models	20 Hz	50 ms
Simscape closed-loop logging	20 Hz	50 ms
Simscape open-loop excitation	100 Hz	10 ms

The AP estimator reads the current 60 Hz observation. Its exponential moving average introduces smoothing and resets to nominal at each episode boundary.

Table A.4: Frozen artifacts and their SHA-256 digests, truncated to 32 hexadecimal digits. The context-column checkpoints share the frozen AP base and differ only in their 1 024 trained parameters.

Artifact	Role	SHA-256 (truncated)
`best_id_cylinder_control.pt`	inverse model	`8e26c52e82d89efe193e001d0a8b07a6`
`best_fd_cylinder_strict.pt`	forward model	`1f41e44309a8f320430a3382ca88ce5f`
`rom_s6_aligned/model_598.pt`	R policy	`ce7e3ab9649b1193f123f9f91107bbf8`
`idfd_s6_aligned/model_598.pt`	IF policy	`06533b8e8614b0ad982dd8c238a17772`
`distilled_v1/phase1_init_70.pt`	AP frozen base	`641e476cfcd765a8d68a17980d6cbbf2`
`ctxonly17/model_1596.pt`	AP context columns, seed 17	`805da12505d214ca9f40c405cf02e505`
`ctxonly23/model_1596.pt`	AP context columns, seed 23	`79778e16722bbf2ace1b5c0e680a47dd`
`ctxonly101/model_1596.pt`	AP context columns, seed 101	`80b15d39d0b61ed3ae5ac72b248d9329`
`s17_fit.npz`	payload estimator, seed 17	`1b48070e035f05b07f52449ee2a42b83`
`s23_fit.npz`	payload estimator, seed 23	`422a5f498b03426ab2fe26d52d724174`
`s101_fit.npz`	payload estimator, seed 101	`d7abd08b30d0489f8da593f3c5c0541e`

A.4 Frozen Artifacts

A.5 Reproduction Entry Points

The closed-loop results of Chapter 7 are generated by the scripts under `experiments/E1` through `experiments/E6`, each recording its model, meshes, trajectory families, carrier speed and scored horizon in a manifest. AP arms are trained and scored under `PPO_AAM_FD/experiment_scripts`, and the adaptation tables and figures are recomputed from `outputs/final`, `outputs/heldout8191`, `outputs/aa_floor` and `outputs/kappa_loaded`. Model-level results use the frozen collections of Appendix A.1. Each pre-registered comparison has a dated design note under `reports/` fixing the success rule, checkpoint rule, statistic and declared outcomes before its arms were trained; with the digests of Table A.4 these identify the data, models, rules and code behind every reported number.

Bibliography

[Åström 95] K. J. Åström, B. Wittenmark, *Adaptive Control*, 2nd ed., Reading, MA: Addison-Wesley, 1995.

[Ba 16] J. L. Ba, J. R. Kiros, G. E. Hinton, "Layer Normalization", arXiv preprint arXiv:1607.06450, 2016.

[Bengio 15] S. Bengio, O. Vinyals, N. Jaitly, N. Shazeer, "Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28. 2015, pp. 1171–1179.

[Benner 15] P. Benner, S. Gugercin, K. Willcox, "A Survey of Projection-Based Model Reduction Methods for Parametric Dynamical Systems", *SIAM Review*, vol. 57, no. 4, pp. 483–531, 2015.

[Chen 19] D. Chen, B. Zhou, V. Koltun, P. Krähenbühl, "Learning by Cheating", in *Proceedings of the Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 100. 2019, pp. 66–75.

[Cho 14] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, Y. Bengio, "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation", in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014, pp. 1724–1734.

[Chua 18] K. Chua, R. Calandra, R. McAllister, S. Levine, "Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31. 2018.

[Cover 06] T. M. Cover, J. A. Thomas, *Elements of Information Theory*, 2nd ed., Hoboken, NJ: Wiley-Interscience, 2006.

[Efron 93] B. Efron, R. J. Tibshirani, *An Introduction to the Bootstrap*, Boca Raton, FL: Chapman & Hall/CRC, 1993.

[Egli 20] P. Egli, M. Hutter, "Towards RL-Based Hydraulic Excavator Automation", in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020, pp. 2692–2697.

[Egli 22] P. Egli, M. Hutter, "A General Approach for the Automation of Hydraulic Excavator Arms Using Reinforcement Learning", *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 5679–5686, 2022.

[Elfwing 18] S. Elfwing, E. Uchibe, K. Doya, "Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning", *Neural Networks*, vol. 107, pp. 3–11, 2018.

[He 16] K. He, X. Zhang, S. Ren, J. Sun, "Deep Residual Learning for Image Recognition", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778.

[Hochreiter 97] S. Hochreiter, J. Schmidhuber, "Long Short-Term Memory", *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[Hu 26] P. Hu, T. Wen, D. Zhang, H. Chen, J. Gong, "Bayesian reinforcement learning for adaptive control of energy recuperation in hydraulic excavator arms", *Scientific Reports*, vol. 16, p. 6195, 2026.

[Huber 64] P. J. Huber, "Robust Estimation of a Location Parameter", *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.

[Hwangbo 19] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, M. Hutter, "Learning Agile and Dynamic Motor Skills for Legged Robots", *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[Isermann 11] R. Isermann, M. Münchhof, *Identification of Dynamic Systems: An Introduction with Applications*, Berlin, Heidelberg: Springer, 2011.

[Jelali 03] M. Jelali, A. Kroll, *Hydraulic Servo-systems: Modelling, Identification and Control*, London: Springer, 2003.

[Johannink 19] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, S. Levine, "Residual Reinforcement Learning for Robot Control", in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2019, pp. 6023–6029.

[Kakade 02] S. Kakade, J. Langford, "Approximately Optimal Approximate Reinforcement Learning", in *Proceedings of the 19th International Conference on Machine Learning (ICML)*. 2002, pp. 267–274.

[Karniadakis 21] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, "Physics-Informed Machine Learning", *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, 2021.

[Kingma 15] D. P. Kingma, J. Ba, "Adam: A Method for Stochastic Optimization", in *International Conference on Learning Representations (ICLR)*. 2015.

[Kumar 21] A. Kumar, Z. Fu, D. Pathak, J. Malik, "RMA: Rapid Motor Adaptation for Legged Robots", in *Proceedings of Robotics: Science and Systems (RSS)*. 2021.

[Kutner 05] M. H. Kutner, C. J. Nachtsheim, J. Neter, W. Li, *Applied Linear Statistical Models*, 5th ed., New York: McGraw-Hill/Irwin, 2005.

[Lee 20] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, M. Hutter, "Learning Quadrupedal Locomotion over Challenging Terrain", *Science Robotics*, vol. 5, no. 47, p. eabc5986, 2020.

[Liu 25] X. Liu, C. Lu, H. Wu, B. Hu, X. Li, Z. Li, X. Guo, "FRMA: Four-Phase Rapid Motor Adaptation Framework", *Machines*, vol. 13, no. 10, p. 885, 2025.

[Ljung 99] L. Ljung, *System Identification: Theory for the User*, 2nd ed., Upper Saddle River, NJ: Prentice Hall, 1999.

[Lutter 19] M. Lutter, C. Ritter, J. Peters, "Deep Lagrangian Networks: Using Physics as Model Prior for Deep Learning", in *International Conference on Learning Representations (ICLR)*. 2019.

[Maitan 26] L. Maitan, L. Toschi, C. Zanette, E. G. Vergamini, L. F. Santos, T. Boaventura, "Contraction Metric Based Safe Reinforcement Learning Force Control for a Hydraulic Actuator with Real-World Training", arXiv preprint arXiv:2602.08977, 2026.

[Makoviychuk 21] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, G. State, "Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning", arXiv preprint arXiv:2108.10470, 2021.

[Mattila 17] J. Mattila, J. Koivumäki, D. G. Caldwell, C. Semini, "A Survey on Control of Hydraulic Robotic Manipulators With Projection to Future Trends", *IEEE/ASME Transactions on Mechatronics*, vol. 22, no. 2, pp. 669–680, 2017.

[McKay 79] M. D. McKay, R. J. Beckman, W. J. Conover, "A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code", *Technometrics*, vol. 21, no. 2, pp. 239–245, 1979.

[Merritt 67] H. E. Merritt, *Hydraulic Control Systems*, New York: John Wiley & Sons, 1967.

[Miki 22] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, M. Hutter, "Learning robust perceptive locomotion for quadrupedal robots in the wild", *Science Robotics*, vol. 7, no. 62, p. eabk2822, 2022.

[Mittal 23] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, A. Garg, "Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments", *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3740–3747, 2023.

[MULAG Fahrzeugwerk 26] MULAG Fahrzeugwerk, "FME 500 Front Mower — Product Documentation", Heinz Wössner GmbH u. Co. KG, Oppenau, Germany. `https://www.mulag.de/en/roadside-equipment/products/unimog-equipment/front-mower/fme-500/`, 2026. Accessed July 2026.

[Nagabandi 18] A. Nagabandi, G. Kahn, R. S. Fearing, S. Levine, "Neural Network Dynamics for Model-Based Deep Reinforcement Learning with Model-Free Fine-Tuning", in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 7559–7566.

[Nakamura 86] Y. Nakamura, H. Hanafusa, "Inverse Kinematic Solutions With Singularity Robustness for Robot Manipulator Control", *ASME Journal of Dynamic Systems, Measurement, and Control*, vol. 108, no. 3, pp. 163–171, 1986.

[Nguyen-Tuong 11] D. Nguyen-Tuong, J. Peters, "Model Learning for Robot Control: A Survey", *Cognitive Processing*, vol. 12, no. 4, pp. 319–340, 2011.

[Nosek 18] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, D. T. Mellor, "The preregistration revolution", *Proceedings of the National Academy of Sciences*, vol. 115, no. 11, pp. 2600–2606, 2018.

[Paszke 19] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32. 2019, pp. 8024–8035.

[Peng 18] X. B. Peng, M. Andrychowicz, W. Zaremba, P. Abbeel, "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization", in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 3803–3810.

[Piaggio 12] G. Piaggio, D. R. Elbourne, S. J. Pocock, S. J. W. Evans, D. G. Altman, "Reporting of noninferiority and equivalence randomized trials: extension of the CONSORT 2010 statement", *JAMA*, vol. 308, no. 24, pp. 2594–2604, 2012.

[Rajeswaran 18] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, S. Levine, "Learning Complex Dexterous Manipulation with Deep Reinforcement Learning and Demonstrations", in *Proceedings of Robotics: Science and Systems (RSS)*. 2018.

[Rockafellar 00] R. T. Rockafellar, S. Uryasev, "Optimization of conditional value-at-risk", *Journal of Risk*, vol. 2, no. 3, pp. 21–41, 2000.

[Ross 11] S. Ross, G. J. Gordon, J. A. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning", in *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*. 2011, pp. 627–635.

[Rudin 22] N. Rudin, D. Hoeller, P. Reist, M. Hutter, "Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning", in *Proceedings of the Conference on Robot Learning (CoRL)*. 2022, pp. 91–100.

[Schoukens 19] J. Schoukens, L. Ljung, "Nonlinear System Identification: A User-Oriented Road Map", *IEEE Control Systems Magazine*, vol. 39, no. 6, pp. 28–99, 2019.

[Schulman 15] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, P. Abbeel, "Trust Region Policy Optimization", in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 37. 2015, pp. 1889–1897.

[Schulman 16] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation", in *International Conference on Learning Representations (ICLR)*. 2016.

[Schulman 17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, "Proximal Policy Optimization Algorithms", arXiv preprint arXiv:1707.06347, 2017.

[Siciliano 09] B. Siciliano, L. Sciavicco, L. Villani, G. Oriolo, *Robotics: Modelling, Planning and Control*, London: Springer, 2009.

[Silver 18] T. Silver, K. Allen, J. Tenenbaum, L. Kaelbling, "Residual Policy Learning", arXiv preprint arXiv:1812.06298, 2018.

[Slotine 91] J.-J. E. Slotine, W. Li, *Applied Nonlinear Control*, Englewood Cliffs, NJ: Prentice Hall, 1991.

[Sobol' 67] I. M. Sobol', "On the Distribution of Points in a Cube and the Approximate Evaluation of Integrals", *USSR Computational Mathematics and Mathematical Physics*, vol. 7, no. 4, pp. 86–112, 1967.

[Sofrony 24] J. Sofrony, M. C. Turner, C. M. Richards, "Anti-windup compensation for model reference adaptive control schemes", *International Journal of Robust and Nonlinear Control*, vol. 34, no. 15, pp. 10176–10193, 2024.

[Spinelli 24] F. A. Spinelli, P. Egli, J. Nubert, F. Nan, T. Bleumer, P. Goegler, S. Brockes, F. Hofmann, M. Hutter, "Reinforcement Learning Control for Autonomous Hydraulic Material Handling Machines with Underactuated Tools", in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2024. arXiv:2410.05093.

[Srivastava 14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting", *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[Sutton 18] R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., Cambridge, MA: MIT Press, 2018.

[The MathWorks, Inc. 24a] The MathWorks, Inc , "Simscape Fluids: User's Guide", Natick, MA, USA, 2024. `https://www.mathworks.com/products/simscape-fluids.html`.

[The MathWorks, Inc. 24b] The MathWorks, Inc , "Simscape Multibody: User's Guide", Natick, MA, USA, 2024. `https://www.mathworks.com/products/simscape-multibody.html`.

[Tobin 17] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World", in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017, pp. 23–30.

[von Szadkowski 20] K. von Szadkowski, S. Reichel, "Phobos: A Tool for Creating Complex Robot Models", *Journal of Open Source Software*, vol. 5, no. 45, p. 1326, 2020.

[Wampler 86] C. W. Wampler, “Manipulator Inverse Kinematic Solutions Based on Vector Formulations and Damped Least-Squares Methods”, *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 16, no. 1, pp. 93–101, 1986.

[Yamamori 26] S. Yamamori, K. Ishihara, K. Minamikawa, R. Ohmori, T. Yasaki, N. Sugimoto, J. Morimoto, “Actuator Reality Shaping for Zero-Shot Sim-to-Real Robot Learning”, arXiv preprint arXiv:2607.02205, 2026.

[Yang 26] N. Yang, C. Ji, J. Chen, H. Zheng, “State-Constrained Control for Hydraulic Manipulator Position Servo Systems with Valve Dead-Band Compensation”, *Actuators*, vol. 15, no. 4, p. 196, 2026.

[Yao 00] B. Yao, F. Bu, J. Reedy, G. T. C. Chiu, “Adaptive Robust Motion Control of Single-Rod Hydraulic Actuators: Theory and Experiments”, *IEEE/ASME Transactions on Mechatronics*, vol. 5, no. 1, pp. 79–91, 2000.

[Yin 25] P. Yin, T. Westenbroek, S. Bagaria, K. Huang, C.-A. Cheng, A. Kolobov, A. Gupta, “Rapidly Adapting Policies to the Real World via Simulation-Guided Fine-Tuning”, arXiv preprint arXiv:2502.02705, 2025.

[Yu 26] S.-Y. Yu, J.-W. Wang, M. Horii, M. Tomizuka, V. Garg, “Learning Context-Aware Neural ODE Dynamics for Adaptive Robotic Control”, arXiv preprint arXiv:2606.15469, 2026.